OPSA: Efficient and Verifiable One-Pass Secure Aggregation with TEE for Federated Learning

Zhangshuang Guan, Yulin Zhao, Zhiguo Wan, and Jinsong Han

***Abstract*—Federated learning enables collaborative model training while preserving data privacy by keeping data local. To protect user privacy during model aggregation, secure aggregation (SA) protocols are widely adopted to mask models. However, existing SA protocols require at least three round trips per aggregation and lack mechanisms to verify aggregation results. Verifiable SA addresses the verification gap but incurs high communication costs. TEE-based SA minimizes round trips but faces computational bottlenecks due to TEE's limited physical memory, especially when handling larger models or numerous clients. In this work, we introduce OPSA, an efficient and verifiable one-pass SA protocol based on TEE. By handling client dropouts via server-side TEE, OPSA enables the server to aggregate masked models in a single pass, significantly reducing round trips. To mitigate TEE's limitations, OPSA offloads tasks like model aggregation and mask elimination outside TEE, with only shared keys processed within TEE. Building on this design, we propose KhPRF-OPSA (single masking) and POT-OPSA (double masking) protocols, both incorporating novel cryptographic primitives. Furthermore, OPSA integrates commitment and signature mechanisms to ensure result verifiability with only $O(1)$ additional communication overhead per client. Compared to state-of-the-art schemes, OPSA achieves a 2$\sim$10$\times$ speedup in multi-round aggregation while guaranteeing result verification.**

***Index Terms*—Federated learning, secure aggregation, verifiable aggregation, trusted execution environment.**

I. Introduction

FEDERATED learning (FL) represents a paradigm shift in machine learning, replacing traditional centralized model training methods with decentralized collaborative frameworks [1]. In this approach, a global machine learning model is trained on multiple decentralized devices with the assistance of a central server, each possessing its own local dataset. The distinguishing feature of FL is that model updates are computed locally on these devices and uploaded to the server, eliminating the necessity to transfer raw data to a central server. These devices (clients) and the server engage in interactions through the above operations for multiple rounds, ultimately achieving a convergent model. However, the original model update may also lead to the leakage of original private data [2], [3].

Corresponding authors: Zhiguo Wan and Jinsong Han.

Z. Guan and J. Han are with the College of Computer Science and Technology, Zhejiang University, Hangzhou, 310027, China (e-mail: {guanzs, hanjinsong}@zju.edu.cn).

Y. Zhao is with Hangzhou Institute for Advanced Study, University of Chinese Academy of Sciences, Hangzhou 310024, China (e-mail: zhaoyulin22@mails.ucas.ac.cn).

Z. Wan is with the Zhejiang Lab, Hangzhou, 311121, China (e-mail: wanzhiguo@zhejianglab.com).

This work was supported by the National Natural Science Foundation of China under Grant (Nos. U22B2032, 62272425, U21A20462, 62372400).

To protect the privacy of model updates, *secure aggregation* (SA) has been designed for model aggregation in FL. The core idea involves each client masking its local model in a certain manner, while the server aggregates these masked models to recover a plaintext global model. Furthermore, secret sharing schemes are adopted to ensure that the global model remains recoverable even after some clients drop out. The above idea is first proposed by Bonawitz et al. [4][1], upgraded by Bell et al. [5], and subsequently adapted into various variants [6], [7]. However, these schemes are more suitable for single-round FL, as each aggregation requires clients to regenerate and share their keys, significantly increasing communication overhead.

Fortunately, MicroFedML [8] addresses this issue by introducing a one-time setup phase to reuse secret keys, Flamingo [9] designs a multi-round SA protocol using pairwise masks while LERNA [10] achieves the same goal through a single masking mechanism. While these schemes achieve a reusable setup phase by introducing an additional group, they still require three client-server round trips per aggregation (as shown in Fig. 1), which poses challenges in scenarios with high network latency or limited bandwidth. Essentially, the special group, such as decryptors in Flamingo and a committee in LERNA, can be treated as a trusted entity. To this end, we ask the following research question: *Can hardware-based Trusted Execution Environments (TEEs) replace these special groups to enable round-trip-optimal aggregation while maintaining computational efficiency within TEE?*

Executing SA entirely within TEE optimizes communication round trips, but the limited physical memory of TEE poses significant computational bottlenecks, particularly when aggregating larger models or handling numerous clients. For example, a global model containing a million parameters takes about 3.8MB of memory, with each parameter represented in 4 bytes. However, in scenarios involving hundreds of clients, the memory requirements can easily exceed the 128MB physical memory capacity of TEE. To address this issue, SEAR [11] adopts a strategy of aggregating one layer of the model at a time. However, this approach does not alleviate the overall computation burden on TEE, and similar challenges may arise when processing layers with a large number of parameters. Of course, sparse gradient aggregation can reduce TEE memory usage, but Kato et al. [12] have introduced an attack targeting the gradient index within TEE.

Moreover, all the multi-round or TEE-based SA protocols mentioned above lack mechanisms to verify final aggregation results, making them vulnerable to manipulation by a mali-

[1]We write the SecAgg and SecAgg+ term as [4] and [5], respectively.

OPSA：基于TEE技术实现高效、可验证的一次性安全聚合，适用于联邦学习场景

张双关、赵玉林、万志国和韩金松

***摘要*——联邦学习能够在保持数据本地化的同时实现协作式模型训练并保护数据隐私。为在模型聚合过程中保障用户隐私，安全聚合（SA）协议被广泛用于对模型进行遮蔽处理。然而现有SA协议每次聚合至少需要三次往返通信，并缺乏验证聚合结果的机制。可验证SA虽能弥补这一验证缺陷，但会产生高昂的通信成本；基于TEE的SA方案虽可减少往返通信次数，却因TEE物理内存有限（尤其在处理大型模型或大量客户端时）而面临计算瓶颈。本研究提出 OPSA ——一种基于TEE的高效且可验证的一次性SA协议：通过服务器端TEE机制处理客户端退出情况， OPSA 使服务器能在单次运算中完成掩码模型的聚合，显著降低通信次数。为缓解TEE的局限性， OPSA 将模型聚合和掩码消除等任务卸载至TEE外部，仅在TEE内部处理共享密钥。在此基础上，我们进一步提出了KhPRF- OPSA（单层掩码）和POT- OPSA（双层掩码）两种协议，均采用了新型密码学原语。此外， OPSA 整合了承诺机制与签名机制，仅需为每个客户端增加$O(1)$倍的通信开销即可确保结果可验证性。与现有最先进方案相比， OPSA 在多轮聚合过程中实现了2至10*倍*的速度提升，并同时保证结果可验证性。**

***索引术语*—联邦学习、安全聚合、可验证聚合、可信执行环境。**

I. 引言

联邦学习（FL）代表了机器学习领域的一种范式转变，它用去中心化的协作框架取代了传统的集中式模型训练方法[1]。在此方法中，全局机器学习模型在多个去中心化设备上进行训练，并得到中央服务器的辅助支持；每个设备均拥有自己的本地数据集。联邦学习的核心特征在于：模型更新是在这些设备上本地计算后上传至服务器，从而无需将原始数据传输至中央服务器。这些设备（客户端）与服务器通过上述操作进行多轮交互，最终实现模型收敛。然而，初始模型更新也可能导致原始私有数据泄露[2][3]。

通讯作者：万志国、韩金松。

Z. 关和韩俊松任职于浙江大学计算机科学与技术学院，杭州，310027，中国（电子邮箱：{guanzs，hanjinsong}@zju.edu.cn）。

Y. 赵教授任职于中国科学院大学杭州高等研究院，地址：杭州市310024（电子邮箱：zhaoyulin22@mails.ucas.ac.cn）。

Z. 万先生任职于浙江省实验室（杭州，311121，中国）（电子邮箱：wanzhiguo@zhejianglab.com）。

本研究由国家自然科学基金委员会资助，项目编号分别为U22B2032、62272425、U21A20462和62372400。

为保护模型更新的隐私性，针对联邦学习中的模型聚合问题设计了*安全聚合*（SA）方法。其核心思想是要求每个客户端以特定方式对其本地模型进行掩码处理，服务器则对这些经过掩码处理的模型进行聚合，从而恢复出明文形式的全局模型。此外，该方法采用秘密共享方案，确保即使部分客户端退出参与，全局模型仍可被成功恢复。这一思路最初由Bonawitz等人[4][1]提出，后经Bell等人[5]改进，并进一步发展出多种变体[6]，[7]。然而这些方案更适合单轮联邦学习场景，因为每次聚合都需要客户端重新生成并共享密钥，显著增加了通信开销。

幸运的是，MicroFedML[8]通过引入一次性设置阶段来实现密钥复用；Flamingo[9]设计了采用成对掩码的多轮安全协议；而 LERNA[10]则通过单一掩码机制实现了相同目标。尽管这些方案通过引入额外组别实现了可复用的设置阶段，但每次聚合仍需三次客户端-服务器通信（如图1所示），这在高网络延迟或带宽受限场景中存在明显局限。本质上，Flamingo中的解密者组和 LERNA 中的委员会组均可视为可信实体。基于此，我们提出以下研究问题：*基于硬件的可信执行环境（TEE）能否替代这些特殊组别，在保持TEE内部计算效率的同时实现通信次数最优化的聚合？*

在TEE环境中完全执行SEAR操作可优化通信往返过程，但TEE有限的物理内存会带来显著的计算瓶颈，尤其是在聚合大型模型或处理大量客户端时。例如，包含百万个参数的全局模型约需3.8MB内存空间（每个参数占用4字节）。但在涉及数百个客户端的场景中，内存需求很容易超过TEE 128MB的物理内存容量。为解决这一问题，SEAR[11]采用了逐层聚合模型参数的策略。然而该方法并未缓解TEE的整体计算负担，在处理含大量参数的模型层级时仍可能面临类似挑战。虽然稀疏梯度聚合可降低TEE的内存消耗，但Kato等人[12]已提出针对TEE内梯度索引的攻击方案。

此外，上述所有基于多轮或TEE的SA协议均缺乏验证最终聚合结果的机制，因此易受到恶意攻击者的篡改。

[1]我们将SecAgg和SecAgg+项分别记为[4]和[5]。

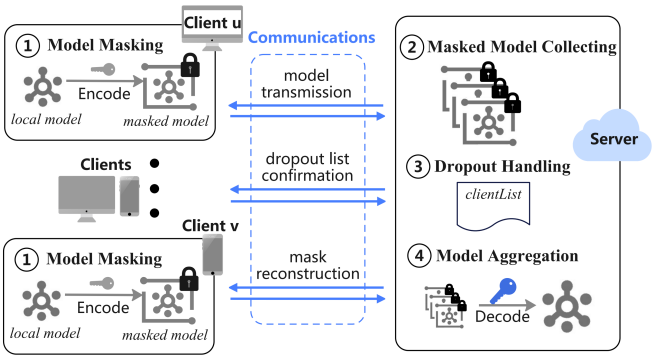


Fig. 1: Workflow of the state-of-the-art SA schemes

cious server (e.g., through inconsistent aggregation), which could lead to privacy attacks [13]–[15]. Although several verifiable SA protocols [16]–[20] have been proposed under stronger security assumptions (e.g., assuming no client-server collusion), they suffer from high communication overhead and require at least four round trips per aggregation.

In this work, we propose OPSA to effectively address these issues. First, we minimize communication round trips by handling client dropouts via server-side TEE. Then, we require TEE to process only shared keys internally, improving its computational efficiency. Finally, we present two multi-round SA protocols based on novel cryptographic primitives specifically designed for OPSA. Moreover, we design a mechanism to verify final aggregation results under a weaker security assumption. Our contributions can be summarized as follows:

- We propose OPSA, an efficient and verifiable multi-round one-pass SA framework. With the help of TEE, the server aggregates masked models in a single pass, significantly reducing communication round trips. Meanwhile, TEE is tasked only with key processing operations, keeping its memory consumption independent of model size, thereby effectively mitigating computational bottlenecks.
- Building on OPSA, we present two multi-round SA protocols: KhPRF-OPSA, a single masking protocol based on key-homomorphic pseudo-random function, and POT-OPSA, a double masking protocol built upon proxy oblivious transfer. Both protocols are designed to efficiently aggregate larger models and handle more clients within the restricted capacity of TEE.
- By incorporating commitment and signature mechanisms, we introduce OPSA-RV, a result verification method that protects against forged aggregation results. OPSA-RV achieves this under the weaker security assumption that a malicious server may collude with any semi-honest client. Compared to SOTA schemes, OPSA-RV incurs only $O(1)$ additional communication overhead per client.
- We provide a rigorous security analysis to establish the theoretical safety guarantees of OPSA. Additionally, we implement the KhPRF-OPSA and POT-OPSA protocols, along with the OPSA-RV method, and compare their performance with SOTA schemes through extensive experiments. The results demonstrate that our design excels in terms of dropout resilience and efficiency.

II. Preliminaries

Before introducing the preliminaries, we list the notations used in this work.

Notations. Unless specified otherwise, we use λ to denote the security parameter, lowercase letters (e.g., n, k, r) to denote scalars, and bold lowercase letters (e.g., $\mathbf{x}, \mathbf{y}, \mathbf{k}$) to denote vectors. The format $\{s_u\}_u$ denotes a set of values indexed by u. The variable $\mathbf{x}_u^{(r)}$ denotes the input vector $\mathbf{x}$ held by client u in round r. The sets $\mathcal{U}, \mathcal{D}, \mathcal{V}$, where $\mathcal{U} = \mathcal{D} \cup \mathcal{V}$, contain all clients, dropped clients, and surviving clients, respectively. Note that we use the terms "user" and "client" interchangeably.

A. Federated Learning

A general federated learning (FL) scenario involves n clients and a server, all participants repeat the following processes for each round r until the current global model $\boldsymbol{\theta}^{(r)}$ converges:

- Local training: each client $u \in \mathcal{U}$ receives the global model $\boldsymbol{\theta}^{(r-1)}$, which is updated in the previous round, and combines it with its private dataset to train a local model $\mathbf{x}_u^{(r)}$ to be sent to the server.
- Model aggregation: the server collects the individual model $\mathbf{x}_u^{(r)}$ from distinct clients and computes an aggregated model $\mathbf{z}^{(r)} = \sum_{u \in \mathcal{V}} \mathbf{x}_u^{(r)}$ to update the global model $\boldsymbol{\theta}^{(r)}$ to be delivered to clients for next round.

Note that these clients can drop out at any steps in each FL training round.

B. Trusted Execution Environment

Trusted Execution Environment (TEE), whose notable implementations include Intel Software Guard Extensions (SGX) and ARM TrustZone, is a hardware solution designed to ensure computational integrity and confidentiality by creating a secure and isolated space, often referred to as an enclave, where the data and its sensitive computations can be performed without the risk of interference or compromise. In particular, TEE allows remote users to verify the application and the hardware it is running via remote attestation.

Given the widespread use of Intel SGX in personal computers and cloud computing servers, we leverage it for secure aggregation in FL. However, Intel SGX presents the following challenges in achieving efficient SA protocols: (1) limited physical memory, Intel SGX v1 only has 128MB physical memory, exceeding its limit results in high overhead; (2) data transmission overhead, the SGX application is divided into trusted and untrusted parts, transmitting data between the two parts will incur significant overhead due to additional encryption/decryption operations; (3) side-channel attacks, which are an important factor to be considered, although they are not part of the Intel SGX threat model [21].

Similar to SEAR [11], we assume that Intel SGX is trusted and do not consider side-channel information leakage. In our approach, the enclave negotiates a shared key $k_{e,u}$ with each client $u \in \mathcal{U}$. Through remote attestation, client u can verify that the intended code is securely running within the enclave on a fully patched, Intel SGX-enabled platform. Notably, side-channel attacks, including physical attacks, do

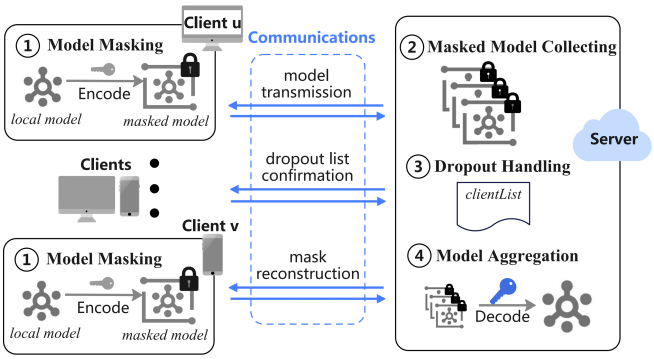


图 1：最先进的SA方案的工作流程

这种服务器行为（例如通过不一致的聚合操作）可能导致隐私攻击[13]–[15]。尽管已在更强的安全假设下（例如假设不存在客户端与服务器之间的串通）提出了多种可验证的SA协议[16]–[20]，但这些协议存在较高的通信开销，并且每次聚合至少需要四次往返通信。

在本研究中，我们提出了 OPSA 方案以有效解决上述问题。首先，通过服务器端TEE机制处理客户端退出情况，从而最大限度减少通信往返次数；其次，要求TEE仅在内部处理共享密钥，显著提升其计算效率；最后，我们提出两种基于专为 OPSA 设计的新型密码学原语的多轮安全聚合协议，并设计了一种在较弱安全假设下验证最终聚合结果的机制。我们的贡献可归纳如下：

- 我们提出 OPSA ——一种高效且可验证的多轮单次执行随机算法框架。借助TEE技术，服务器可在单次运算中聚合经过掩码处理的模型，显著减少通信往返次数；同时TEE仅负责关键处理操作，其内存消耗与模型规模无关，从而有效缓解计算瓶颈问题。

- 基于 OPSA 的研究成果，我们提出两种多轮安全认证协议：KhPRF- OPSA ——一种基于密钥同态伪随机函数的单层掩码协议；以及potopsa——一种基于代理感知传输机制的双层掩码协议。这两种协议均旨在高效聚合大型模型，并在TEE的有限资源容量下支持更多客户端接入。

- 通过引入承诺机制与签名机制，我们提出了 OPSA -RV——一种能防范伪造聚合结果的验证方法。该方案在较弱的安全假设下实现这一目标：即恶意服务器可与任何半诚实客户端串通。相较于 SOTA 方案，OPSA -RV每客户端仅产生$O(1)$量的额外通信开销。

- 我们通过严谨的安全性分析，确立了 OPSA 方案的理论安全性保障。此外，我们实现了KhPRF- OPSA 和 POT- OPSA 协议以及 OPSA -RV方法，并通过大量实验将其性能与 SOTA 方案进行对比。结果表明，我们的设计方案在抗丢包能力和效率方面均表现卓越。

第二部分。前言

在介绍绪论部分之前，我们先列出本研究中使用的符号体系。

符号说明。除非另有说明，我们使用λ表示安全参数，小写字母（如n、k、r）表示标量，粗体小写字母（如$\mathbf{x}$、$\mathbf{y}$、$\mathbf{k}$）表示向量。格式$\{s_u\}_u$表示由u索引的一组值。$\mathbf{x}_u^{(r)}$变量表示客户端u在轮次r中持有的输入向量$\mathbf{x}$。U、D、V集合（其中$U=D \cup V$）分别包含所有客户端、已退出客户端及存活客户端。需注意，“用户”与“客户端”这两个术语可互换使用。

A. 联邦式学习

一个典型的联邦学习（FL）场景包含n个客户端和一台服务器，所有参与者在每轮r中重复执行以下流程，直至当前全局模型$\boldsymbol{\theta}^{(r)}$收敛：

- 本地训练：每个客户端$u \in U$接收全局模型$\boldsymbol{\theta}^{(r-1)}$（该模型已在上一轮更新），并将其与自身的私有数据集结合，训练出本地模型$\mathbf{x}_u^{(r)}$并发送至服务器。
- 模型聚合：服务器从不同客户端收集各个模型$\mathbf{x}_u^{(r)}$，并计算聚合模型$\mathbf{z}^{(r)}=\sum_{u \in \mathcal{V}} \mathbf{x}_u^{(r)}$，以更新全局模型$\boldsymbol{\theta}^{(r)}$，供下一轮交付给客户端。

需注意，这些受试者可在每轮FL训练的任何阶段退出。

B. 可信执行环境

可信执行环境（TEE）是一种硬件解决方案，其典型实现包括英特尔软件保护扩展（SGX）和ARM信任区。该环境通过创建一个安全且隔离的空间（通常称为“飞地”），确保计算过程的完整性和机密性，使数据及其敏感运算能够在不受干扰或泄露风险的情况下完成。具体而言，TEE支持远程用户通过远程验证机制来检测应用程序及其运行所用硬件。

鉴于英特尔 SGX 芯片在个人计算机和云计算服务器中的广泛应用，我们将其应用于联邦学习（FL）中的安全数据聚合。然而，英特尔 SGX 在实现高效的安全聚合协议时存在以下挑战：(1)物理内存有限——英特尔 SGX v1版仅配备128MB物理内存，超出该限制会带来显著开销；(2)数据传输开销—— SGX 应用被划分为可信部分与不可信部分，两部分之间的数据传输因需额外进行加密/解密操作而产生巨大开销；(3)侧信道攻击——这是需要重点考虑的因素，尽管其并不属于英特尔 SGX 威胁模型的范畴[21]。

与SEAR[11]类似，我们假设Intel SGX 是可信的，并不考虑侧信道信息泄露问题。在我们的方案中，飞地会与每个客户端$u \in U$协商共享密钥$k_{e,u}$。通过远程验证机制，客户端u可确认目标代码确实在已完全打补丁且支持Intel SGX 的平台上安全运行于飞地内。值得注意的是，包括物理攻击在内

not reveal any new or unexpected vulnerabilities in the Intel SGX architecture [11]. Recent work [22] shows that SGX attacks can often be mitigated through constant-time code by developers or microcode updates from Intel.

C. Cryptographic Primitives

In our protocols, we will employ the following cryptographic primitives for randomness generation and secure communication. A cryptographically secure pseudo-random generator (PRG) takes a uniformly random seed of some fixed length and outputs a string that is computationally indistinguishable from a random string; for example, it can be instantiated with AES-CTR in practice [4]. A cryptographically secure pseudo-random function (PRF) indexed by a key can also be regard as the PRG. A secure key-agreement (KA) protocol enables any user to input their private key and the public key of the other party, producing a shared secret key as output; for example, it can be instantiated with a Diffie–Hellman key agreement protocol followed by a key derivation function in practice [4], [5]. A standard secure signature scheme $\Pi_\sigma = (\mathsf{Sign}, \mathsf{VerSig})$ that is existentially unforgeable under chosen message attacks (EUF-CMA); for example, it can be instantiated with ECDSA followed by a key derivation function in practice. A commitment scheme that is perfectly hiding and computationally binding under the discrete logarithm assumption; for example, it can be instantiated with vector Pedersen commitment in practice [7].

III. OPSA Framework

In this section, we first give a threat model, and then present our OPSA framework after describing its high-level idea.

A. Threat Model

Let $N = |\mathcal{U}|$, there are N clients and a single untrusted server equipped with TEE, each client $u \in \mathcal{U}$ holds a local model $\mathbf{x}_u$, and all clients want to obtain the summation $\sum \mathbf{x}_u$ with the help of the server S revealing nothing about $\mathbf{x}_u$. Similar to MicroFedML [8] and Flamingo [9], we target the following failures and attacks: (1) honest clients that may dynamically drop out or respond too slowly due to factors such as unstable network conditions, power loss, etc.; and (2) arbitrary actions by an adversary who has control over the server and a bounded fraction of the clients. Note that the latter corresponds to the malicious setting.

In any given aggregation round, we assume that at most δ fraction of N clients drop out during any step, and that the adversary compromises γN clients before each aggregation round begins, where γ is a maximum fraction of corrupt clients, possibly also colluding with the server (which we call the malicious setting). To preserve the privacy of the final aggregation, we require $\delta + \gamma < 1$. For KhPRF-OPSA and POT-OPSA, we assume the server is fully malicious, but for OPSA-RV we prove security only in the case of semi-honest clients. Moreover, since the privacy of client temporary keys in our scheme relies on TEE, we assume TEE is trusted and exclude side-channel attacks and physical attacks from the scope of our work, as discussed in Section II-B. This is analogous to the assumption in SEAR [11].

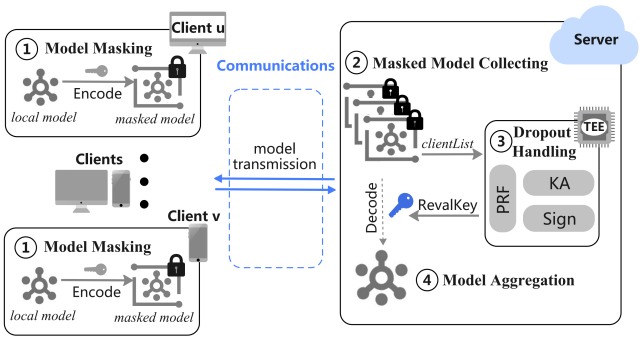


Fig. 2: Workflow of OPSA

B. Intuition of the idea

Applying masks to local models has become an effective approach for preserving privacy in FL. Each client masks its local model using a shared key, and the server subsequently utilizes revealed keys to remove these masks during model aggregation. However, this method imposes significant overhead, requiring clients to regenerate and share a new key in each round, as well as at least three client-server round trips to achieve dropout resilience. Additionally, message authentication codes (MACs) are commonly used to verify aggregation results, but they double the communication cost, adding further inefficiency.

To address these challenges, we use TEE to design multi-round aggregation. By handling client dropouts through server-side TEE, we reduce communication round trips, achieving one-pass aggregation. However, TEE-based SA protocols, which decrypt and aggregate all client models within the limited physical memory of TEE, often face significant computational bottlenecks when aggregating larger models or handling numerous clients. To mitigate this issue, we adopt a novel strategy that offloads tasks like model aggregation and mask elimination outside TEE, requiring TEE to handle only shared keys. Based on these shared keys, clients generate temporary keys to mask their local models, while TEE selectively reveals specific temporary keys to handle client dropouts. This design ensures that TEE memory consumption remains unaffected by the aggregated model size, reducing computational overhead and enhancing scalability.

During the key revealing process, the computational overhead of TEE is further impacted by mask constructions which require different numbers of temporary keys. To this end, we present two mask constructions: (1) KhPRF-OPSA, a single masking protocol based on key-homomorphic pseudo-random function (KhPRF), aggregates temporary keys of all surviving clients within TEE; (2) POT-OPSA, a double masking protocol built upon proxy oblivious transfer (POT), chooses temporary individual keys for surviving clients and temporary pairwise keys for dropped clients within TEE. Both protocols are designed to minimize the inputs and cryptographic operations performed within TEE, enabling them to aggregate larger models and handle more clients within TEE's restricted capacity.

Moreover, we integrate commitment and signature mechanisms to provide verifiable aggregation results. Specifically, each client commits to its local model, and server-side TEE

的侧信道攻击并未揭示Intel SGX 架构中存在任何新的或意外的漏洞[11]。近期研究[22]表明， SGX 攻击通常可通过开发者编写的常量时间代码或英特尔提供的微码更新来缓解。

C. 密码学基本元素

在我们的协议中，我们将采用以下密码学原语来实现随机数生成和安全通信。密码学安全伪随机数生成器（Prg）接收长度固定的均匀随机种子，并输出与随机字符串在计算上无法区分的字符串；例如，在实际应用中可使用AES-CTR实现[4]。由密钥索引的密码学安全伪随机函数（PRF）也可视为Prg。安全密钥协商（KA）协议允许任何用户输入自身私钥及对方公钥，生成共享密钥作为输出结果；例如在实践中可采用Diffie-Hellman密钥协商协议并结合密钥派生函数实现[4][5]。标准安全签名方案Π_σ=(Sign，VerSig)在选定消息攻击（EUF -CMA）下具有存在不可伪造性；例如在实践中可采用 ECDSA 算法配合密钥派生函数实现。这是一种在离散对数假设下完全隐蔽且具有计算约束性的承诺方案；例如，在实际应用中可采用向量Pedersen承诺来实现[7]。

三.OPSA 框架

在本节中，我们首先提出威胁模型，随后在阐述其核心理念后介绍我们的 OPSA 框架。

A. 威胁模型

设$N=|U|$，存在N个客户端和一台配备TEE的单一不可信服务器；每个客户端$u \in U$均持有本地模型$\mathbf{x}_u$，所有客户端均希望借助服务器S获取summation $\sum \mathbf{x}_u$，且服务器不会泄露关于$\mathbf{x}_u$的任何信息。与MicroFedML[8]和Flamingo[9]类似，我们针对以下故障和攻击场景：(1)因网络条件不稳定、断电等因素导致可能动态退出或响应过慢的诚实客户端；(2)由控制服务器及部分客户端的攻击者实施的任意攻击行为。需注意后者对应恶意攻击场景。

在任何一次聚合轮中，我们假设在任意步骤期间最多仅有$\boldsymbol{\delta}$比例的N个客户端退出；并且攻击者在每次聚合轮开始前已攻陷了$\boldsymbol{\gamma}N$个客户端，其中$\boldsymbol{\gamma}$表示被篡改客户端的最大比例（这些客户端可能还会与服务器合谋，我们称之为恶意场景）。为保障最终聚合结果的隐私性，我们要求$\boldsymbol{\delta}+\boldsymbol{\gamma}<1$。对于KhPRF- OPSA 和POT- OPSA 方案，我们假设服务器完全具有恶意行为；而对于 OPSA -RV方案，我们仅在半诚实客户端的情况下证明其安全性。此外，由于本方案中客户端临时密钥的隐私保护依赖于TEE技术，我们假定TEE是可信的，并将侧信道攻击和物理攻击排除在研究范围之外（详见第II-B节），这与SEAR[11]中的假设一致。

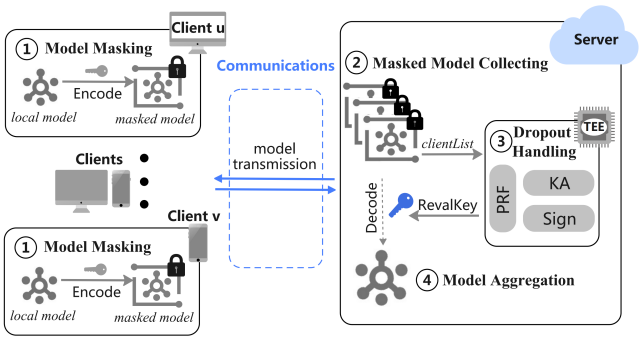


图2： OPSA 工作流程

B. 这个想法的直觉来源

在联邦学习（FL）中，为本地模型添加掩码已成为保护隐私的有效方法。每个客户端使用共享密钥对本地模型进行掩码处理，服务器随后利用已公开的密钥在模型聚合阶段移除这些掩码。然而，这种方法会带来显著的额外开销：客户端需要在每轮运算中重新生成并共享新密钥，并且至少需要三次客户端与服务器之间的通信交互才能实现抗丢弃能力。此外，消息认证码（MAC）虽常用于验证聚合结果，但会使通信成本翻倍，进一步降低系统效率。

为应对这些挑战，我们采用TEE技术设计了多轮聚合机制。通过在服务器端使用TEE处理客户端退出情况，我们减少了通信往返次数，实现了单次计算完成聚合。然而，基于TEE的SA协议需要在TEE有限的物理内存中解密并聚合所有客户端模型，在处理大型模型或大量客户端时往往面临显著的计算瓶颈。为解决这一问题，我们提出了一种创新策略：将模型聚合和掩码消除等任务卸载至TEE外部，仅由TEE负责管理共享密钥。基于这些共享密钥，客户端可生成临时密钥来屏蔽本地模型数据；而TEE则选择性地公开特定临时密钥以应对客户端退出情况。这种设计确保TEE的内存消耗不受聚合模型规模的影响，有效降低了计算开销并提升了系统可扩展性。

在密钥揭示过程中，TEE的计算开销会因掩码构建方式的不同而进一步增加——这些方式需要使用不同数量的临时密钥。为此，我们提出了两种掩码构建方案：(1) KhPRF- OPSA ：这是一种基于密钥同态伪随机函数（KhPRF）的单层掩码协议，可聚合TEE中所有存活客户端的临时密钥；(2)POT- OPSA ：这是一种基于代理无感知传输（POT）的双层掩码协议，分别为存活客户端选择临时个体密钥，并为失效客户端生成临时成对密钥。这两种协议均旨在最大限度减少TEE内部所需的输入数据和加密运算量，从而支持聚合更大规模模型并处理更多客户端。此外，我们还整合了承诺机制与签名机制以确保聚合结果的可验证性：具体而言，每个客户端需对其本地模型

then evaluates these commitments to output an aggregated commitment along with its corresponding signature. The TEE-generated signature binds the aggregation range to a valid list of semi-honest surviving clients, while the aggregated commitment guarantees the authenticity of an aggregation result within this range. These novel methods come together to form OPSA, whose brief workflow is illustrated in Fig. 2.

C. OPSA framework

Before introducing our OPSA framework, we formally define a multi-round SA protocol as follows:

Definition 1 (Multi-round SA protocol)**.** *A multi-round SA protocol involving a client set $\mathcal{U}$, a server S, and an integer R, consists of two phases:*

- **Setup phase***: it is executed only once at the initiation of the protocol to complete initialization.*
- **Training phase***: for round $r \in [1, R]$, the SA protocol is executed once, beginning with a client generating its local model $\mathbf{x}_u^{(r)}$ and ending with the server computing an aggregated model $\sum \mathbf{x}_u^{(r)}$.*

Based on Definition 1, our OPSA is composed of three key components: one-time setup, one-pass aggregation, and result verification. Note that aggregation results are verifiable against a malicious server in the case of semi-honest clients, as discussed in our threat model.

One-Time Setup. During setup phase, all clients and the server execute the following Setup algorithm respectively to correctly generate public parameters.

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, m)$: takes as inputs a security parameter λ, and a model dimension m. It outputs public parameters pp, which defines a key space $\mathcal{K}$, a model space $\mathcal{M}$, a mask space $\mathcal{Y}$, and commitment parameters $\mathsf{pp}_{\mathsf{com}}$.

Then, the server loads a program code into its TEE to create an instance, and distributes an initial global model to all clients. After verifying the correctness of the server's TEE instance through remote attestation, each client accepts the initial global model and proceeds to a FL training round.

We assume that all parties (client u, server S, and TEE instance M_e) have its own authenticated key pair (pk_*, sk_*) based on Public Key Infrastructure (PKI). Each client u can share a key with TEE instance M_e as a root seed to regenerate its temporary key as follows:

$$k_{u,e} = \mathsf{KA}(sk_u, pk_e) = \mathsf{KA}(sk_e, pk_u) = k_{e,u}. \tag{1}$$

Note that all parties initialize a round as $r = 1$, and the TEE instance sets an extra round as $r_e = 0$. The constraint $r > r_e$ means that the server can invoke the TEE instance only once during round r.

One-Pass Aggregation. By incorporating TEE and reusable shared keys, any SA protocol can be optimized into an efficient one-pass aggregation, as long as its mask construction can be decoupled into the following algorithms:

- $\mathbf{mask}_u \leftarrow \mathsf{Encode}(\mathsf{pp}, \mathsf{shKey}, r)$: takes as inputs public parameters pp, a set of shared keys shKey, and round r. It outputs a one-time mask $\mathbf{mask}_u$.
- $(\mathsf{reKey}, \mathsf{rand}) \leftarrow \mathsf{RevealKey}(\mathsf{pp}, \mathcal{V}, \mathcal{D}, r)$: takes as inputs public parameters pp, a list of surviving clients $\mathcal{V}$, a dropout list $\mathcal{D}$, and round r. It outputs a set of revealed keys reKey and a revealed random number rand.
- $\sum \mathbf{mask}_u \leftarrow \mathsf{Decode}(\mathsf{pp}, \mathsf{reKey}, r)$: takes as inputs public parameters pp, a set of revealed keys reKey, and round r. It outputs an aggregated mask $\sum \mathbf{mask}_u$.

Specifically, in FL training round r, each client $u \in \mathcal{U}$ masks its local model as $\mathbf{y}_u^{(r)} = \mathbf{x}_u^{(r)} + \mathsf{Encode}(\mathsf{pp}, \mathsf{shKey}, r)$. Once the TEE instance outputs a set of revealed keys $\mathsf{reKey} = \mathsf{RevealKey}(\mathsf{pp}, \mathcal{V}, \mathcal{D}, r)$, the server computes the plaintext aggregated model as $\sum \mathbf{x}_u^{(r)} = \sum \mathbf{y}_u^{(r)} - \mathsf{Decode}(\mathsf{pp}, \mathsf{reKey}, r)$.

Result Verification (RV). To provide verifiable aggregation results, we divide the aggregation verification into two parts: a valid aggregation range (i.e., semi-honest surviving clients), secured through a signature corresponding to TEE integrity, and an accurate aggregation result, guaranteed by a commitment scheme. In OPSA-RV, the output of TEE, protected by its integrity, binds a valid aggregation range and proves the authenticity of an aggregation result within that range.

We require that a vector commitment scheme consists of the following algorithms:

- $\mathsf{com}_{\mathbf{v}_i} \leftarrow \mathsf{Commit}(\mathbf{v}_i, \mathsf{rand}_i)$: takes as inputs a vector $\mathbf{v}_i$ and a random number rand_i. It outputs a vector commitment $\mathsf{com}_{\mathbf{v}_i}$.
- $b \leftarrow \mathsf{Open}(\mathsf{com}_{\mathbf{v}_i'}, \mathbf{v}_i, \mathsf{rand}_i)$: takes as inputs a commitment $\mathsf{com}_{\mathbf{v}_i'}$, a vector $\mathbf{v}_i$, and a random number rand_i. It outputs a bit b, which equals 1 iff the equation $\mathsf{com}_{\mathbf{v}_i'} = \mathsf{Commit}(\mathbf{v}_i, \mathsf{rand}_i)$ holds.
- $\mathsf{com} \leftarrow \mathsf{Eval}(\{\mathsf{com}_u\}_u)$: takes as input a set of commitments $\{\mathsf{com}_u\}_u$. It outputs a final commitment com.

Each client u commits to its local model as $\mathsf{com}_u = \mathsf{Commit}(\mathbf{x}_u, \mathsf{rand}_u)$, where rand_u can be instantiated randomly via its shared key. This commitment, along with its signature, ensures that the server cannot tamper with the local model. Furthermore, TEE can aggregate these commitments into a final commitment using the Eval algorithm.

Building upon the commitment scheme, a valid aggregation range $\mathcal{V}$ can be processed in TEE, as follows:

- assert: $\forall u \in \mathcal{V}, \mathsf{VerSig}(pk_u, \sigma_u, \mathsf{com}_u) = 1$.
- compute: (dummy, rand) := $\mathsf{RevealKey}(\mathsf{pp}, \mathcal{V}, \mathcal{D}, r)$.
- compute: com := $\mathsf{Eval}(\{\mathsf{com}_u\}_{u \in \mathcal{V}})$.
- sign: $\sigma_e := \mathsf{Sign}(sk_e, \mathsf{com}||\mathsf{rand}||r)$.
- output: $(\sigma_e, \mathsf{com}, \mathsf{rand})$.

Once processed, the final result $\sum_{u \in \mathcal{V}} \mathbf{x}_u$ is uniquely bound to $\mathsf{com} = \mathsf{Commit}(\sum_{u \in \mathcal{V}} \mathbf{x}_u, \mathsf{rand}) = \mathsf{Eval}(\{\mathsf{com}_u\}_{u \in \mathcal{V}})$, ensuring its authenticity. In general, the output of TEE can be regarded as a proof $\boldsymbol{\pi}^{\mathsf{valid}(\sum \mathbf{x}_u^{(r)})} = (r, \sigma_e, \mathsf{com}, \mathsf{rand})$ satisfying the following relation:

$$\begin{cases} \mathsf{VerSig}(pk_e, \sigma_e, \mathsf{com}||\mathsf{rand}||r) = 1 \\ \mathsf{Open}(\mathsf{com}, \sum_{u \in \mathcal{V}} \mathbf{x}_u, \mathsf{rand}) = 1 \end{cases} \tag{2}$$

Finally, the server sends the plaintext aggregated model along with its proof to all clients, and each client verifies this global model before initiating a new FL training round.

Putting it all Together. A complete description without verifiable aggregation in the malicious setting is provided

进行承诺验证，服务器端TEE随后通过评估这些承诺结果，输出聚合后的承诺值及其对应签名。TEE生成的签名将聚合范围与一组有效的半诚实存活客户端列表相绑定，而聚合承诺则确保该范围内聚合结果的真实性。这些创新方法共同构成了 OPSA 系统，其简要工作流程如图2所示。

C. OPSA 框架

在介绍我们的 OPSA 框架之前，我们首先正式定义多轮SA协议如下：

定义 1（多轮SA协议）。*一种涉及客户端集合U、服务器S及整数R的多轮SA协议包含两个阶段：*

- **设置阶段**：*仅在协议启动时执行一次，用于完成初始化。*
- **训练阶段**：*对于轮次 $r \in [1, R]$，SA协议执行一次，从客户端生成其本地模型 $\mathbf{x}_u^{(r)}$ 开始，到服务器计算聚合模型 $\sum \mathbf{x}_u^{(r)}$ 结束。*

根据定义1，我们的 OPSA 包含三个关键组成部分：一次性配置、单次聚合及结果验证。需注意，在半诚实客户端场景下（如我们的威胁模型所述），聚合结果可针对恶意服务器进行验证。

一次性配置。在配置阶段，所有客户端和服务器分别执行以下配置算法以正确生成公钥参数。

- pp ←设置（1^{λ}，m）：接收安全参数λ和模型维度m作为输入，输出公共参数 pp，该参数定义了密钥空间K、模型空间M、掩码空间Y以及承诺参数$\mathsf{pp}_{\mathsf{com}}$。

随后，服务器将程序代码加载至其TEE中以创建实例，并向所有客户端分发初始全局模型。通过远程验证确认服务器TEE实例的正确性后，每个客户端均接受该初始全局模型并进入联邦学习训练轮次。

我们假设所有参与者（客户端u、服务器S以及TEE实例M_e）均基于公钥基础设施（PKI）拥有各自的认证密钥对（pk_*，$sk*$）。每个客户端u均可将密钥与TEE实例M_e共享作为根种子，以如下方式重新生成其临时密钥：

$$k_{u,e} = \mathsf{KA}(sk_u, pk_e) = \mathsf{KA}(sk_e, pk_u) = k_{e,u}. \qquad (1)$$

请注意，所有参与方均将一轮初始化为r=1，而TEE实例会额外设置一轮为r_e=0。约束条件r>r_e意味着服务器仅能在第r轮调用该TEE实例一次。

单次聚合。通过引入TEE和可复用共享密钥，任何SA协议均可优化为高效的单次聚合方案，前提是其掩码构造可分解为以下算法：

- $\mathbf{mask}_u$ ← Encode(pp，shKey，r)：接收公共参数 pp、一组共享密钥 shKey 以及轮次r作为输入，输出一次性掩码$\mathbf{mask}_u$。
- (reKey，rand)←揭示密钥 (pp，V，D，r)：接收公共参数 pp、存活客户端列表V、掉线列表D以及轮次r作为输入，输出一组已揭示的密钥 reKey 和一个已揭示的随机数 rand。
- **Σ masku** ←解码 (pp，reKey，r)：输入参数包括公共参数 pp、一组已公开密钥 reKey 以及轮次信息。

  r. 它输出一个聚合掩码 **Σ masku**。

具体而言，在FL训练轮r中，每个客户端$u \in U$将其本地模型掩码为$\mathbf{y}_u^{(r)}$=$\mathbf{x}_u^{(r)}$+Encode(pp，shKey，r)。当TEE实例输出一组公开密钥reKey = RevealKey(pp，V，D，r)后，服务器计算明文聚合模型为$\sum \mathbf{x}_u^{(r)}$= **Σ yu**$^{(r)}$ − Decode (pp，reKey，r)。

结果验证（RV）。为确保聚合结果可验证，我们将验证过程分为两部分：一是通过对应TEE完整性的签名保障的有效聚合范围（即半诚实存活客户端），二是由承诺机制保证的精确聚合结果。在 OPSA -RV中，受TEE完整性保护的输出结果既限定了一个有效的聚合范围，又证明了该范围内聚合结果的真实性。

我们要求向量承诺方案必须包含以下算法：

- $\mathsf{com}_{\mathbf{v}_i}$ ← Commit($\mathbf{v}_i$，randi)：接受向量$\mathbf{v}_i$和随机数rand_i作为输入，输出向量承诺值$\mathsf{com}_{\mathbf{v}_i}$。
- b ←Open($\mathsf{com}_{\mathbf{v}_i'}$, $\mathbf{v}_i$，randi)：输入参数包括承诺值$\mathsf{com}_{\mathbf{v}}$、向量$\mathbf{v}_i$以及随机数$\mathsf{rand}_i$。它输出一个位 b，该位等于 1 当且仅当方程$\mathsf{com}_{\mathbf{v}_i'} = \mathsf{Commit}(\mathbf{v}_i$，randi) 成立。
- com ← Eval($\{\mathsf{com}_u\}_u$)：以一组承诺 $\{\mathsf{com}_u\}_u$作为输入，输出最终承诺 com。

每个客户端u均通过com_u =Commit($\mathbf{x}_u$将其本地模型提交至系统，其中rand_u可通过其共享密钥随机生成。该提交内容连同签名共同确保服务器无法篡改本地模型。此外，TEE可利用Eval算法将这些提交结果聚合为最终提交结果。

基于该承诺方案，有效的聚合范围V可在TEE中按以下方式处理：

- 断言：, u ʷ $\mathcal{V}$，$\mathsf{VerSig}(pk_u, \sigma_u, \mathsf{com}_u) = 1$。
- 计算：(dummy，rand) := RevealKey(pp，V，D，r)。
- 计算：com :=Eval($\{\mathsf{com}_u\}_{u\in\mathcal{V}}$)。
- 符号：σ_e:= Sign(sk_e，com||rand||r)。
- 输出结果：(σ_e，com，rand)。

经过处理后，最终结果 ᴎ $_{u\,ʷ V}$ $\mathbf{x}_u$将唯一地绑定到com= Commit(ᴎ $_{u\,ʷ V}$ $\mathbf{x}_u$，rand) =Eval($\{\mathsf{com}_u\}_{u\in\mathcal{V}}$)上，从而确保其真实性。一般而言，TEE的输出可视为证明 $\boldsymbol{\pi}$ $^{\mathrm{valid}(\boldsymbol{\Sigma}\,\mathbf{x}_u^{(r)})}$= ($r$，$\sigma_e$，com，rand)，该证明需满足以下关系：

$$\begin{cases} \mathsf{VerSig}(pk_e, \sigma_e, \mathsf{com}||\mathsf{rand}||r) = 1 \\ \mathsf{Open}(\mathsf{com}, \sum_{u\in\mathcal{V}} \mathbf{x}_u, \mathsf{rand}) = 1 \end{cases} \qquad (2)$$

最后，服务器将明文聚合模型及其证明发送至所有客户端，每个客户端在启动新的联邦学习训练轮之前均需验证该全局模型。

综合阐述。本文提供了恶意场景下缺乏可验证聚合数据的完整描述。

Parties: N clients and a TEE-equipped server.
Let $r := 1$ be the round shared by all parties.
Let $r_e := 0$ be the round maintained by TEE.

Setup phase

All parties compute $\mathsf{pp} := \mathsf{Setup}(1^\lambda, m)$.
Server S:
- load TEE instance M_e.
- send the initial global model $\boldsymbol{\theta}^{(1)}$ to clients.

Client $u \in \mathcal{U}$:
- verify the validity of the server-side TEE M_e.
- generate a shared key $k_{u,e} := \mathsf{KA}(sk_u, pk_e)$.

Training phase // repeat steps for all $r \in [1, R]$

1. **Mask step.**

Client $u \in \mathcal{U}$: // ① *model masking*
- receive from the server $\boldsymbol{\theta}^{(r)}$.
- train a local model $\mathbf{x}_u^{(r)}$ based on $\boldsymbol{\theta}^{(r)}$.
- send the server a masked model $\mathbf{y}_u^{(r)} := \mathbf{x}_u^{(r)} + \mathsf{Encode}(\mathsf{pp}, \mathsf{shKey}, r)$.

Server S: // ② *masked model collecting*
- collect $\mathbf{y}_u^{(r)}$ from users (denote with $\mathcal{V}^{(r)} \subseteq \mathcal{U}$).
- load ($\mathcal{V}^{(r)}$, $\mathcal{D}^{(r)}$, r) into M_e.

2. **Unmask step.**

TEE instance M_e: // ③ *dropout handling*
- if $r_e \geq r$, abort; otherwise, $r_e := r$.
- assert $|\mathcal{V}^{(r)}| \geq \lceil (1-\delta)N \rceil$.
- return a set of revealed keys (reKey,dummy) := $\mathsf{RevealKey}(\mathsf{pp}, \mathcal{V}^{(r)}, \mathcal{D}^{(r)}, r)$.

Server S: // ④ *model aggregation*
- $\sum \mathbf{x}_u^{(r)} := \sum \mathbf{y}_u^{(r)} - \mathsf{Decode}(\mathsf{pp}, \mathsf{reKey}, r)$.
- update $\boldsymbol{\theta}^{(r)}$ to $\boldsymbol{\theta}^{(r+1)}$ using $\sum \mathbf{x}_u^{(r)}$.
- send $\boldsymbol{\theta}^{(r+1)}$ to clients.

Fig. 3: Detailed description of the OPSA framework without verifiable aggregation in the malicious setting.

Parties: N clients and a TEE-equipped server.
Let $r := 1$ be the round shared by all parties.
Let $r_e := 0$ be the round maintained by TEE.

Setup phase

All parties compute $\mathsf{pp} := \mathsf{Setup}(1^\lambda, m)$.
Server S:
- load TEE instance M_e.
- send the initial global model $\boldsymbol{\theta}^{(1)}$ to clients.

Client $u \in \mathcal{U}$:
- verify the validity of the server-side TEE M_e.
- generate a shared key $k_{u,e} := \mathsf{KA}(sk_u, pk_e)$.

Training phase // repeat steps for all $r \in [1, R]$

1. **Mask step.**

Client $u \in \mathcal{U}$: // ① *model masking*
- receive from the server $\boldsymbol{\theta}^{(r)}$ and a proof, and check the proof $\boldsymbol{\pi}^{\mathsf{valid}(\sum \mathbf{x}_u^{(r-1)})}$ if $r > 1$.
- train a local model $\mathbf{x}_u^{(r)}$ based on $\boldsymbol{\theta}^{(r)}$.
- send the server a message $msg_u^{(r)}$ consisting of
  - $\mathbf{y}_u^{(r)} := \mathbf{x}_u^{(r)} + \mathsf{Encode}(\mathsf{pp}, \mathsf{shKey}, r)$,
  - $\mathsf{com}_u^{(r)} := \mathsf{Commit}(\mathbf{x}_u, \mathsf{rand}_u)$,
  - $\sigma_u^{(r)} := \mathsf{Sign}(sk_u, \mathsf{com}_u^{(r)})$.

Server S: // ② *masked model collecting*
- collect $msg_u^{(r)}$ from users (denote with $\mathcal{V}^{(r)} \subseteq \mathcal{U}$).
- load ($\mathcal{V}^{(r)}$, $\mathcal{D}^{(r)}$, $\{\mathsf{com}_u^{(r)}, \sigma_u^{(r)}\}_u$, r) into M_e.

2. **Unmask step.**

TEE instance M_e: // ③ *dropout handling*
- if $r_e \geq r$, abort; otherwise, $r_e := r$.
- assert $|\mathcal{V}^{(r)}| \geq \lceil (1-\delta)N \rceil$.
- $\mathsf{VerSig}(pk_u, \sigma_u^{(r)}, \mathsf{com}_u^{(r)}) = 1$ for all $u \in \mathcal{V}^{(r)}$.
- return an output consisting of
  - $(\mathsf{reKey}, \mathsf{rand}) := \mathsf{RevealKey}(\mathsf{pp}, \mathcal{V}^{(r)}, \mathcal{D}^{(r)}, r)$,
  - $\mathsf{com} := \mathsf{Eval}(\{\mathsf{com}_u\}_{u \in \mathcal{V}^{(r)}})$,
  - $\sigma_e := \mathsf{Sign}(sk_e, \mathsf{com}||\mathsf{rand}||r)$.

Server S: // ④ *model aggregation*
- $\sum \mathbf{x}_u^{(r)} := \sum \mathbf{y}_u^{(r)} - \mathsf{Decode}(\mathsf{pp}, \mathsf{reKey}, r)$.
- update $\boldsymbol{\theta}^{(r)}$ to $\boldsymbol{\theta}^{(r+1)}$ using $\sum \mathbf{x}_u^{(r)}$.
- form $\boldsymbol{\pi}^{\mathsf{valid}(\sum \mathbf{x}_u^{(r)})}$ according to Equation 2.
- send $\boldsymbol{\theta}^{(r+1)}$ and $\boldsymbol{\pi}^{\mathsf{valid}(\sum \mathbf{x}_u^{(r)})}$ to clients.

Fig. 4: Detailed description of the OPSA framework with verifiable aggregation against a malicious server.

in Fig. 3, while that with verifiable aggregation against a malicious server is provided in Fig. 4. OPSA first performs the Setup algorithm to generate public parameters, the server loads the TEE instance and delivers an initial model, and each client checks the TEE instance through remote attestation and negotiates a shared key with it (**Setup phase**).

Upon receiving the global model, during the **Training phase**, all clients validate it using its corresponding proof to start a new FL training round. Each client combines the approved global model with its private data to train a local model, which is then encoded as a masked model. To achieve result verifiability, each client commits to its local model and signs the corresponding commitment. A message, including the masked model, commitment, and signature, is sent to the server (① *Model Masking*). Subsequently, the server collects masked models from distinct clients, which are appended into a list of surviving clients. Based on this, the server easily determines a dropout list. Both the surviving and dropout lists are then loaded into the TEE instance (② *Masked Model Collecting*). Following that, the TEE instance verifies that it has been invoked only once per round to prevent multiple executions. Utilizing the loaded lists, the TEE instance publishes revealed keys, an aggregated commitment, and its signature to the server (③ *Dropout Handling*). Upon obtaining the output from the TEE instance, the server utilizes the revealed keys

参与方：N个客户端和一台配备TEE的服务器。
令r:= 1 作为所有参与方共享的轮次。
令r_e:= 0 为由 TEE 维护的轮次。

设置阶段
所有参与方均计算 pp := Setup(1^{λ}，m)。
服务器S：
- 加载 TEE 实例M_e。
- 将初始全局模型$\vartheta^{(1)}$发送给客户端。$u \in U$：
- 验证服务器端TEE M_e的有效性。
- 生成共享密钥$k_{u,e}$:=KA(sk_u，pke)。**训练阶段**// 对所有$r \in [1, R]$重复上述步骤

1. **掩码步骤。**
客户端$u \in U$：　　// ◯1 *模型掩蔽*
- 从服务器接收$\vartheta^{(r)}$。
- 基于$\vartheta^{(r)}$训练一个本地模型$\mathbf{x}_u{}^{(r)}$。
- 向服务器发送经过掩码处理的模型 $\mathbf{y}_u{}^{(r)}$:=$\mathbf{x}_u{}^{(r)}$+Encode(pp，shKey，r)。

服务器S：　　// ② *掩模模型收集*
- 从用户处收集$\mathbf{y}_u{}^{(r)}$（记为$V^{(r)} \subseteq U$）。
- 将$V^{(r)}$、$D^{(r)}$及r输入至M_e中。

2. **解除遮罩步骤。**
TEE 实例M_e：　　// ③ *数据丢弃处理*
- 如果$r_e \geq r$，则终止；否则，r_e:=r。
- 断言 $|V^{(r)}| \geq \gamma(1-\delta)N$。
- 返回一组已公开的密钥 (reKey，dummy) := RevealKey (pp，$V^{(r)}$，$D^{(r)}$，r)。

服务器S：　　// ④ *模型聚合*
- $\sum \mathbf{x}_u{}^{(r)}$:= $\mathbf{yu}^{(r)}$– Decode (pp，reKey，r)。
- 将 $\theta^{(r)}$更新为 $\theta^{(r+1)}$，使用$\sum \mathbf{x}_u{}^{(r)}$。
- 向客户端发送 $\theta^{(r+1)}$。

图3：在恶意场景下，未采用可验证聚合机制的 OPSA 框架详细说明。

图3展示了相关流程，而针对恶意服务器的可验证聚合方案则展示于图4。 OPSA 首先执行设置算法以生成公共参数；服务器加载TEE实例并提供初始模型；各客户端通过远程认证验证TEE实例，并与其协商共享密钥（**设置阶段**）。

在接收全局模型后，于**训练阶段**，所有客户端均使用对应的证明文件进行验证，从而启动新一轮联邦学习训练。每个客户端将经过验证的全局模型与其私有数据结合，训练本地模型，并将其编码为掩码模型。为确保结果可验证性，每个客户端需提交其本地模型并签署相应的承诺声明。包含掩码模型、承诺声明及签名的消息会被发送至服务器（○1 *模型掩码化*）。随后，服务器从各客户端收集掩码模型，并将其汇总至存活客户端列表中；基于此列

参与方：N个客户端和一台配备TEE的服务器。
令r:= 1 作为所有参与方共享的轮次。
令r_e:= 0 为由 TEE 维护的轮次。

设置阶段
所有参与方均计算 pp := Setup(1^{λ}，m)。
服务器S：
- 加载 TEE 实例M_e。
- 将初始全局模型$\theta^{(1)}$发送给客户端。$u \in U$：
- 验证服务器端TEE M_e的有效性。
- 生成共享密钥$k_{u,e}$:=KA(sk_u，pke)。**训练阶段**// 对所有$r \in [1, R]$重复上述步骤

1. **掩码步骤。**
客户端$u \in U$：　　// ◯1 *模型掩蔽*
- 从服务器接收$\theta^{(r)}$及证明，并在r>1时检查该证明$\pi^{\text{有效}}$ $(\Sigma \mathbf{x}_{u(r-1)})$。
- 基于$\theta^{(r)}$训练一个本地模型$\mathbf{x}_u{}^{(r)}$。
- 向服务器发送一条由$msg_u{}^{(r)}$组成的消息
  - $\mathbf{y}_u{}^{(r)}$:=$\mathbf{x}_u{}^{(r)}$+Encode(pp，shKey，r)，
  - $\mathsf{com}_u{}^{(r)}$:=Commit($\mathbf{x}_u$，randu)；
  - $\sigma_u{}^{(r)}$:= 符号($sk_u, \mathsf{com}_u(r)$)。

服务器S：　　// ② *掩模模型收集*
- 从用户处收集$msg_u{}^{(r)}$（记为$V^{(r)} \subseteq U$）。
- 将（$V^{(r)}$、$\mathcal{D}(r)$以及$\{\mathsf{com}_u{}^{(r)}, \sigma_u{}^{(r)}\}_u$，$r$）载入$M_e$中。

2. **解除遮罩步骤。**
TEE 实例M_e：　　// ③ *数据丢弃处理*
- 如果$r_e \geq r$，则终止；否则，r_e:=r。
- 断言 $|V^{(r)}| \geq \gamma(1-\delta)N$。
- VerSig($pk_u, \sigma_u{}^{(r)}$，$\mathsf{com}_u{}^{(r)}$) = 1 对于所有$u \in V^{(r)}$。
- 返回一个由以下内容组成的输出结果：
  - (reKey，rand) := RevealKey (pp，$V^{(r)}$，$D^{(r)}$，r)，- com :=Eval($\{\mathsf{com}_u\}_{u \in \mathcal{V}(r)}$)，
  - σ_e:= 符号(sk_e，com||rand||r)。

服务器S：　　// ④ *模型聚合*
- $\sum \mathbf{x}_u{}^{(r)}$:= Σ $\mathbf{yu}^{(r)}$– Decode (pp，reKey，r)。
- 将$\theta(r)$更新为$\theta^{(r+1)}$，使用$\sum \mathbf{x}_u{}^{(r)}$。
- 根据公式2，形式$\pi^{\text{valid}}(\Sigma \mathbf{x}_{u(t)})$成立。
- 向客户端发送$\theta^{(r+1)}$和$\pi^{\text{valid}}(\Sigma \mathbf{xu}^{(r)})$。

图4： OPSA 框架的详细描述，包含针对恶意服务器的可验证聚合功能。

表，服务器可轻松生成淘汰列表。最终，存活列表与淘汰列表均会被载入TEE实例中（② *隐藏模型收集阶段*）。随后，TEE实例会验证其在每轮中仅被调用一次，以防止多次执行。利用已加载的列表，TEE实例将公开密钥、聚合承诺结果及其签名发布至服务器（③ *丢弃处理*）：服务器在获取TEE实例的输出后，利用所揭示的密钥解密聚合模型，并将承诺值与签名作为证明，以验证该模型的真

to unmask an aggregated model and takes the commitment and the signature as a proof, proving the authenticity of this model (④ *Model Aggregation*). Finally, a new global model is updated and sent to all clients for the next round.

IV. Construction

Based on the OPSA framework, we instantiate two types of SA protocols: one is a single masking scheme based on the key-homomorphic pseudo-random function (KhPRF), and the other is a double masking scheme based on proxy oblivious transfer (POT). Moreover, the one-time setup and the result verification (RV) are independent of SA protocols, allowing us to share them for both constructions mentioned above.

A. Construction Based on KhPRF

We construct a single masking scheme with KhPRF [23] based on the learning with rounding (LWR) assumption [24].

Definition 2 (LWR [24])**.** *Given a security parameter λ, let $\ell = \ell(\lambda)$, $q = q(\lambda)$, $p = p(\lambda)$, and moduli $q \geq p \geq 2$ be integers, for a vector $\mathbf{s} \in \mathbb{Z}_q^\ell$, the LWR$_{\ell,q,p}$ assumption states that for any $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^\ell$, $b \xleftarrow{\$} \mathbb{Z}_q$, the following indistinguishability holds: $(\mathbf{a}, \lfloor \langle \mathbf{a}, \mathbf{s} \rangle \rceil_p) \approx_c (\mathbf{a}, \lfloor b \rceil_p)$, where $\lfloor \cdot \rceil_p : \mathbb{Z}_q \to \mathbb{Z}_p$ is the rounding function defined as $\lfloor x \rceil_p = \lfloor x \cdot (p/q) \rfloor \mod p$.*

Definition 3 (KhPRF by LWR [23])**.** *Let $H_1 : \mathcal{X} \to \mathbb{Z}_q^\ell$ be a hash function modeled as a random oracle, define the function $F_{\mathrm{LWR}} : \mathbb{Z}_q^\ell \times \mathcal{X} \to \mathbb{Z}_p$ as $F_{\mathrm{LWR}}(\mathbf{k}, x) \leftarrow \lfloor \langle \mathbf{k}, H_1(x) \rangle \rceil_p$ under the LWR$_{\ell,q,p}$ assumption. For every $\mathbf{k}_1, \mathbf{k}_2 \in \mathbb{Z}_q^\ell$, F_{LWR} is 1-almost key homomorphic in the sense that $F_{\mathrm{LWR}}(\mathbf{k_1}, x) + F_{\mathrm{LWR}}(\mathbf{k_2}, x) = \mathrm{F}(\mathbf{k_1} + \mathbf{k_2}, x) + e$ where $e \in \{0, 1\}$.*

Note that KhPRF can also be instantiated based on the Ring-LWR assumption [24]. In this work, we instantiate it as $\mathrm{KF}(\mathbf{k}, \mathbf{x}) = (\lfloor \langle \mathbf{k}, H_1(x_1) \rangle \rceil_p, \cdots, \lfloor \langle \mathbf{k}, H_1(x_m) \rangle \rceil_p)$.

Construction 1 (KhPRF-OPSA)**.** *Let $F_1 : \mathbb{Z}_q \times \mathbb{Z}_q^* \to \mathbb{Z}_q^\ell$ be a secure PRF, and $F_0 : \mathbb{Z}_q \times \mathbb{Z}_q^* \to \mathbb{Z}_q$ be a secure PRF. Based on the* LWR$_{\ell,q,p}$ *assumption, let* $\mathrm{KF}(\cdot, \cdot)$ *be 1-almost KhPRF, the key space be $\mathcal{K} = \mathbb{Z}_q^\ell$, the model space be $\mathcal{M} = \mathbb{Z}_p^m$, and the mask space be $\mathcal{Y} = \mathbb{Z}_p^m$.*

- $\mathbf{mask}_u \leftarrow \mathsf{Encode}(\mathsf{pp}, \mathsf{shKey}, r)$*: each client u parses the shared key $k_{u,e}$ from* shKey*, computes a temporary key $\mathbf{k}_u = F_1(k_{u,e}, r)$, and outputs $\mathbf{mask}_u = \mathrm{KF}(\mathbf{k}_u, \mathbf{b})$ where $\mathbf{b} \in \mathbb{Z}_q^m$ is parsed from public parameters* pp*.*
- $(\mathsf{reKey}, \mathsf{rand}) \leftarrow \mathsf{RevealKey}(\mathsf{pp}, \mathcal{V}, \mathcal{D}, r)$*: TEE instance M_e loads the surviving list $\mathcal{V}$, computes the temporary key $\mathbf{k}_u = F_1(k_{e,u}, r)$ and $\mathsf{rand}_u = F_0(k_{e,u}, 0||r)$ for all $u \in \mathcal{V}$, and sums up temporary keys $\mathbf{k}_\mathcal{V} = \sum_{u \in \mathcal{V}} \mathbf{k}_u$ and random numbers $\mathsf{rand} = \sum_{u \in \mathcal{V}} \mathsf{rand}_u$. It outputs $\mathsf{reKey} = \{\mathbf{k}_\mathcal{V}\}$ and* rand*.*
- $\mathbf{mask}_\mathcal{V} \leftarrow \mathsf{Decode}(\mathsf{pp}, \mathsf{reKey}, r)$*: the server parses the revealed key $\mathbf{k}_\mathcal{V}$ from* reKey*, and outputs a mask $\mathbf{mask}_\mathcal{V} = \mathrm{KF}(\mathbf{k}_\mathcal{V}, \mathbf{b})$ where $\mathbf{b} \in \mathbb{Z}_q^m$ is parsed from public parameters* pp*.*

Since 1-almost KhPRF causes an error $\mathbf{e} \in \{0, 1\}^m$, the summation of masks is given by $\sum_{u \in \mathcal{V}} \mathbf{mask}_u = \mathbf{mask}_\mathcal{V} + \mathbf{e}_\mathcal{V}$ where $|\mathbf{e}_{\mathcal{V},i}|_{i \in [1,m]} < |\mathcal{V}|$. To remove the mask correctly, we can set a model scaling factor Δ sufficiently large to ensure that errors are removed by the rounding operation. The process of the above construction is the same as shown in Fig. 3 and Fig. 4, except for the following modifications:

- In the masking step, each client u masks its local model $\mathbf{y}_u = \Delta \cdot \mathbf{x}_u + \mathsf{Encode}(\mathsf{pp}, \mathsf{shKey}, r)$.
- In the unmasking step, the server obtains the result $\sum_{u \in \mathcal{V}} \mathbf{x}_u = \frac{1}{\Delta} \left(\sum_{u \in \mathcal{V}} \mathbf{y}_u - \mathsf{Decode}(\mathsf{pp}, \mathsf{reKey}, r) \right)$.

B. Construction Based on Proxy-OT

Here, we extend the notion of non-interactive proxy oblivious transfer (POT_2^1) defined by Choi et al. [25] to *k-out-of-n proxy oblivious transfer* (POT_n^k), and then utilize it to construct a double masking scheme.

We introduce the syntax of our POT_n^k protocol in Appendix A. The POT_n^k protocol involves three parties: a sender, a chooser, and a proxy. Let $F_2 : \mathbb{Z}_q \times \mathbb{Z}_q^* \to \mathbb{Z}_q^n$ be a secure PRF, and $k_{s,c} = \mathrm{KA}(pk_C, sk_S)$ be the shared key between the sender and the chooser, we can instantiate POT_n^k from the key agreement protocol as follows:

- $A \leftarrow \mathsf{Send}(r, sk_S, pk_C, \{m_i\}_{i \in [n]})$: the sender computes $(z_1, z_2, \cdots, z_n) = F_2(k_{s,c}, r)$ where $|z_i| = \lambda$, generates a permuted order $\pi := \mathrm{Perm}[n]$, and sets $A = \{ct_i\}_{i \in [n]}$ where $ct_{\pi(i)} = z_i \oplus m_i$.
- $B \leftarrow \mathsf{Choose}(r, sk_C, pk_S, \{b_j\}_{j \in [k]})$: the chooser computes $(z_1, z_2, \cdots, z_n) = F_2(k_{c,s}, r)$ where $|z_i| = \lambda$, generates a permuted order $\pi = \mathrm{Perm}[n]$, and reveals only the part associated with the choice $B = \{aux_j\}_{j \in [k]}$ where $aux_j = (\pi(b_j), z_{b_j})$.
- $M \leftarrow \mathsf{Proxy}(r, pk_S, pk_C, A, B)$: the proxy parses $A = (ct_1, ct_2, \cdots, ct_n)$ and $B = \{(b'_j, z_{b_j})\}_{j \in [k]}$, and computes $M = \{m_{b'_j}\}_{j \in [k]}$ where $m_{b'_j} = ct_{b'_j} \oplus z_{b_j}$.

In our POT_n^k protocol designed for one-pass aggregation, we designate TEE as the chooser (rather than the proxy) to alleviate its computational burden, with the clients as the senders and the server as the proxy. Given that OPSA only requires safeguarding the privacy of clients, our focus is solely on ensuring *sender privacy* in POT. This implies that the proxy can know choice of the chooser, thereby enabling TEE to make choices based on the client list provided by the server. Moreover, the permuted operation in the above instantiation can also be removed. Note that the chooser (i.e., TEE) is required to choose only once per round.

Construction 2 (POT-OPSA)**.** *Let $F_0 : \mathbb{Z}_q \times \mathbb{Z}_q^* \to \mathbb{Z}_q$ be a secure PRF, $G : \mathbb{Z}_q \to \mathbb{Z}_q^m$ be a secure PRG. Based on the instantiation POT_n^k, let the key space be $\mathcal{K} = \mathbb{Z}_q$, the model space be $\mathcal{M} = \mathbb{Z}_q^m$, and the mask space be $\mathcal{Y} = \mathbb{Z}_q^m$.*

- $\mathbf{mask}_u \leftarrow \mathsf{Encode}(\mathsf{pp}, \mathsf{shKey}, r)$*: each client u parses its shared keys $\{k_{u,v}\}_{v \in \mathcal{U}}$ from* shKey*, samples a temporary individual key $\widetilde{k}_u \xleftarrow{\$} \{0, 1\}^\lambda$, and computes a temporary pairwise key $\widetilde{k}_{u,v} = F_0(k_{u,v}, r)$ for $v \in \mathcal{U}$. It outputs a one-time mask summation $\mathbf{mask}_u = G(\widetilde{k}_u) + \sum_{v \in \mathcal{U}, u < v} G(\widetilde{k}_{u,v}) - \sum_{v \in \mathcal{U}, u > v} G(\widetilde{k}_{u,v})$.*
- $(\mathsf{reKey}, \mathsf{rand}) \leftarrow \mathsf{RevealKey}(\mathsf{pp}, \mathcal{V}, \mathcal{D}, r)$*: TEE instance M_e loads the surviving list $\mathcal{V}$ and the dropout list $\mathcal{D}$,*

实性。④ *模型聚合*）。最后，新的全局模型将被更新并发送给所有客户端以进行下一轮运算。

四.构建

基于 OPSA 框架，我们设计了两种SA协议：一种是基于密钥同态伪随机函数（KhPRF）的单层掩码方案；另一种是基于代理盲传输（POT）的双层掩码方案。此外，一次性设置过程与结果验证（RV）均独立于SA协议本身，因此可同时应用于上述两种架构中。

A. 基于KhPRF 的构建方法

我们基于舍入学习（LWR）假设[24]，构建了一种采用Kh-PRF[23]的单一掩码方案。

定义 2（LWR[24]）。*给定一个安全参数 ，令 ℓ = ℓ（ ），q = q（ ），p = p（ ），且模数 q ≥ p ≥ 2 be integers，for a vector* $\mathbf{s} \cdot \mathsf{Z}_q$*；LWR* $_{\ell, q, p}$*假设表明对于任意* $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^{\ell}$, $b \xleftarrow{\$} \mathbb{Z}_q$ *中，以下不可区分性成立：(*$\mathbf{a}$，l〈$\mathbf{a}$，$\mathbf{s}$〉lp) ≈ $_c$($\mathbf{a}$，$lblp$)，*where* $l\cdot\rceil_p : \mathbb{Z}_q \to \mathbb{Z}_p$*是定义为* $\lfloor x \rceil_p = lx \cdot (p/q) \rfloor \bmod p$ *的舍入函数。*

定义 3（基于 LWR 的 KhPRF[23]*: d* → Z_q*是一个建模为随机预言机的哈希函数，定义函数* F_{LWR}:$\mathsf{Z}_q \times d \to \mathbb{Z}_p$*为* $F_{\mathrm{LWR}}(\mathbf{k}, x) \leftarrow l$〈$\mathbf{k}$，$H_1(x)$〉$\rceil_p$*，该定义基于LWR* $_{\ell, q, p}$*假设。对于每个* $\mathbf{k}_1, \mathbf{k}_2 \cdot \mathsf{Z}_{\ell_q}$ F_{LWR}*在*1*-几乎键同态的意义下满足：* $F_{\mathrm{LWR}}(\mathbf{k}_1, x) + F_{\mathrm{LWR}}(\mathbf{k}_2, x) = \mathrm{F}(\mathbf{k}_1 + \mathbf{k}_2, x) + e$*，其中* $e \cdot \{0, 1\}$。

请注意，KhPRF也可以基于Ring- LWR 假设[24]进行实例化。在本研究中，我们将其实例化为 KF ($\mathbf{k}$，$\mathbf{x}$) = (l〈$\mathbf{k}$，$H_1(x_1)$〉l_p，⋯，l〈$\mathbf{k}$，$H_1(xm)$〉lp)。

构造 1(KhPRF- OPSA)。*设* $F_1 : \mathbb{Z}_q \times \mathsf{Z}_{\nu q} \to \mathsf{Z}_{\ell q}$*为一个安全的PRF函数，且* $F_0 : \mathbb{Z}_q \times \mathsf{Z}_{\nu q} \to \mathbb{Z}_q$*也为一个安全的PRF函数。基于*LWR $_{\ell, q, p}$*假设，令*KF (·，·) *为*1*-近似KhPRF函数，密钥空间为* $K = \mathsf{Z}_{\ell q}$*，模型空间为* $M = \mathsf{Zmp}$*，掩码空间为* $Y = \mathsf{Zmp}$。

- $\mathbf{mask}_u \leftarrow$ Encode (pp，shKey，*r): 每个客户端 u 从*shKey*中解析共享密钥* $k_{u,e}$*，计算临时密钥* $\mathbf{k}_u = F_1(k_{u,e}, r)$*，并输出* $\mathbf{mask}_u = \mathrm{KF}(\mathbf{k}_u, \mathbf{b})$*，其中* $\mathbf{b} \cdot \mathsf{Zmq}$*是从公共参数*pp*中解析得出的。*
- *(*reKey，rand)← RevealKey (pp，y，D，*r): TEE实例* M_e*加载存活列表* y*，为所有* $u \cdot y$ *计算临时密钥* $\mathbf{k}_u = F_1(k_{e,u}, r)$ *和* $\mathsf{rand}_u = F_0(k_{e,u}, 0||r)$*，并将这些临时密钥相加得到* $\mathbf{k}_{\mathcal{V}} = \Sigma_{u \cdot V} \mathbf{k}_u$ *and random numbers* rand = $\Sigma_{u \cdot V} \mathsf{rand}_u$*。最终输出*reKey ={$\mathbf{k}_{\mathcal{V}}$} *和*rand。
- $\mathbf{mask}_{\mathcal{V}} \leftarrow$ Decode (pp，reKey，*r)：服务器从*reKey*中解析出公开密钥* $\mathbf{k}_{\mathcal{V}}$*，并输出掩码* $\mathbf{mask}_{\mathcal{V}} = \mathrm{KF}(\mathbf{k}_{\mathcal{V}}, \mathbf{b})$*，其中* $\mathbf{b} \cdot \mathsf{Zmq}$*是从公共参数*pp*中解析得出的。*

由于1-近似KhPRF会产生误差 $\mathbf{e} \cdot \{0,1\}^m$，掩码的求和表达式为 $\Sigma_{u \cdot V} \mathbf{mask}_u = \mathbf{mask}_{\mathcal{V}} + \mathbf{e}_{\mathcal{V}}$，其中 $|\mathbf{e}_{\mathcal{V},i}|_{i \cdot [l,\, m]} < |y|$。为正确消除掩码效应，可设置足够大的模型缩放因子Δ以确保误差通过舍入运算被消除。上述构建过程与图3和图4所示方法相同，仅存在以下修改：

- 在掩码化步骤中，每个客户端u对其本地模型进行掩码处理：$\mathbf{y}_u = \Delta \cdot \mathbf{x}_u$+Encode(pp，shKey，$r$)。
- 在解掩码步骤中，服务器获取结果 $\sum_{u \cdot V} xu = \frac{1}{\Delta}$ (∑u · V yu _ Decode (pp，reKey，r))。

B. 基于Proxy-OT 的构建方法

在此，我们将Choi等人[25]定义的非交互式代理感知传输（POT12）概念扩展为*k-out-of-n 代理感知传输*（POT-kn），并利用该概念构建了一种双重掩码方案。

我们在附录A中介绍了POTkn协议的语法。该协议涉及三方：发送方、选择方和代理方。设 $F_2 : \mathbb{Z}_q \times \mathsf{Z}_{\nu q} \to \mathsf{Znq}$ 为安全伪随机函数，$k_{s,c}$=KA(pk_C, sk_S)为发送方与选择方之间的共享密钥，则可如下从密钥协商协议中实例化POTkn：

- A ←发送（r，$sk_S, pk_C, \{m_i\}_{i \in [n]}$）：发送方计算（$z_1, z_2, \cdots, z_n$) = $F_2(k_{s,c}, r)$，其中 $|z_i|$ =λ；生成置换顺序 π:= Perm[n]；并定义A= $\{ct_i\}_{i \in [n]}$，其中$ct_{\pi(i)} = z_i$牛m_i。
- B ←选择（r，$sk_C, pk_S, \{b_j\}_{j \in [k]}$）：选择者计算（$z_1, z_2, \cdots, z_n$) = $F_2(k_{c,s}, r)$，其中 $|z_i|$ =λ；生成置换顺序π= Perm [n]；并仅公开与选择B= $\{aux_j\}_{j \in [k]}$ 相关的部分，其中aux_j= ($\pi(bj)$，z_{b_j})。
- M ←代理函数（r，pk_S, pk_C，A，B）：该代理函数解析A= (ct_1, ct_2，⋯，ctn) 和B= $\{(b'_j, z_{b_j})\}_{j \in [k]}$，并计算$M$= $\{m_{b_j}$，$\}_{j \in [k]}$，其中$m_{b'_j} = ct_{b_j}$，牛z_{b_j}。

在我们为单次聚合设计的POTkn协议中，我们将TEE指定为选择者（而非代理），以减轻其计算负担；客户端作为发送方，服务器则作为代理。鉴于 OPSA 仅需保护客户端隐私，我们的重点完全集中在确保POT中的*发送方隐私*。这意味着代理能够知晓选择者的具体选择，从而使TEE能够根据服务器提供的客户端列表进行决策。此外，上述方案中的置换操作亦可省略。需要注意的是，选择者（即TEE）每轮仅需执行一次选择操作。

构造 2(POT- OPSA)。*设* $F_0 : \mathbb{Z}_q \times \mathsf{Z}_{\nu q} \to \mathbb{Z}_q$*为安全伪随机函数，* $G : \mathbb{Z}_q \to \mathsf{Zmq}$*为安全伪随机生成器。基于实例化POTkn，令密钥空间为* $K = \mathbb{Z}_q$*，模型空间为* $M = \mathsf{Zmq}$*，掩码空间为* $Y = \mathsf{Zmq}$。

- $\mathbf{mask}_u \leftarrow$ Encode (pp，shKey，*r): 每个客户端 u 从*shKey*中解析其共享密钥* $\{k_{u,v}\}_{v \in \mathcal{U}}$*，并采样一个临时个体密钥* $\sim k_{\underset{\sim}{u}} \xleftarrow{\$}$ <sup> $\{0, 1\}^{\lambda}$*，并计算一个临时的成对密钥* $k_{u,v} = F_0(k_{u,v}, r)$*（其中* $v \cdot \underset{\sim}{U}$*）。它输出一个一阶* $\underset{\sim}{}$*阶掩码求和结果* $\sim$ $\mathbf{mask}_u = G(k_u) + \Sigma_{v \cdot U,\, u < v} G(ku, v) _ \Sigma_{v \cdot U,\, u > v} G(ku, v)$。
- *(*reKey，rand)← RevealKey (pp，y，D，*r): TEE实例* M_e*加载*

constructs a choice set $\{b_v\}_{v\in\mathcal{D}}$*, and computes an auxiliary set* $B_u = \mathrm{POT}_n^k.\mathsf{Choose}(r, sk_e, pk_u, \{0\}\cup\{b_v\}_{v\in\mathcal{D}})$ *and a random number* $\mathsf{rand}_u = F_0(k_{e,u}, 0||r)$ *for* $u \in \mathcal{V}$*. It outputs* $\mathsf{reKey} = \bigcup_{u\in\mathcal{V}} B_u$ *and* $\mathsf{rand} = \sum_{u\in\mathcal{V}} \mathsf{rand}_u$*.*

- $\mathbf{mask}_{\mathcal{U}} \leftarrow \mathsf{Decode}(\mathsf{pp}, \widetilde{\mathsf{reKey}}, r)$*: the server parses the revealed keys* $\{\widetilde{k}_u, \widetilde{k}_{u,v}\}_{u\in\mathcal{V},v\in\mathcal{D}}$ *from* $\widetilde{\mathsf{reKey}}$*, and outputs the summation of all masks* $\mathbf{mask}_{\mathcal{U}} = \sum_{u\in\mathcal{V},v\in\mathcal{D}} \Big(G(\widetilde{k}_u) + \sum_{u<v} G(\widetilde{k}_{u,v}) - \sum_{u>v} G(\widetilde{k}_{u,v})\Big)$.

In fact, POT_n^k is instantiated as $\mathrm{POT}_{|\mathcal{U}|}^{|\mathcal{D}|+1}$. For all $u \in \mathcal{V}$, the TEE instance chooses $\{0\}$ and $\{b_v\}_{v\in\mathcal{D}}$ to obtain the auxiliary messages corresponding to $\widetilde{k}_u$ and $\{\widetilde{k}_{u,v}\}_{v\in\mathcal{D}}$, respectively. The process of this construction is the same as shown in Fig. 3 and Fig. 4, except for the following new additions and modifications:

- In the masking step, each client u sends the server
$$A_u = \mathrm{POT}_n^k.\mathsf{Send}(r, sk_u, pk_e, \{\widetilde{k}_u\}\cup\{\widetilde{k}_{u,v}\}_{v\in\mathcal{U}}),$$
and the server stores A_u received from distinct clients.
- In the unmasking step, the server parses $\bigcup_{u\in\mathcal{V}} B_u$ from reKey, runs $\mathrm{POT}_n^k.\mathsf{Proxy}(r, pk_u, pk_e, A_u, B_u)$ to recover $\widetilde{\mathsf{reKey}}_u = \{\widetilde{k}_u\}\cup\{\widetilde{k}_{u,v}\}_{v\in\mathcal{D}}$, and obtains the aggregated model $\sum_{u\in\mathcal{V}} \mathbf{x}_u = \sum_{u\in\mathcal{V}} \mathbf{y}_u - \mathsf{Decode}(\mathsf{pp}, \widetilde{\mathsf{reKey}}, r)$ where $\widetilde{\mathsf{reKey}} = \bigcup_{u\in\mathcal{V}} \widetilde{\mathsf{reKey}}_u$.

To reduce the number of pairwise masks, we can employ the sub-graph technique [5], [9] to generate fewer pairwise masks, directly reducing the values of n and k to improve the performance of our POT_n^k for one-pass aggregation.

C. Construction of Result Verification

Let $\mathsf{pp}_{\mathsf{com}} = (\mathbf{g}, h)$ be commitment public parameters where $\mathbf{g} = (g_1, \cdots, g_m) \xleftarrow{\$} \mathbb{G}^m$ and $h \xleftarrow{\$} \mathbb{G}$, and $F_0 : \mathbb{Z}_q \times \mathbb{Z}_q^* \to \mathbb{Z}_q$ be a secure PRF. Each client u can generate a random number $\mathsf{rand}_u = F_0(k_{u,e}, 0||r)$ where r is the round and $k_{u,e}$ is the shared key between client u and TEE instance M_e.

Commitments. We first extend the Pedersen commitment scheme [26] on vectors as follows:

- $\mathsf{com}_u \leftarrow \mathsf{Commit}(\mathbf{x}_u, \mathsf{rand}_u)$: outputs a model commitment $\mathsf{com}_u = \mathbf{g}^{\mathbf{x}_u} h^{\mathsf{rand}_u}$.
- $b \leftarrow \mathsf{Open}(\mathsf{com}_{u'}, \mathbf{x}_u, \mathsf{rand}_u)$: outputs $b = 1$ iff the equation $\mathsf{com}_{u'} = \mathsf{Commit}(\mathbf{x}_u, \mathsf{rand}_u)$ holds.
- $\mathsf{com} \leftarrow \mathsf{Eval}(\{\mathsf{com}_u\}_u)$: outputs an aggregated commitment $\mathsf{com} = \prod_u \mathsf{com}_u$.

TEE Verification. Upon loading $(\mathcal{V}, \mathcal{D}, \{\mathsf{com}_u, \sigma_u\}_u, r)$, TEE instance M_e performs the following checks to ensure that the aggregation range $\mathcal{V}$ is valid:

- assert $\mathsf{VerSig}(pk_u, \sigma_u, \mathsf{com}_u) = 1$ for all $u \in \mathcal{V}$.
- run RevealKey to obtain $\mathsf{rand} := \sum_{u\in\mathcal{V}} \mathsf{rand}_u$.
- run Eval to obtain $\mathsf{com} := \prod_{u\in\mathcal{V}} \mathsf{com}_u$.
- generate $\sigma_e := \mathsf{Sign}(sk_e, \mathsf{com}||\mathsf{rand}||r)$.
- output $(\sigma_e, \mathsf{com}, \mathsf{rand})$.

In practice, these TEE operations can be encapsulated within another TEE instance, which only requires data and code integrity to be protected (transparent enclave model [27]) and can be deployed on any other party equipped with TEE.

Proofs. Based on the above instantiations, the server can form a proof $\boldsymbol{\pi}^{\mathrm{valid}(\sum \mathbf{x}_u)} = (\mathsf{rand}, \sigma_e)$ and send it along with an aggregated model $\sum_{u\in\mathcal{V}} \mathbf{x}_u^{(r)}$ to all clients. Each client first computes a commitment $\mathsf{com}' = \mathbf{g}^{\sum_{u\in\mathcal{V}} \mathbf{x}_u^{(r)}} h^{\mathsf{rand}}$ and then verifies the signature using $\mathsf{VerSig}(pk_e, \sigma_e, \mathsf{com}'||\mathsf{rand}||r) = 1$ to confirm the authenticity of $\sum_{u\in\mathcal{V}} \mathbf{x}_u^{(r)}$.

V. Security Analysis and Discussion

Here, we first analyze the security of our framework given in Fig. 3 and Fig. 4, and then discuss its privacy attacks and robustness.

A. Security of OPSA

Following Definition 4.1 and Theorem 4.9 in SecAgg+ [5], we define a SA protocol as being α-secure if a client can be guaranteed that its private input is aggregated at most once with at least $\alpha \cdot |\mathcal{H}|$ other inputs from honest clients, where $\mathcal{H}$ is a list of all honest clients. Note that the following ideal functionality can be queried once for round $r \in [1, R]$.

Definition 4 (α-Summation Ideal Functionality)**.** *Let* q, m *be integers,* $\alpha \in [0, 1]$*,* $\mathcal{H} \subseteq \mathcal{U}$*, and* $\mathcal{X}_{\mathcal{H}} = \{\mathbf{x}_u\}_{u\in\mathcal{H}}$*, where* $\mathbf{x}_u \in \mathbb{Z}_q^m$*. For any subset* $\mathcal{L} \subseteq \mathcal{H}$*, the* α*-summation ideal functionality over* $\mathcal{X}_{\mathcal{H}}$ *for* $\mathcal{L}$ *is defined as follows:*

$$F_{\alpha,\mathcal{X}_{\mathcal{H}}}(\mathcal{L}) = \begin{cases} \sum_{u\in\mathcal{L}} \mathbf{x}_u & \text{if } |\mathcal{L}| \geq \alpha \cdot |\mathcal{H}|, \\ \perp & \text{otherwise.} \end{cases}$$

Then, the security of our protocol is given by the following theorems, and their full proofs, based on a standard hybrid argument, are presented in Appendix B.

Theorem 1 (Dropout resilience)**.** *Given secure cryptographic primitives and TEE, as described in Section II, a dropout rate* δ*, and a fraction* α *defined in Definition 4, the OPSA framework (Fig. 3 and Fig. 4), instantiated with Construction 1 or 2 with parameter* $(1-\delta) \geq \alpha$*, satisfies dropout resilience: when all parities follow the protocol, for round* $r \in [1, R]$*, all input vectors* $\mathcal{X}^{(r)}$*, and all sets of dropout clients* $\mathcal{D} \subseteq \mathcal{U}$ *with* $|\mathcal{D}| \leq \lfloor \delta N \rfloor$*, the server* S *outputs* $\sum_{u\in\mathcal{U}\setminus\mathcal{D}} \mathbf{x}_u^{(r)}$ *at the end of round* r*, except with negligible probability.*

Theorem 2 (Security)**.** *Given secure cryptographic primitives and TEE, as described in Section II, a corruption rate* γ*, and a fraction* α *defined in Definition 4, the OPSA framework (Fig. 3), instantiated with Construction 1 or 2 with parameter* $(1-\gamma) \geq \alpha$*, guarantees security: when there exists a PPT simulator* SIM *such that for any round* $r \in [1, R]$*, all input vectors* $\mathcal{X}^{(r)}$*, and all sets of corrupted parities* $\mathcal{C} \subseteq \mathcal{U}\cup\{S\}$ *of size* $|\mathcal{C}\setminus\{S\}| \leq \lfloor \gamma N \rfloor$*, the output of* SIM *is computationally indistinguishable from the view of a malicious adversary* $\mathcal{A}$*, i.e.,* $\mathsf{REAL}_{\mathcal{C},r}^{\mathcal{U},S,R}(\mathcal{A}, \{\mathbf{x}_u\}_{u\in\mathcal{U}\setminus\mathcal{C}}) \approx_c \mathsf{SIM}_{\mathcal{C},r}^{\mathcal{U},S,R,F_{\alpha,\{\mathbf{x}_u^{(r)}\}_{u\in\mathcal{U}\setminus\mathcal{C}}}}(\mathcal{A})$.

The next theorem states that OPSA achieves result authenticity against a malicious server. Let $\mathbf{z}_{\mathcal{H}} = \sum_{u\in\mathcal{U}\setminus\mathcal{C}} \mathbf{x}_u$ be the sum of honest survivors, $\mathbf{e}$ be a random vector, and $\mathcal{Z} = \{\mathbf{z}_r\}_{r\in R}$ be aggregation results to be sent to clients. We introduce $\mathsf{verify}(\mathcal{Z})$ to verify aggregation results of the server.

Theorem 3 (Result authenticity)**.** *Given secure cryptographic primitives and TEE, as described in Section II, a dropout*

存活列表 y 和丢弃列表 D，构建选择集 $\{b_v\}_{v\in\mathcal{D}}$，并计算辅助集 B_u=POTkn.选择 (r，sk_e, pk_u，$\{0\}_{\circ}\{b_v\}_{v\in\mathcal{D}}$) 及随机数 $\mathsf{rand}_u = F_0(k_{e,u}$，$0||r)$ 对于 $u \cdot V$. It outputs reKey = Uu $\cdot_V B_u$ *and* rand = $\Sigma_{u\cdot V}\mathsf{rand}_u$。

- $\mathbf{mask}_{\mathcal{U}}$ ←解码 (pp，r 密钥，r)：*服务器从* reKey *中获取已公开的密钥* $\{k_u, k_{u,v}\}_{u\cdot V,\ v\cdot D}$，*并输出所有掩码的求和值* $\mathbf{mask}_{\mathcal{U}}=\sum_{u\cdot V,\ v\cdot D}\big(G(\sim k_u)+\sum_{u<v}G(\sim ku，v)-\sum_{u>v}G(\sim ku，v)\big)$。

事实上，POTkn 被实例化为 POT。$\left|DU_{|}\right|+1$。对于所有$u \cdot V$，TEE实例选择 {0} 和一个 d $\{b_v\}_{v\in\mathcal{D}}$，分别获取对应于$k_u$的辅助消息以及 $\{k_{u,v}\}_{v\in\mathcal{D}}$的辅助消息。该构建过程与图 3 和图 4 所示相同，仅包含以下新增内容及修改：

- 在掩蔽步骤中，每个客户端u向服务器发送数据。
  A_u= POTkn.发送（r，sk_u, pk_e, $\{\widetilde{k}_u\}$。$\{\sim k_{u,v}\}_{v\in\mathcal{U}}$），服务器则存储从不同客户端接收到的$A_u$。
- 在解密步骤中，服务器从reKey中解析出Uu $\cdot_V B_u$，运行 POTkn.Proxy(r，pk_u, pk_e, A_u，Bu)以恢复$\mathsf{reKey}_u = \{k_u\}$。$\{k_{u,v}\}_{v\in\mathcal{D}}$，并获取聚合门控模型 $\Sigma_{u\in\mathcal{V}}\mathbf{x}_u=\Sigma_{u\in\mathcal{V}}\mathbf{y}_u$_Decode(pp，reKey，$r$)，其中reKey=Uu $\cdot_V \mathsf{reKey}_u$。

为减少成对掩码的数量，我们可以采用子图技术[5][9]来生成更少的成对掩码，直接降低n和k的值，从而提升我们POTkn在单次聚合过程中的性能。

C. 结果验证的构建

令$\mathsf{pp}_{\mathsf{com}}$= ($\mathbf{g}$，$h$) 为承诺公共参数，其中$\mathbf{g}$= ($g_1$，…，$gm$)。$\xleftarrow{\$}$ G ^{m}和$h\xleftarrow{\$}$ 设 G、$F_0 : \mathbb{Z}_q \times \mathsf{Z}_{rq} \to \mathbb{Z}_q$为一个安全的伪随机函数。每个客户端$u$均可生成随机数$\mathsf{rand}_u = F_0(k_{u,e}$，$0||r)$，其中$r$表示轮次，$k_{u,e}$为客户端$u$与TEE实例$M_e$之间的共享密钥。

承诺方案。我们首先将Pedersen承诺方案[26]扩展到向量领域，具体如下：

- com_u ←Commit($\mathbf{x}_u$，randu)：输出模型提交结果 ment $\mathsf{com}_u = \mathbf{g}^{\mathbf{x}_{uh}\mathsf{rand}_u}$。
- b ←Open($\mathsf{com}_{u'}, \mathbf{x}_u$，randu)：当且仅当方程$\mathsf{com}_{u'} = \mathsf{Commit}(\mathbf{x}_u$，randu) 成立时，输出$b$= 1。
- com ← Eval($\{\mathsf{com}_u\}_u$)：输出聚合后的承诺值 com = $\Pi_u \mathsf{com}_u$。

TEE验证。在加载（$\mathcal{V}$、$\mathcal{D}$、$\{\mathsf{com}_u, \sigma_u\}_u$、$r$）时，TEE实例$M_e$会执行以下检查以确保聚合范围$V$有效：

- 对所有$u \cdot V$，均满足$\mathsf{VerSig}(pk_u, \sigma_u, \mathsf{com}_u) = 1$。
- 运行 RevealKey 以获取 rand := $\Sigma_{u\cdot V}\mathsf{rand}_u$。
- 运行 Eval 以获取 com := $\Pi_{u\cdot V}\mathsf{com}_u$。
- 生成σ_e:= Sign(sk_e，com||rand||r)。
- 输出结果：σ_e、com、rand。

在实际应用中，这些TEE操作可封装于另一个TEE实例中，仅需保护数据和代码完整性（透明飞地模型[27]），并可部署于任何配备TEE的设备上。

证明。基于上述实例化结果，服务器可生成证明$\pi^{\mathsf{valid}(\Sigma xu)}$= (rand，$\sigma e$) 并将其与聚合模型$\sum_{u\in\mathcal{V}}\mathbf{x}_u^{(r)}$一并发送给所有客户端。每个客户端首先计算承诺值 com′ =$\mathbf{g}^{\sum_{u\cdot r}\mathbf{x}_u^{(r)}}h^{\mathsf{rand}}$，随后提出否决意见。rifi co这是该公司的标识。na a请使用印章。ing city $\mathsf{VerSig}(pk_e, \sigma_e, \mathsf{com}'||\mathsf{rand}||r) = 1$ of $\sum_{u\in\mathcal{V}}\mathbf{x}_u^{(r)}$.

V. 安全分析与讨论

首先，我们分析图3和图4所示框架的安全性；随后探讨其隐私攻击抵御能力及鲁棒性。

A. OPSA 安全性

根据SecAgg+[5]中的定义4.1和定理4.9，我们将SA协议定义为α安全的：若能保证客户端的私有输入最多仅与来自诚实客户端的至少$\alpha \cdot |H|$ 个其他输入进行聚合（其中H为所有诚实客户端的列表）。需注意，以下理想功能可在轮次$r \cdot [1，R]$内查询一次。

定义 4（α-求和理想函数性）。*设 q、m 为整数，$\alpha \cdot [0, 1]$，$H\subseteq U$，and $\mathcal{X}_{\mathcal{H}} = \{\mathbf{x}_u\}_{u\in\mathcal{H}}$，where $\mathbf{x}_u \cdot \mathsf{Zmq}$. For any subset $L\subseteq H$，则定义 L 上 $\mathcal{X}_{\mathcal{H}}$的 α-求和理想函数性如下：*

$$F_{\alpha,\mathcal{X}_{\mathcal{H}}}(\mathcal{L}) = \begin{cases} \sum_{u\in\mathcal{L}}\mathbf{x}_u & \text{if } |\mathcal{L}| \geq \alpha\cdot|\mathcal{H}|, \\ \perp & \text{otherwise.} \end{cases}$$

随后，我们协议的安全性由以下定理予以证明；基于标准混合论证方法的完整证明详见附录B。

定理 1（丢弃抗性）。*给定如第 II 节所述的安全密码学原语和 TEE、一个丢弃率 δ 以及定义 4 中给出的分数 α，当使用构造 1 或 2 并设置参数 $(1_\delta) \geq \alpha$ 时，OPSA 框架（图 3 和图 4）满足丢弃抗性：当所有奇偶校验均遵循协议时，在轮次 $r \cdot [1，R]$ 的结束时，所有输入向量 $X^{(r)}$ 以及所有丢弃客户端集合 $D\subseteq U$ with $|D| \leq \mathsf{q}\delta N\mathsf{q}$，the server S outputs $\sum_{u\in\mathcal{U}\setminus\mathcal{D}}\mathbf{x}_u^{(r)}$ 均成立，除非概率可忽略不计。*

定理 2（安全性）。*给定如第II节所述的安全密码学原语和 TEE、一个错误率 γ 以及定义4中给出的分数 α，当使用构造1或构造2（参数 $1_\gamma \geq \alpha$）实例化 OPSA 框架（图3）时，该框架能保证安全性：存在一个PPT模拟器* SIM，*使得对于任意轮次 $r \cdot [1，R]$、所有输入向量 $X^{(r)}$ 以及所有被破坏的奇偶校验集 $C\subseteq U_{\circ}\{S\}$ of size $|C\setminus\{S\}| \leq \mathsf{q}\gamma N\mathsf{q}$，* SIM *的输出在计算上与恶意攻击者 A 的观测结果无法区分，即* REALUC，$_{Sr'R}(\mathcal{A}，\{\mathbf{x}_u\}_{u\in\mathcal{U}\setminus\mathcal{C}}) \approx_c$ SIM ^{UC}，，$_{Sr'R,Fa,\{xu^{(r)}\}_{u\cdot U}\setminus C(\mathcal{A})}$）。

下一个定理表明 OPSA 能够在面对恶意服务器时确保结果的真实性。设$\mathbf{z}_{\mathcal{H}}=\Sigma_{u\cdot U\setminus C}\mathbf{x}_u$为诚实存活节点的总和，$\mathbf{e}$为随机向量，$Z$=$\{\mathbf{z}_r\}_{r\in R}$为需发送给客户端的聚合结果。我们引入验证函数($Z$)用于验证服务器生成的聚合结果。

定理3（结果真实性）。*给定如第II节所述的安全密码学原语*

rate δ, and a fraction α defined in Definition 4, we say that the OPSA framework (Fig. 4), with parameter $(1-\delta) \geq \alpha$, achieves security (Theorem 2) and result authenticity in the malicious setting for any round $r \in [1, R]$, all $\mathcal{X}_{\mathcal{U}}$, and $\mathcal{C}$ with $|\mathcal{C}| \leq \lfloor \gamma N \rfloor$, if a PPT adversary $\mathcal{A}$ can get clients to accept its forged output with a negligible probability,

$$\Pr\left[\mathsf{verify}(\mathbf{z}_{\mathcal{A}}) = 1 \;\middle|\; \begin{array}{l} \mathbf{z}_{\mathcal{A}} = \mathbf{z}_{\mathcal{H}} + \sum_{u \in \mathcal{C}} \mathbf{x}_u + \mathbf{e} \\ \mathsf{verify}(\mathbf{z}_{\mathcal{H}}) = 1,\ \mathbf{z}_{\mathcal{H}} \neq \mathbf{z}_{\mathcal{A}} \end{array}\right] \leq \mathsf{negl}(\lambda).$$

B. Privacy Attacks

Here, we discuss privacy attacks on SA protocols and analyze how to resist them within the OPSA framework. Fowl et al. [13] make changes to the shared model architecture on a malicious server, enabling it to directly access precise copies of user data from gradient updates. Pasquini et al. [14] manipulate model parameters on a malicious server to execute two privacy attacks targeting SA protocols. Zhao et al. [15] demonstrate how the privacy of SA protocols in FL can be compromised by a malicious server sending customized models to clients. It is evident that such attacks are initiated by a malicious server modifying the model, and result verification of the OPSA framework can prevent these attacks.

Furthermore, the work [28] proposes a privacy attack that leverages the aggregated models and participation information across multiple training rounds to reconstruct individual models, leading to privacy leakage. To counteract this attack, the OPSA framework requires hiding client participation information, either by enhancing the structured strategy for client selection [28] or mandating clients to form minimum batches [29] for uploading models. Another study [30] introduces $t-1$ sybil devices to assist a malicious server in reconstructing individual models, and the OPSA framework can prevent it by setting a low dropout rate $\delta \in [0, 1 - \frac{t-1}{N}]$.

In our threat model, we consider two scenarios: (1) up to δN honest clients may drop out during any step, and (2) up to γN clients may be corrupted before each aggregation. Our scheme is designed to handle any dropout rate $\delta \in [0, 1]$, as the enclave securely stores shared keys for all clients, enabling the server to remove these masks from any subset of clients. However, a malicious server can simulate client dropouts by arbitrarily disrupting communication during any round, excluding up to $(\delta + \gamma)N$ clients from the final aggregation. Since clients cannot distinguish real from simulated dropouts, we require corruption fraction $\gamma \in [0, 1 - \delta]$ and utilize TEE to check $|\mathcal{V}| \geq \lceil (1-\delta)N \rceil$, where $\mathcal{V}$ is a surviving client list. This condition prevents the malicious server, controlling γN corrupted clients, from inferring any client's private input. To guarantee that an aggregation result includes contributions from at least $c \geq 3$ honest clients, this condition is expressed as $1 - \delta - \gamma \geq \frac{c}{N}$, with $\gamma \in [0, 1 - \delta]$. For instance, if the dropout rate is set to $\delta < 0.4$, our scheme provides privacy against at most $\gamma < 0.6$ fraction of clients being corrupted.

C. Extension with Robustness

OPSA relies on the integrity and confidentiality of TEE to process the clients' temporary secret keys. At the core of TEE, memory isolation ensures that the code, data, and runtime states of enclaves are inaccessible to untrusted entities, typically implemented through static partitioning mechanisms, such as the fixed-size Processor Reserved Memory in Intel SGX and the secure world memory in ARM TrustZone. Additionally, enclave management uses hardware instructions to control the life cycle of enclaves, including their creation, execution, and teardown. Specifically, enclaves are mapped to an encrypted physical memory area called Enclave Page Cache (EPC), and their life cycle is closely tied to enclave resource management, especially EPC page allocation. Due to memory usage of model aggregation exceeding EPC limits, previous TEE-based SA schemes often encounter EPC paging, leading to performance bottlenecks. In contrast, OPSA decouples TEE computations from aggregated model size and minimizes input and cryptographic operations within TEE, enabling efficient aggregation of larger models without exceeding EPC limits, indirectly reducing the attack surface. For scenarios where EPC limits might still be exceeded (e.g., handling numerous clients), OPSA can integrate strategies like dynamic memory management [31], [32] and enclave management monitors [33], [34] to enhance efficiency and robustness.

Although OPSA follows the assumption of SEAR [11] (i.e., TEE is fully trusted), its practical deployment must consider potential risks like compromised enclaves, physical attacks, and side-channel attacks. Since OPSA uses TEE solely for handling temporary keys, it remains compatible with various TEE frameworks, allowing the integration of existing approaches to mitigate these threats. For instance, tools (e.g., SgxMonitor [35]) can detect malicious enclave behavior, and mechanisms like HyperEnclave [36] can defend against compromised enclaves by preventing arbitrary memory accesses and control flows. Furthermore, hardware memory encryption techniques, such as AMD SME and Intel MKTME, could be used to protect enclaves from physical attacks [36]. To thwart side-channel attacks, OPSA can employ side-channel resistant cryptographic libraries [37] to safeguard secret keys, frameworks like eOPF [33] to implement defenses at a resource level (e.g., page tables, caches), and tools like OBELIX [38] to provide software-level protections through oblivious code execution and data accesses. These countermeasures, inherently tied to specific TEE implementations, enable OPSA to achieve robustness and comprehensive protection.

While the OPSA framework can prevent the server from tampering with the aggregation results when all clients are semi-honest, challenges arise when clients exhibit malicious behavior. In cases like model poisoning attacks, which involve intentional submission of errors or malicious models, the OPSA framework cannot guarantee the validity of the final aggregated model. To enhance client robustness, the OPSA framework can further integrate existing strategies to counteract these attacks. One strategy is to permit the server to aggregate models on a small scale, followed by scoring the aggregation results using cosine similarity [39] to identify malicious clients. Another approach is to prove that the model input satisfies the L_∞ specification through zero-knowledge proofs [7]. However, when both the server and client may act maliciously simultaneously, it is essential to combine

和TEE、一个丢弃率δ以及定义4中给出的比例α，我们称：当参数$(1-\delta)\geq\alpha$时，OPSA框架（图4）在恶意攻击场景下对于任意轮次$r\in[1,R]$，all $\mathcal{X}_{\mathcal{U}}$，and C with $|C|\leq q\gamma N$ q均能实现安全性（定理2）与结果真实性——前提是PPT攻击者A能够使客户端以可忽略的概率接受其伪造输出。

$$\Pr\left[\begin{array}{l|l}\mathsf{verify}(\mathbf{z}_{\mathcal{A}})=1 & \mathbf{z}_{\mathcal{A}}=\mathbf{z}_{\mathcal{H}}+\sum_{u\in\mathcal{C}}\mathbf{x}_u+\mathbf{e}\\ & \mathsf{verify}(\mathbf{z}_{\mathcal{H}})=1,\ \mathbf{z}_{\mathcal{H}}\neq\mathbf{z}_{\mathcal{A}}\end{array}\right]\leq\mathsf{negl}(\lambda).$$

B. 隐私攻击

本文中，我们探讨了针对SA协议的隐私攻击，并分析了如何在OPSA框架内抵御此类攻击。Fowl等人[13]对恶意服务器上的共享模型架构进行了修改，使其能够直接从梯度更新中获取用户数据的精确副本；Pasquini等人[14]则通过篡改恶意服务器上的模型参数，实施了两种针对SA协议的隐私攻击；Zhao等人[15]证明了恶意服务器向客户端发送定制化模型会破坏联邦学习中SA协议的隐私性。显然，此类攻击均源于恶意服务器对模型的篡改，而对OPSA框架的验证机制可有效防范这些攻击。

此外，文献[28]提出了一种隐私攻击方法：该方法利用多轮训练中聚合的模型数据及参与信息来重建个体模型，从而导致隐私泄露。为应对此类攻击，OPSA框架要求隐藏客户端参与信息，具体可通过改进客户端选择的结构化策略[28]，或强制客户端上传模型时使用最小批量[29]。另一项研究[30]提出了$t-1$个Sybil设备以协助恶意服务器重建个体模型；而OPSA框架可通过设置较低的丢弃率$\delta\in[0,1-\frac{t-1}{N}]$来防范此攻击。

在我们的威胁模型中，我们考虑两种场景：(1) 在任何步骤中最多有δN个诚实客户端可能退出；(2) 在每次聚合前最多有γN个客户端可能被篡改。我们的方案旨在处理任意退出率$\delta\in[0,1]$，因为隔离域安全地存储了所有客户端的共享密钥，使服务器能够从任意客户端子集中移除这些掩码。然而，恶意服务器可通过在任意轮次中任意干扰通信来模拟客户端退出，从而将最多$(\delta+\gamma)N$个客户端排除在最终聚合之外。由于客户端无法区分真实退出与模拟退出，我们要求篡改比例$\gamma\in[0,1-\delta]$，并利用TEE验证$|V|\geq(1-\delta)N$（其中V为存活客户端列表）。该条件可防止控制γN个被篡改客户端的恶意服务器推断出任何客户端的私有输入。为确保聚合结果至少包含$c\geq 3$个诚实客户端的贡献，该条件可表示为$1-\delta-\gamma\geq\frac{c}{N}$其中$\gamma\in[0,1-\delta]$。例如，若将丢弃率设置为$\delta<0.4$，则我们的方案最多可保护$\gamma<0.6$比例的客户端数据不被篡改。

C. 具有鲁棒性的扩展功能

OPSA依赖TEE的完整性和保密性来处理客户端的临时密钥。TEE的核心机制在于内存隔离技术，它确保飞地内的代码、数据及运行状态对不可信实体不可访问——这一机制通常通过静态分区机制实现，例如英特尔SGX中的固定大小处理器保留内存以及ARM TrustZone中的安全世界内存。此外，飞地管理采用硬件指令来控制飞地的生命周期（包括创建、执行和销毁）。具体而言，飞地被映射到一个名为飞地页缓存（EPC）的加密物理内存区域，其生命周期与飞地资源管理（尤其是EPC页面分配）紧密关联。由于模型聚合所需的内存使用量常超出EPC限制，传统的基于TEE的安全协议方案常面临EPC页面溢出问题，从而导致性能瓶颈。相比之下，OPSA将TEE计算与模型规模相分离，并最大限度地减少TEE内部的输入操作和加密运算，从而能够高效地聚合大型模型而不超出EPC限制，间接缩小了攻击面。对于仍可能超出EPC限制的场景（例如处理大量客户端时），OPSA可集成动态内存管理[31]、[32]以及飞地管理监控器[33]、[34]等策略，以提升系统效率与稳健性。

尽管OPSA遵循SEAR[11]的假设（即TEE可完全可信），但其实际部署时仍需考虑诸如安全飞地被攻破、物理攻击及侧信道攻击等潜在风险。由于OPSA仅使用TEE处理临时密钥，因此仍与多种TEE框架兼容，可整合现有方案以缓解这些威胁。例如，工具（如SgxMonitor[35]）能够检测恶意安全飞地行为；而HyperEnclave[36]等机制可通过阻止任意内存访问和控制流来防御安全飞地被攻破的情况。此外，硬件内存加密技术（如AMD SME和Intel MKTME）可用于保护安全飞地免受物理攻击[36]。为抵御侧信道攻击，OPSA可采用抗侧信道密码库[37]保护密钥；利用eOPF[33]等框架在资源层面（如页表、缓存）实施防护；或借助OBELIX[38]等工具通过无感知代码执行和数据访问提供软件级保护。这些对策与特定TEE实现方案紧密相关，使OPSA能够实现高稳健性与全面防护。

虽然OPSA框架能够在所有客户端均半诚实的情况下防止服务器篡改聚合结果，但当客户端表现出恶意行为时便会面临挑战。在模型中毒攻击（即故意提交错误或恶意模型）等情况下，OPSA框架无法保证最终聚合模型的有效性。为增强客户端的鲁棒性，OPSA框架可进一步整合现有策略来抵御此类攻击：一种策略是允许服务器进行小规模模型聚合，随后利用余弦相似度[39]对聚合结果进行评分以识别恶意客户端；另一种方法则是通过零知识证明[7]验证模型输入满足L_∞规范要求。然而，当服务器与客户

TABLE I: Comparison of communication (Comm.) and computation (Comp.) overhead in a secure aggregation round.

Setting	**Flamingo**	**POT-OPSA**	**LERNA**	**khPRF-OPSA**
Client Comm.	Regular client: $O(m+L+A)$ Decryptor: $O(L+\delta AN+(1-\delta)N)$	$O(m+A)$	Regular client: $O(m)$ Committee: $O(m+M)$	$O(m)$
Client Comp.	Regular client: $O(mA+L)$ Decryptor: $O(\delta AN+(1-\delta)N+\epsilon N^2)$	$O(mA)$	Regular client: $O(m\log\ell)$ Committee: $O(m\log\ell+\delta N\ell)$	$O(m\log\ell)$
Server Comm.	$O((m+L+A)N)$	$O((m+A)N)$	$O(m(N+M))$	$O(mN)$
Server Comp.	$O(t^2+\delta mAN+(1-\delta)mN)$	$O(\delta mAN+(1-\delta)mN+\epsilon N^2)$ TEE: $O((1-\delta)AN)$	$O((1-\delta)mN+t^*m)$	$O((1-\delta)mN+m\log\ell)$ TEE: $(1-\delta)N\ell$
C/S Comm.	3	1	3	1

Remark N: the number of clients, m: the model vector size, δ: the dropout rate. C/S Comm. denotes the number of client-server round-trip communications.
Flamingo A: the number of neighbors of a client, L: the number of decryptors, ϵ: graph generation parameter, t: threshold of Shamir secret sharing.
LERNA M: the number of committee members, ℓ: the key size of KhPRF, t^*: threshold of flat secret sharing.

result verification with a robust defense strategy to ensure the authenticity of the final aggregation result.

VI. Evaluation

In this section, we compare KhPRF-OPSA and POT-OPSA with state-of-the-art schemes, conducting theoretical analysis and performance analysis, respectively.

A. Theoretical Analysis

The number of client-server communications has a critical impact on the overall execution time of SA protocols. In real-world FL training scenarios across wide area networks, the round-trip time (RTT) delay can reach tens of milliseconds, resulting in notable differences in the arrival time of messages from different clients. Consequently, the server spends considerable waiting time in each communication round.

Table I provides a comprehensive comparison of OPSA's communication and computation costs with those of Flamingo and LERNA. Compared with double masking schemes, the computation cost of single masking schemes increases more obviously with the model dimension. This is because single mask computations involve the multiplication of polynomials of the model dimensions by the Ring-LWR key polynomials, requiring a computational complexity of $O(m\log\ell)$ with NTT acceleration. In contrast, double mask calculations can be regarded as a PRG instantiated by AES encryption, involving an $O(m)$ computational complexity. When considering server computation cost, double masking schemes are more expensive than single masking schemes, especially in scenarios with a large number of dropped clients, which require recovering their pairwise masks. Moreover, single masking schemes support dynamic client access. Overall, among the two proposed constructions, KhPRF-OPSA is more suitable for scenarios with low-dimensional models and more dropped clients, while POT-OPSA is better suited for situations with high-dimensional models and fewer dropped clients.

B. Experimental Setup

Implementation. The OPSA framework is implemented in Python, and the TEE code is developed in C++. In our implementation, we leverage the Intel SGX SDK to construct trusted applications inside the enclave, instantiate cryptographic primitives as detailed in subsection II-C, and employ the NIST P-256 curve. All trainable parameters of each client's model can be considered as a high-dimensional input vector, with the model dimension corresponding to the size of the input vector. For experiments evaluating the performance of SA protocols, we extract fixed-size segments from the real model parameters to serve as client input vectors. Meanwhile, the entire model parameters are used as client input vectors to evaluate model performance and accuracy. For the input vector, each element is set to 64 bits, requiring the encoding of floating-point model weights into 64 bits. To achieve a 128-bit security level, we set the key size to 128 bits for AES-CTR, configure the RLWR dimension as 2^{11}, and set the modulus q to $n\Delta\cdot 2^{286}$, consistent with the LERNA setting. The hardness of this configuration can be validated by the evaluator [40].

Experimental environment. We integrate all of our code into the ABIDES simulation framework [41] and run simulation on a machine equipped with a 2.90GHz Intel Core i7-10700 CPU and 32GB memory, running 64-bit Ubuntu 20.04.2 LTS. Moreover, this machine supports Intel SGX, and its protected physical memory region, known as Processor Reserved Memory (PRM), is limited to 128MB. Our baseline is set as the current state-of-the-art solutions, Flamingo and LERNA. Since the committee members or decryptors are selected from the clients, we set them to the same dropout rate. To ensure an equivalent level of security, Flamingo is set with 60 decryptors for a 1% dropout rate and 170 decryptors for a 5% dropout rate, while LERNA's committee size is consistently set to 2^{14}. Unless otherwise specified, the default dropout rate of clients in our experiments is set as $\delta = 5\%$.

Training setting. We conduct extensive experiments using common datasets, MNIST and CIFAR-10. MNIST comprises 60K training images and 10K testing images of handwritten digits across 10 classes. CIFAR-10 consists of 60K total samples, each being a 32×32 pixel RGB image, divided into 50K training samples and 10K testing samples. The number of SA rounds for the global model is set to 50 for MNIST and 200 for CIFAR-10. In every SA round, each client undergoes 5 iterations of local mini-batch training, with batch size 32 and learning rate 0.01. To simulate realistic FL training, we distribute the training data among clients in a non-IID

表I：安全聚合轮中通信（Comm.）与计算（Comp.）开销的比较。

设置	红鹳	POT-OPSA	LERNA	khPRF-OPSA
客户委员会。	常规客户端：$O(m+L+A)$；解密器：$O(L+\delta AN+(1-\delta)N)$	$O(m+A)$	常客：$O(m)$ 委员会：$O(m+M)$	$O(m)$
客户公司。	常规客户端：$O(mA+L)$ 解密器：$O(\delta AN+(1-\delta)N+\epsilon N^2)$	$O(mA))$	常规客户端：$O(m\log \ell)$ 委员会：$O(m\log \ell+\delta N\ell)$	$O(m\log \ell)$
服务器通信。	$O(m+L+A)N)$	$O(m+A)N)$	$O(m(N+M))$	$O(mN)$
服务器组件。	$O(t^2+\delta mAN+(1-\delta)mN)$	$O(\delta mAN+(1-\delta)mN+\epsilon N^2)$ TEE：$O((1-\delta)AN)$	$O((1-\delta)mN+t^*m)$	$O(11-\delta)mN+m\log \ell)$ TEE：$(1-\delta)N\ell$
C/S 委员会。	3	1	3	1

注N：客户端数量；m：模型向量大小；δ：丢弃率。C/S Comm. 表示客户端与服务器之间的往返通信次数。
Flamingo A：客户端的邻居数量；L：解密者数量；ϵ：图生成参数；t：Shamir秘密共享阈值。
LERNA M：委员会成员数量；ℓ：KhPRF密钥长度；t^*：平坦秘密共享的阈值。

端可能同时存在恶意行为时，必须将结果验证与稳健防御策略相结合，以确保最终聚合结果的真实性。

第六节。评估

在本节中，我们分别通过理论分析和性能分析，将KhPRF-OPSA 与POT- OPSA 与当前最先进的方案进行比较。

A. 理论分析

客户端与服务器之间的通信次数对SA协议的整体执行时间具有决定性影响。在广域网环境下的实际联邦学习训练场景中，往返时间（RTT）延迟可能高达数十毫秒，导致来自不同客户端的消息到达时间存在显著差异。因此，服务器在每轮通信过程中都需要耗费大量等待时间。

表I全面比较了 OPSA 与Flamingo及 LERNA 在通信量和计算成本方面的差异。相较于双掩码方案，单掩码方案的计算成本随模型维度增加而显著上升。这是因为单掩码计算需要将模型维度对应的多项式与Ring- LWR 密钥多项式相乘，在NTT加速条件下其计算复杂度为$O(m\log \ell)$；而双掩码计算可视为基于AES加密实现的程序实例化方案，其计算复杂度仅为$O(m)$。从服务器端计算成本来看，双掩码方案比单掩码方案更为昂贵，尤其是在需要恢复大量丢失客户端成对掩码的情况下；此外，单掩码方案支持动态客户端访问功能。总体而言，在两种提出的架构方案中，KhPRF- OPSA 更适用于低维模型且丢弃客户端较多的场景；而POT- OPSA 则更适合高维模型且丢弃客户端较少的情况。

B. 实验设置

实现。OPSA 框架采用Python语言实现，TEE代码则使用C++开发。在我们的实现中，我们利用Intel SGX SDK在安全隔离区内构建可信应用程序，按照II-C小节所述方式实例化加密原语，并采用NIST P-256曲线。每个客户端模型的所有可训练参数均可视为高维输入向量，其模型维度与输入向量大小相对应。在评估SA协议性能的实验中，我们从真实模型参数中提取固定长度片段作为客户端输入向量；同时，亦可使用完整模型参数作为客户端输入向量以评估模型性能与准确率。对于输入向量，每个元素均设置为64位，需将浮点型模型权重编码为64位。为实现128位安全级别，我们为AES-CTR算法设置密钥长度为128位，配置 RLWR 维度为2^{11}，并将模数q设为$n\,\Delta\cdot 2^{286}$，这与 LERNA 设置要求一致。该构型的硬度可由评估人员[40]进行验证。

实验环境。我们将所有代码集成到abides仿真框架[41]中，并在配备2.90GHz英特尔酷睿i7-10700处理器和32GB内存、运行64位Ubuntu 20.04.2 LTS系统的机器上进行仿真。该机器支持Intel SGX 架构，其受保护的物理内存区域（称为处理器保留内存PRM）上限为128MB。我们的基准测试方案采用当前最先进的Flamingo和 LERNA 两种方法。由于委员会成员或解密者均从客户端中选取，因此设置相同的丢弃率：Flamingo使用60个解密者对应1%丢弃率，使用170个解密者对应5%丢弃率；而 LERNA 的委员会规模始终设定为2^{14}。除非另有说明，实验中客户端的默认丢弃率为δ=5%。

训练环境。我们使用常见的数据集 MNIST 和 CIFAR -10 进行了大量实验。 MNIST 包含涵盖10个类别的6万张手写数字训练图像和1万张测试图像； CIFAR -10则包含总计6万份样本，每份均为32 × 32像素的RGB图像，其中5万份作为训练样本，1万份作为测试样本。全局模型在 MNIST 上的优化轮数设置为50轮，在 CIFAR -10上则设置为200轮。每个优化轮次中，所有客户端均需进行5次局部小批量训练（批量大小为32，学习率设为0.01）。为模拟真实的联邦学习训练场景，我们采用非 IID 方式将训练数据分配给各客户端，并

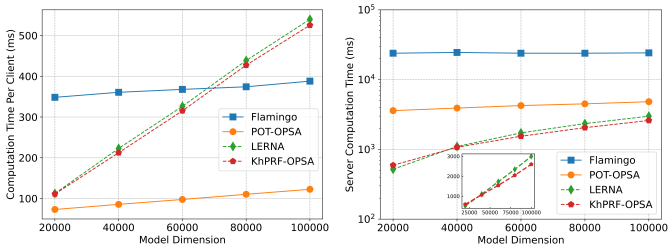
setting, using a Dirichlet distribution with hyperparameter 0.9 as described in [42].

To perform our experiments on the aforementioned datasets, we employ two models based on the classic CNN structure. For MNIST, the model includes two 5×5 convolution layers with ReLU activation, max pooling, and dropout. It also includes a ReLU-activated dense layer and a softmax-activated output layer, totaling $21,840$ trainable parameters. For CIFAR-10, the model contains $551,722$ parameters, consisting of three pairs of ReLU-activated convolution layers with batch normalization after each layer, and max pooling and dropout after each pair. Finally, the model also includes a ReLU-activated dense layer and a softmax-activated output layer.

C. Experimental Performance

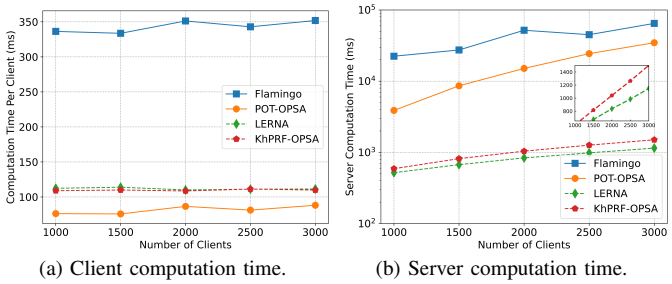
Here, we provide a benchmark to compare our two types of SA protocols with Flamingo and LERNA. For detailed comparison, we divide the entire protocol into five stages: "setup" (one-time setup phase), "mask" (masked model generation and upload), "check" (consistency check), "unmask" (aggregated model recovery), and "verify" (result verification).

Computation costs. To fairly evaluate the computation time of clients and server, we only choose the regular clients of Flamingo and LERNA for comparison with our scheme. Note that the server computation time of our scheme includes the execution time within TEE. For computation time measurements, we report an average over 10 experiment runs.



(a) Client computation time. (b) Server computation time.

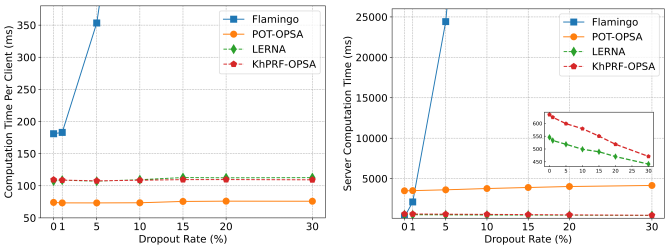
Fig. 5: Computation time vs. model dimension, with fixed number of clients $N = 1$K and dropout rate $\delta = 5\%$.



(a) Client computation time. (b) Server computation time.

Fig. 6: Computation time vs. number of clients, with fixed model dimension $m = 20$K and dropout rate $\delta = 5\%$.

Fig. 5 and Fig. 6 show the computation time for different model dimensions and different number of clients respectively. For double masking schemes, the client computation time of POT-OPSA is notably shorter than that of Flamingo. This advantage arises from the fact that Proxy-OT in our scheme utilizes simple XOR calculations instead of complex encryption operations. Moreover, at the server side, the process of mask recovery in POT-OPSA is significantly lighter compared to Flamingo, highlighting the efficiency of our approach. For single masking schemes, the server computation time of KhPRF-OPSA is slightly higher than that of LERNA due to the additional execution time required by the RevealKey operation within TEE, which is calculated by the committee in LERNA. However, when the model dimension is larger, our server computation time will be more advantageous, because TEE aggregates all low-dimensional secret keys, while LERNA aggregates all high-dimensional masks received from the committee. Overall, the computation time of KhPRF-OPSA is comparable to that of LERNA.



(a) Client computation time. (b) Server computation time.

Fig. 7: Computation time vs. dropout rate, with fixed number of clients $N = 1$K and model dimension $m = 20$K.

As shown in Fig. 7, Flamingo's computation time increases significantly as the dropout rate rises. This can be attributed to the varying number of decryptors required by Flamingo to maintain a same level of security across different dropout rates. In contrast, the client-side computation time for other schemes remains relatively stable, since the number of LERNA's committee is fixed, and our scheme only requires to interact with TEE. At the server side, double masking schemes experience a slight increase in computation time with the dropout rate due to their involvement in recovering pairwise masks of dropped clients. In contrast, the computation time of single masking schemes decreases as the dropout rate increases, as the server doesn't need to recover masks for dropped clients. While maintaining the same security level, our scheme can tolerate any dropout rate $\delta \in [0, 1]$, whereas Flamingo and LERNA are limited to smaller ranges of $\delta \in [0, \frac{1}{3})$ and $\delta \in [0, \frac{1}{2})$, respectively.

We compare our scheme with Flamingo and LERNA across various stages of a SA round, and their computation times are presented in Table II. The key advantage of our scheme lies in its significant reduction of the overall computation time. This efficiency is achieved by transforming the complex recovery operation carried out by Flamingo's decryptors and LERNA's committee members into a concise and lightweight TEE responsible for performing the recovery of secret seeds. Moreover, our scheme provides verifiable aggregation results, and the detailed computation times are provided in Table III.

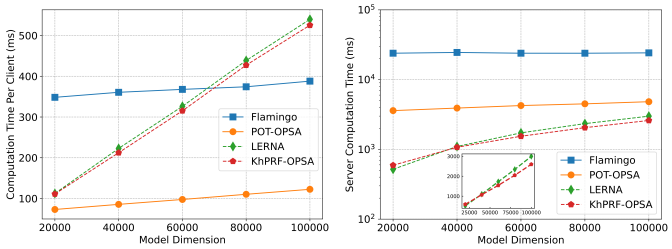
使用超参数值为0.9的狄利克雷分布进行分配（详见[42]）。

为在上述数据集上开展实验，我们采用了两种基于经典CNN架构的模型。对于 MNIST 数据集，该模型包含两个5 × 5卷积层（均采用ReLU激活函数）、最大池化层及丢弃层；同时配备一个ReLU激活的全连接层和一个softmax激活输出层，共计21,840个可训练参数；而对于 CIFAR -10 数据集，模型参数量为551,722个，由三对ReLU激活卷积层组成（每层后均接批归一化操作），每对卷积层后均配置最大池化层与丢弃层；最终模型同样包含一个ReLU激活全连接层和一个softmax激活输出层。

C. 实验性能

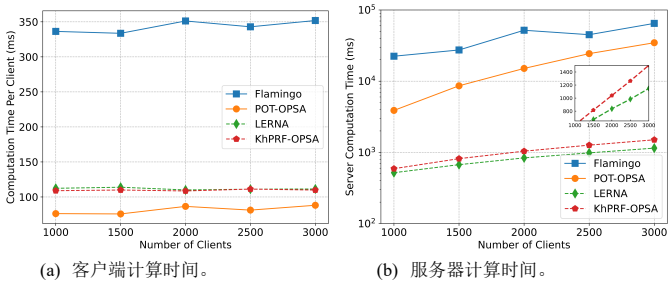
本文提供了一个基准测试框架，用于将我们开发的两种SA协议与Flamingo和 LERNA 进行对比。为实现详细比较，我们将整个协议流程划分为五个阶段：“设置”（一次性配置阶段）、“掩码处理”（生成并上传掩码模型）、“验证”（一致性检查）、“解掩码”（聚合模型恢复）以及“确认”（结果验证）。

计算成本。为公平评估客户端与服务器的计算时间，我们仅选取Flamingo和 LERNA 的常规客户端与我们的方案进行对比。需注意，本方案中的服务器计算时间已包含TEE环境内的执行时间。计算时间测量结果基于10次实验运行的平均值。



(a) 客户端计算时间。 (b) 服务器计算时间。

图5：计算时间与模型维度的关系，其中客户端数量N=1000且丢弃率δ=5%。

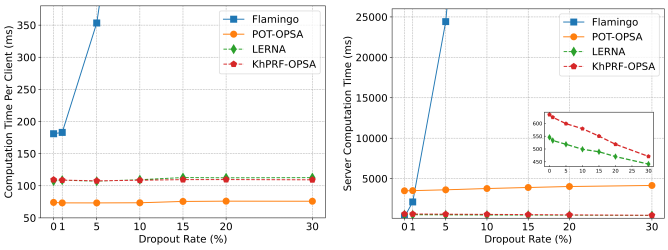


(a) 客户端计算时间。 (b) 服务器计算时间。

图6：计算时间与客户端数量的关系（模型维度m=20K，丢弃率δ=5%）。

图5和图6分别展示了不同模型维度及不同客户端数量下的计算时间。

对于双重掩码方案而言，POT- OPSA 的客户端计算时间显著短于Flamingo。这一优势源于我们方案中的Proxy-OT采用简单的 XOR 运算而非复杂的加密操作；此外，在服务器端，POT- OPSA 的掩码恢复过程也明显轻量化，充分体现了我们方法的高效性。对于单掩码方案，KhPRF-OPSA 的服务器计算时间略高于 LERNA，这是由于TEE中RevealKey操作需要额外执行时间（该时间由 LERNA 中的委员会计算）。然而当模型维度较大时，我们的服务器计算时间更具优势——因为TEE会聚合所有低维秘密密钥，而 LERNA 则需聚合从委员会接收的所有高维掩码。总体而言，KhPRFOPSA的计算时间与 LERNA 相当。



(a) 客户端计算时间。 (b) 服务器计算时间。

图7：计算时间与丢弃率的关系，其中客户端数量N=1000，模型维度m=20000。

如图7所示，随着丢包率的升高，Flamingo的计算时间显著增加。这归因于Flamingo为在不同丢包率下保持相同安全水平所需解密器数量的变化。相比之下，其他方案的客户端计算时间保持相对稳定，因为 LERNA 委员会的规模固定，且我们的方案仅需与TEE交互。在服务器端，双掩码方案由于需要恢复丢失客户端的成对掩码，其计算时间会随丢包率上升而略有增加；而单掩码方案的计算时间则随丢包率增加而下降，因为服务器无需恢复丢失客户端的掩码。在保持相同安全水平的前提下，我们的方案可容忍任意丢包率$\delta \in [0, 1]$，而Flamingo和 LERNA 方案则受限于较小范围$\delta \in [0, \frac{1}{3})$且$\delta \in [0, \frac{1}{2})$, 各自地.

我们将我们的方案与Flamingo和 LERNA 在安全聚合轮次的各个阶段进行对比，其计算时间如表II所示。我们方案的核心优势在于显著缩短了整体计算时间——这一效率提升是通过将Flaminggo解密器及 LERNA 委员会成员执行的复杂恢复操作转化为一个简洁轻量化的可信执行环境（TEE），专门负责秘密种子的恢复来实现的。此外，我们的方案还能提供可验证的聚合结果，具体计算时间详见

TABLE II: Computation time for different stages per round of secure aggregation with $N = 1$K, $m = 20$K, and $\delta = 5\%$.

Scheme	Type	Stages for secure aggregation (ms)				
		setup	mask	check	unmask	verify
Flamingo	Server	40.37	335.75	7.94	24397.08	–
	Client	–	355.34	–	–	–
	Decryptor	7143.84	–	4.44	833.11	–
POT-OPSA	Server	1192.60	–	–	3914.28	–
	TEE	178.64	–	–	13.28	122.60
	Client	0.89	79.23	–	–	64.89
LERNA	Server	208.96	–	–	518.39	–
	Client	4245.30	107.33	–	–	–
	Committee	392.69	–	3.98	107.11	–
KhPRF-OPSA	Server	–	–	–	573.02	–
	TEE	178.73	–	–	104.16	123.94
	Client	0.37	111.97	–	–	65.44

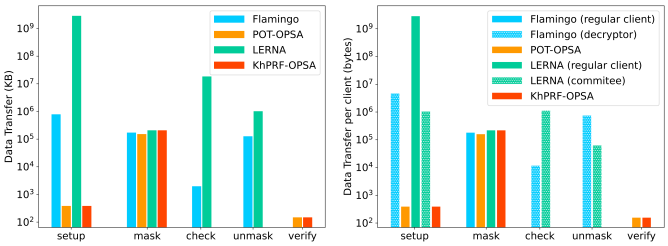
Compared with VerSA [18] and the work proposed by Eltaras et al. [20] (which we call ESLAM), our OPSA-RV has fixed TEE computation time and communication cost.

TABLE III: Computation time and communication (comm.) cost of result verification with $N = 1$K and $\delta = 5\%$.

Model dimension	Scheme	Client (ms)		Server/TEE (ms)	Comm. cost per client (B)
		commit	verify		
20K	VerSA	768.07	6.22	23879.57	160000
	ESLAM	214.49	4.60	21.19	191506
	OPSA-RV	32.91	32.26	125.35	152
40K	VerSA	1526.02	10.18	43921.27	320000
	ESLAM	222.43	10.41	45.71	351506
	OPSA-RV	65.25	64.33	124.89	152
60K	VerSA	2223.32	14.14	64412.10	480000
	ESLAM	224.26	14.18	65.39	511506
	OPSA-RV	98.21	97.18	123.81	152
80K	VerSA	2998.85	18.49	86986.30	640000
	ESLAM	229.63	17.92	86.03	671506
	OPSA-RV	130.02	129.32	121.79	152
100K	VerSA	3769.42	22.45	108853.58	800000
	ESLAM	238.34	23.59	103.32	831506
	OPSA-RV	163.53	163.11	121.52	152

Communication cost. As shown in Fig. 8, we evaluate the communication cost across different stages of a single SA round. During the "setup" phase, our scheme requires only TEE and clients to execute the key agreement protocol and remote attestation, eliminating the need for secret sharing among clients. In the "mask" stage, all schemes must upload masked models, with the communication cost of single masking schemes slightly higher than that of double masking schemes due to the larger value of key-homomorphic PRF. Compared with Flamingo and LERNA, our scheme saves communication costs in the "check" and "unmask" stages with the help of TEE. Furthermore, during the "verify" stage, our communication cost is fixed for the size of a commitment and a signature, significantly lower than that of a global model, which is also provided in Table III.

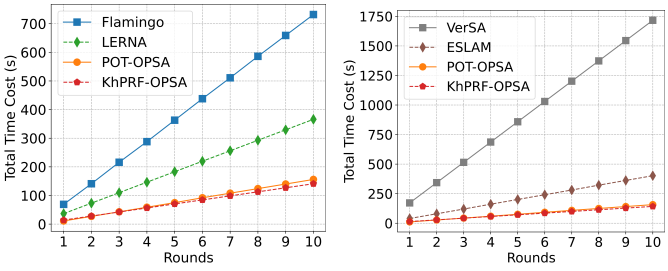
Total time cost. To demonstrate the practicality of our scheme, we evaluate the overall execution time required for the client and server to complete a 10-round SA across all schemes, including the server's wait time, the time for all messages to be delivered and received, and various computation times, but excluding the one-time setup time. As



(a) Server communication cost. (b) Client communication cost.

Fig. 8: Communication cost for different stages in a single round over 1K clients, with $m = 20$K and $\delta = 5\%$.

shown in Fig. 9, our scheme has significant advantages in both verifiable and non-verifiable SA. For SA without verification, our scheme is nearly 5× faster than Flamingo and almost 2× faster than LERNA. When verification is involved, our scheme is nearly 10× faster than VerSA [18] and almost 3× faster than ESLAM [20]. This improvement is expected to be even more significant in scenarios with high network latency, thanks to the one client-server round-trip communication provided by the OPSA framework.



(a) SA without verification. (b) SA with verification.

Fig. 9: End-to-end completion time of 10 rounds of secure aggregation over 1K clients with $m = 20$K and $\delta = 5\%$.

As shown in Table IV, our scheme demonstrates advantages in server-side runtime compared with SEAR [11]. Obviously, the runtime per client is independent of the number of clients, and the OPSA's server-side time cost is 4∼8 times lower than that of SEAR. Furthermore, it also shows that POT-OPSA is better suited for situations with high-dimensional models and fewer dropped clients. Additionally, our scheme enables clients to verify the codes of SA and the server's hardware it is running through remote attestation. Table V presents the total time required for remote attestation between TEE and the clients, considering different numbers of open threads. This process is generally carried out during the setup phase to ensure the integrity of the code executed by Intel SGX.

We further evaluate the memory usage of enclaves and compare our two constructions with SEAR [11]. As shown in Fig. 10, the memory usage of our enclaves is independent of the model dimension and is linearly related to the number of clients. This is because our scheme only requires processing client key seeds within the enclave, while other operations (e.g., model aggregation and unmasking) are performed outside TEE, unlike SEAR. This design significantly

表III。表II：当N=1K、m=20K且δ=5%时，安全聚合每轮各阶段的计算时间。

方案；计划	类型	安全聚合阶段（毫秒）				
		计划	屏蔽	检查	解除屏蔽	核实
红鹳	服务器	40.37	335.75	7.94	24397.08	–
	客户		355.34	–	–	–
	解密器	7143.84		4.44	833.11	–
POT-OPSA	服务器	1192.60			3914.28	–
	TEE		178.64	–	–	13.28 122.60
	客户端	0.89	79.23			64.89
LERNA	服务器	208.96	–	–	518.39	–
	客户	4245.30	107.33	–	–	–
	委员会	392.69	–	3.98	107.11	–
KhPRF-OPSA	服务器	–	–	–	573.02	–
	TEE	178.73	–	–	104.16	123.94
	客户	0.37	111.97	–	–	65.44

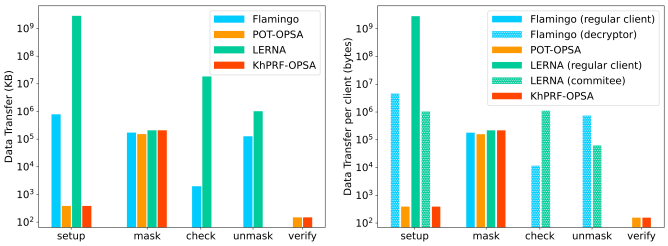
与VerSA[18]以及Eltaras等人提出的方案[20]（我们称之为ESLAM）相比，我们的 OPSA -RV具有固定的TEE计算时间和通信成本。

表III：当N=1000且δ=5%时结果验证的计算时间与通信成本。

模型维度	方案；计划	客户端（毫秒） 犯罪	核实	服务器/TEE (ms)	通信成本 针对每位客户(B)：
20K	VerSA	768.07	6.22	23879.57	160000
	ESLAM	214.49	4.60	21.19	191506
	OPSA-RV	32.91	32.26	125.35	152
40K	VerSA	1526.02	10.18	43921.27	320000
	ESLAM	222.43	10.41	45.71	351506
	OPSA-RV	65.25	64.33	124.89	152
60K	VerSA	2223.32	14.14	64412.10	480000
	ESLAM	224.26	14.18	65.39	511506
	OPSA-RV	98.21	97.18	123.81	152
80K	VerSA	2998.85	18.49	86986.30	640000
	ESLAM	229.63	17.92	86.03	671506
	OPSA-RV	130.02	129.32	121.79	152
100K	VerSA	3769.42	22.45	108853.58	800000
	ESLAM	238.34	23.59	103.32	831506
	OPSA-RV	163.53	163.11	121.52	152

通信成本。如图8所示，我们评估了单次安全协议（SA）轮次各阶段的通信成本。在“设置”阶段，我们的方案仅需TEE和客户端执行密钥协商协议及远程验证流程，无需客户端之间进行秘密共享；在“掩码”阶段，所有方案均需上传经过掩码处理的模型，由于键同态PRF值较大，单次掩码方案的通信成本略高于双次掩码方案；与Flamingo和 LERNA 相比，我们的方案借助TEE技术在“验证”和“解掩码”阶段显著降低了通信成本；此外，在“确认”阶段，我们的通信成本随承诺数据和签名长度的变化而保持固定，远低于全局模型所需的通信成本（详见表III）。

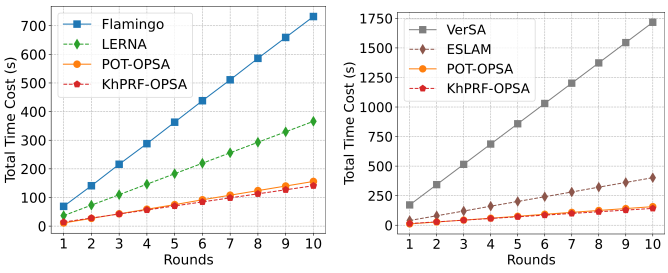
总时间成本。为验证我们方案的实用性，我们评估了客户端与服务器完成全部方案中10轮SA通信所需的总执行时间，该时间包括服务器的等待时间、所有消息的发送与接收时间以及各类计算时间（但不包含一次性设置时间）。



(a) 服务器通信成本 . (b) Client communi猫的贼本 .

图8：在1000个客户端中，单轮通信各阶段的成本对比（m=20000且δ=5%）。

如图9所示，我们的方案在可验证与不可验证的SA场景中均具有显著优势。在无需验证的SA场景下，我们的方案比 LERNA 快近*5* × faster than Flamingo and almost 2 ×；当涉及验证时，其速度比 ESLAM[20]快近*10* × faster than VerSA[18]and almost 3 ×。得益于 OPSA 框架提供的单客户端-服务器往返通信机制，在网络延迟较高的场景中，这一性能提升预计会更加显著。



(a) 未经验证的SA。 (b) 已验证的SA。

图9：在m=20K、δ=5%的条件下，针对1000个客户端完成10轮安全聚合的端到端耗时。

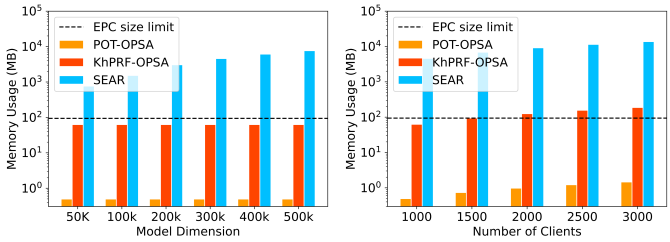
如表IV所示，我们的方案在服务器端运行时性能方面相较于SEAR[11]具有明显优势。显然，每个客户端的运行时间与客户端数量无关，且 OPSA 的服务器端时间成本比SEAR低4至8倍。此外，这还表明POT- OPSA 更适用于高维模型场景及客户端丢弃较少的情况。同时，我们的方案支持客户端通过远程验证机制来确认SA代码及其运行所使用的服务器硬件配置。表V展示了TEE系统与客户端之间进行远程验证所需的总时间（考虑不同线程数的影响）。该过程通常在设置阶段完成，以确保由Intel SGX执行的代码完整性。

我们进一步评估了飞地结构的内存使用情况，并将我们的两种设计方案与SEAR[11]进行对比。如图10所示，我们设计的飞地结构内存消耗与模型维度无关，且与客户端数量呈线性关系。这是因为我们的方案仅需在飞地内部处理客户端密钥种子，而其他操作（如模型聚合和解掩码）均在TEE外部执行，这与SEAR的设计不同。这种架构显

reduces the enclave memory consumption in our schemes. The experimental machine allocates approximately 94 MB for the Enclave Page Cache (EPC). When the memory usage exceeds the EPC size limit, EPC paging occurs, which involves heavy computation costs due to encryption and leads to substantial slowdown in code execution. This further explains the time cost advantage presented in Table IV.

TABLE IV: Time cost comparison with the TEE-based SA scheme with $m = 500$K and $\delta = 0$.

Scheme	Per client runtime (ms)		Server runtime (ms)	
	$N = 500$	$N = 1000$	$N = 500$	$N = 1000$
SEAR	82.79	82.99	33516.24	105756.45
POT-OPSA	438.65	463.08	5245.55	12163.33
KhPRF-OPSA	2578.46	2580.20	8542.36	13647.76



(a) Memory usage vs. model dimension, with 1K clients and $\delta = 0$.

(b) Memory usage vs. number of clients, with $m = 300$K and $\delta = 0$.

Fig. 10: Enclave memory usage for different model dimensions and numbers of clients during an aggregation round.

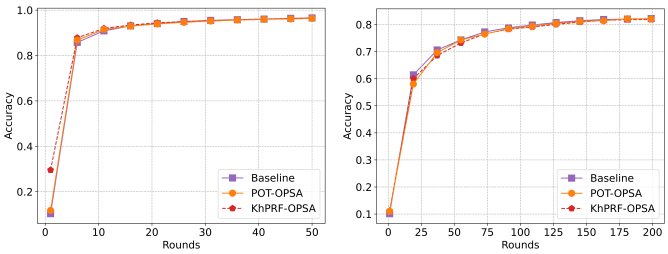
TABLE V: Execution time of remote attestation on the server with multiple threads for different numbers of clients.

Number of threads	Remote attestation (s)		
	$N = 1$K	$N = 2$K	$N = 3$K
20	367.97	737.72	1109.38
30	318.04	634.15	947.06
50	270.78	539.11	809.48

Training accuracy. To evaluate the feasibility of the OPSA framework, we deploy two types of SA protocols in the real model training process. We compare their performance with FedAvg, which serves as the baseline and does not employ any SA scheme. The evaluation is conducted on common datasets MNIST and CIFAR-10, with 100 clients participating in each training session, and its accuracy score is determined by the proportion of correctly predicted samples in the validation set relative to the total number of samples. Similar to Flamingo and LERNA, the primary deviation in our model training process from the baseline lies in the encoding of model floating-point parameters. As shown in Fig. 11, our SA scheme does not impact model performance and accuracy.

VII. Related Work

In this section, we mainly discuss three types of secure aggregation (SA): SA based on Differential Privacy (DP), SA based on Trusted Execution Environment (TEE), and SA based on One-Time Pad (OTP). Moreover, we also discuss some verifiable SA schemes.



(a) MNIST.

(b) CIFAR-10.

Fig. 11: Accuracy comparison under different datasets.

DP-based SA. Differential Privacy (DP) is a concept in data privacy that aims to protect the sensitive information of individuals when analyzing or mining data. The fundamental idea behind DP is to add a controlled amount of noise to the data or query results to prevent the disclosure of specific details about any individual's contribution to the data set.

In DP-based SA schemes [43]–[45], each client adds statistical noise (e.g., Gaussian noise) to hide their local models, but this method introduces significant noise during model aggregation, affecting the accuracy of the results. Moreover, the work [30] has proposed an attack against FL protected with distributed differential privacy and secure aggregation.

TEE-based SA. TEE is a secure and isolated environment within a computer system or a microprocessor, providing a protected execution space for sensitive and critical operations. TEE is designed to prevent data and code from unauthorized access, tampering, and other security threats.

Secure aggregation can also be designed based on TEE [11], [46], [47], i.e., the TEE-equipped server decrypts models received from clients and aggregates them within TEE. However, this method may lead to a notable performance degradation when handling high-dimensional models or a large number of clients, due to the TEE's limited memory. Moreover, TEE still has the risk of being attacked, such as side-channel attacks [48].

OTP-based SA. One-time pad (OTP) is a cryptographic technique that employs a randomly generated secret key of the same length as the message to encrypt and decrypt information. It is a type of symmetric encryption algorithm where the key is used only once for a single message.

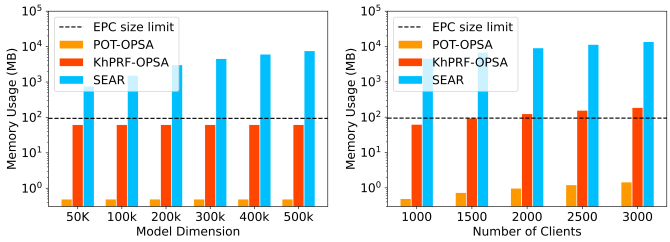
In OTP-based SA schemes, each client generates one-time pads as single or double masks to protect its local models, and the server then can eliminate the sum of masks to recover a original aggregated model. This approach is introduced by SecAgg [4] and the subsequent work, SecAgg+ [5], applies sub-grouping technique to create random connected graphs to reduce the cost of pairwise masks. FLDP [44] and ACORN [7] optimize one-time masks using the additive homomorphism of (Ring) LWE. As the secret keys are one-time in these schemes, each client is required to regenerate and share a new key with others per FL round, leading to a substantial increase in the communication burden.

To achieve a multi-round setting, MicroFedML [8] intro-

著降低了我们方案中的飞地内存占用量。实验设备为飞地页缓存（EPC）分配了约94MB内存空间。当内存使用量超过EPC容量限制时，系统会触发EPC分页机制——该过程因加密运算会产生高昂计算成本，导致代码执行速度大幅下降。这进一步解释了表IV中呈现的时间成本优势。

表IV：基于TEE的SA方案在m=500K且δ=0时的时间成本比较。

方案；计划	按客户端运行时间（毫秒）		服务器运行时间（毫秒）	
	$N=500$	$N=1000$	$N=500$	$N=1000$
SEAR	82.79	82.99	33516.24	105756.45
POT-OPSA	438.65	463.08	5245.55	12163.33
KhPRF-OPSA	2578.46	2580.20	8542.36	13647.76



(a) 内存使用量与模型维度的关系（含1000个客户端且δ=0）。 (b) 内存使用量与客户端数量的关系，其中m=300K且δ=0。

图10：不同模型维度及客户端数量在聚合轮次中的独立内存使用情况。

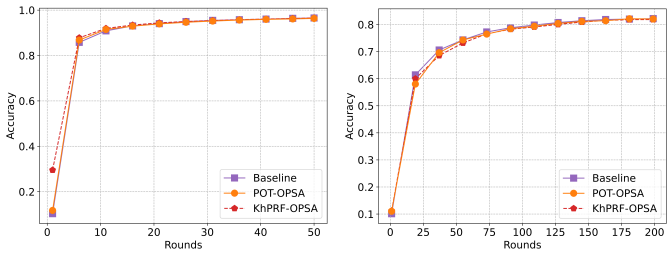
表V：采用多线程服务器对不同数量客户端进行远程认证的执行时间。

线程数量	远程认证（个）		
	$N=1K$	$N=2K$	$N=3K$
20	367.97	737.72	1109.38
30	318.04	634.15	947.06
50	270.78	539.11	809.48

训练准确率。为评估 OPSA 框架的可行性，我们在实际模型训练过程中部署了两种SA协议，并将其性能与作为基准且未采用任何SA方案的FedAvg进行对比。评估在 MNIST 和 CIFAR -10这两个常用数据集上进行，每次训练会涉及100个客户端，其准确率通过验证集中正确预测样本数占总样本数的比例来计算。与Flamingo和 LERNA 类似，我们的模型训练过程与基准方案的主要差异在于模型浮点参数的编码方式。如图11所示，我们的SA方案并未影响模型性能及准确率。

第七章相关工作

在本节中，我们主要讨论三种安全聚合（SA）类型：基于差分隐私（DP）的安全聚合、基于可信执行环境（TEE）的安全聚合以及基于其他基础的安全聚合。



(a) MNIST。 (b) CIFAR-10.

图 11：不同数据集下的准确率比较。

关于一次性密码本（OTP）。此外，我们还讨论了一些可验证的安全认证方案。

基于差分隐私的分析。差分隐私（DP）是数据隐私领域的一个概念，旨在分析或挖掘数据时保护个人的敏感信息。其核心思想是在数据或查询结果中添加受控量的噪声，以防止泄露任何个体对数据集贡献的具体细节。

在基于差分隐私（DP）的协同学习方案[43]–[45]中，每个客户端都会添加统计噪声（例如高斯噪声）以隐藏其本地模型，但该方法会在模型聚合过程中引入显著噪声，从而影响结果的准确性。此外，文献[30]提出了一种针对采用分布式差分隐私和安全聚合机制保护的联邦学习攻击的方法。

基于TEE的安全应用。TEE是计算机系统或微处理器内部的一种安全且隔离的环境，为敏感及关键操作提供受保护的执行空间。其设计旨在防止数据和代码遭受未经授权的访问、篡改及其他安全威胁。

安全聚合也可以基于TEE[11]、[46]、[47]进行设计，即配备TEE的服务器会对从客户端接收的模型进行解密，并在TEE环境中对这些模型进行聚合。然而，由于TEE内存有限，在处理高维模型或大量客户端时，这种方法可能导致显著的性能下降。此外，TEE仍存在被攻击的风险，例如侧信道攻击[48]。

基于一次性密码本（OTP）的安全认证。一次性密码本（OTP）是一种加密技术，它使用与消息长度相同的随机生成密钥来对信息进行加密和解密。这是一种对称加密算法，其密钥仅用于单条消息。

在基于一次性密码本（OTP）的安全协议中，每个客户端会生成单次使用的密钥作为单掩码或双掩码以保护其本地模型；服务器随后可通过合并这些掩码来恢复原始聚合模型。该方法由SecAgg[4]提出，后续研究SecAgg+[5]则采用子分组技术构建随机连通图以降低成对掩码的计算成本。 FLDP[44]与acorn[7]则利用（环）LWE 的加法同态特性优化一次性密钥。由于这些方案中的密钥具有一次性特性，每个客户端在每轮联邦学习中都需要重新生成并与其他节点共享新密钥，从而显著增加通信负担。

为实现多轮协议设计，MicroFedML[8]引入了一次性设

duces a one-time setup phase to reuse secrets. Flamingo [9] designs a multi-round SA protocol integrating threshold decryption with a double masking scheme, while LERNA [10] achieves the same goal integrating flat secret sharing with a key-homomorphic pseudo-random function. Meanwhile, two recent works, OPA [49] and Willow [50], focus on one-shot aggregation, where clients speak only once per aggregation round. Specifically, the former introduces a distributed-key homomorphic PRF, while the latter develops a symmetric encryption scheme that is both key- and message-homomorphic. However, these multi-round SA protocols [9], [10], [49], [50] rely on an additional group, such as decryptors in Flamingo and Willow, or a committee in LERNA and OPA, to improve performance in handling dropout. Even then, their aggregation still require 3 communication round trips. Moreover, none of them consider the verifiability of final aggregation results.

Verifiable SA. In SA protocols, a server provides a list of surviving clients who have successfully uploaded their masked models. The verifiability of the aggregation result, indicating that the result is indeed aggregated according to this list, is achieved by the verifiable SA, which ensures that a malicious server can perform aggregation honestly.

VerifyNet [16], based on a double masking scheme, introduces a homomorphic hash function and non-interactive zero-knowledge proofs (NIZK) to verify the aggregation result. VeriFL [17] builds upon this by replacing NIZK with a commitment mechanism. VerSA [18], derived from a double masking scheme, generates two sets of model masks and employs the message authentication code (MAC) concept for result verification. VOSA [19] designs a single mask scheme under the Decisional Composite Residuosity (DCR) assumption and utilizes NIZK to achieve result verification. Eltaras et al. [20] construct a single mask scheme based on key agreement protocols and also utilize the MAC concept for result verification.

In these verifiable SA schemes, all clients are considered to be semi-honest. VerifyNet [16] and VeriFL [17] further assume that the server is semi-honest but capable of forging the aggregation result. VerSA [18] and ESLAM [20], on the other hand, assume that the server is malicious but does not collude with any client at any stage. This assumption is primarily due to the MAC being computed by the same common factor among all clients, which is unknown to the server. VOSA [19] introduces a semi-honest collector for interaction with the server, but there is no guarantee that the collector is always online. Moreover, the client communication overhead in these schemes [18], [20] is twice that of the original SA schemes, which put huge transmission pressure on clients, especially for high-dimensional models.

VIII. Conclusion

In this paper, we proposed OPSA, an efficient and verifiable multi-round one-pass aggregation via hardware-assisted TEE. By offloading computational tasks outside TEE, OPSA efficiently aggregates larger models and handles more clients within TEE's restricted capacity. Furthermore, we presented KhPRF-OPSA and POT-OPSA, which are built upon novel cryptographic primitives specifically designed for OPSA, and developed OPSA-RV, a method that achieves result verifiability under a weaker security assumption, increasing only $O(1)$ communication overhead per client. Experimental results demonstrate significant improvements in efficiency and dropout resilience. In future work, we aim to extend result verification to address combined malicious server-client attacks.

References

[1] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends® in Machine Learning*, vol. 14, no. 1-2, pp. 1–210, 2021.

[2] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *IEEE Symposium on Security and Privacy*, 2019, pp. 691–706.

[3] A. Hatamizadeh, H. Yin, P. Molchanov, A. Myronenko, W. Li *et al.*, "Do gradient inversion attacks make federated learning unsafe?" *IEEE Transactions on Medical Imaging*, vol. 42, no. 7, 2023.

[4] K. Bonawitz, V. Ivanov, B. Kreuter *et al.*, "Practical secure aggregation for privacy-preserving machine learning," in *ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.

[5] J. H. Bell, K. A. Bonawitz, A. Gascón *et al.*, "Secure single-server aggregation with (poly) logarithmic overhead," in *ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1253–1269.

[6] J. So, B. Güler, and A. S. Avestimehr, "Turbo-aggregate: Breaking the quadratic aggregation barrier in secure federated learning," *IEEE Journal on Selected Areas in Information Theory*, pp. 479–489, 2021.

[7] J. Bell, A. Gascón, T. Lepoint, B. Li, S. Meiklejohn, M. Raykova, and C. Yun, "ACORN: Input validation for secure aggregation," in *32nd USENIX Security Symposium*, 2023, pp. 4805–4822.

[8] Y. Guo, A. Polychroniadou, E. Shi, D. Byrd, and T. Balch, "MicroFedML: Privacy preserving federated learning for small weights," *Cryptology ePrint Archive*, 2022.

[9] Y. Ma, J. Woods, S. Angel *et al.*, "Flamingo: Multi-round single-server secure aggregation with applications to private federated learning," in *IEEE Symposium on Security and Privacy*, 2023, pp. 477–496.

[10] H. Li, H. Lin, A. Polychroniadou, and S. Tessaro, "LERNA: Secure single-server aggregation via key-homomorphic masking," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2023, pp. 302–334.

[11] L. Zhao, J. Jiang, B. Feng *et al.*, "Sear: Secure and efficient aggregation for byzantine-robust federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3329–3342, 2021.

[12] F. Kato, Y. Cao, and M. Yoshikawa, "Olive: Oblivious federated learning on trusted execution environment against the risk of sparsification," *Proceedings of the VLDB Endowment*, pp. 2404–2417, 2023.

[13] L. Fowl, J. Geiping, W. Czaja *et al.*, "Robbing the fed: Directly obtaining private data in federated learning with modified models," in *International Conference on Learning Representations*, 2022.

[14] D. Pasquini, D. Francati, and G. Ateniese, "Eluding secure aggregation in federated learning via model inconsistency," in *ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2429–2443.

[15] J. C. Zhao, A. Sharma, A. R. Elkordy *et al.*, "LOKI: Large-scale data reconstruction attack against federated learning through model manipulation," in *IEEE Symposium on Security and Privacy*, 2024.

[16] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 911–926, 2019.

[17] X. Guo, Z. Liu, J. Li *et al.*, "VeriFL: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1736–1751, 2020.

[18] C. Hahn, H. Kim, M. Kim, and J. Hur, "VerSA: Verifiable secure aggregation for cross-device federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 1, pp. 36–52, 2021.

[19] Y. Wang, A. Zhang, S. Wu, and S. Yu, "VOSA: Verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Transactions on Dependable and Secure Computing*, 2022.

[20] T. Eltaras, F. Sabry, W. Labda, K. Alzoubi, and Q. Malluhi, "Efficient verifiable protocol for privacy-preserving aggregation in federated learning," *IEEE Transactions on Information Forensics and Security*, 2023.

[21] N. C. Will and C. A. Maziero, "Intel software guard extensions applications: A survey," *ACM Computing Surveys*, vol. 55, no. 14s, 2023.

置阶段以复用密钥。Flamingo[9]设计了一种结合阈值解密与双重掩码方案的多轮秘密共享协议；而 LERNA[10]则通过将平面密钥共享与键同态伪随机函数相结合实现了相同目标。与此同时，近期的OPA[49]和Willow[50]两项研究专注于单次聚合机制——客户端在每轮聚合中仅需发言一次：前者引入了分布式密钥同态伪随机函数，后者则开发了兼具密钥同态性和消息同态性的对称加密方案。然而这些多轮秘密共享协议[9]、[10]、[49]、[50]均依赖额外的组结构（如Flamingo和Willow中的解密者，或 LERNA 与 OPA中的委员会）来提升抗丢包性能；即便如此，其聚合过程仍需经历三次通信往返。此外，这些方案均未考虑最终聚合结果的可验证性。

可验证聚合协议（SA）。在SA协议中，服务器会提供一份已成功上传其经过掩码处理模型的存活客户端列表。聚合结果的可验证性（即该结果确实根据此列表进行聚合）通过可验证聚合协议实现，该协议确保恶意服务器也能诚实地执行聚合操作。

VerifyNet[16]基于双重掩码方案，引入了同态哈希函数和非交互式零知识证明（NIZK）来验证聚合结果。VeriFL[17]在此基础上改进，用承诺机制替换了 NIZK 。VerSA[18]源自双重掩码方案，生成两组模型掩码，并采用消息认证码（MAC）概念进行结果验证。 VOSA[19]在决策复合剩余性（DCR）假设下设计了单掩码方案，并利用 NIZK 实现结果验证。Eltaras等人[20]构建了基于密钥协商协议的单掩码方案，并同样运用MAC概念进行结果验证。

在这些可验证的SA方案中，所有客户端均被视为半诚实方。VerifyNet[16]和VeriFL[17]进一步假设服务器也是半诚实的，但能够伪造聚合结果；而VerSA[18]和 ESLAM [20]则认为服务器具有恶意性，但在任何阶段都不会与任何客户端串通——这一假设主要源于MAC值由所有客户端共享的同一公因数计算得出，且该公因数对服务器而言是未知的。 VOSA[19]引入了与服务器交互的半诚实收集器，但无法保证该收集器始终在线运行；此外，这些方案中的客户端通信开销[18]、[20]是原始SA方案的两倍，给客户端（尤其是高维模型场景下的客户端）带来巨大的传输压力。

第八节。结论

本文中，我们提出了 OPSA ——一种基于硬件辅助可信执行环境（TEE）的高效且可验证的多轮单次聚合方案。通过将计算任务卸载至TEE外部， OPSA 能够在TEE有限的资源容量下高效聚合大型模型并支持更多客户端接入。此外，我们提出了基于专为 OPSA 设计的新型密码学原语构建的KhPRF- OPSA 和POT- OPSA，并开发了OPSA -RV方法——该方法仅需在较弱的安全假设下即可实现结果可验证性，且每个客户端的通信开销仅为*O(1)*量级。实验结果表明该方案在效率和抗丢包能力方面均显著提升。未来研究中，我们将致力于扩展结果验证功能以应对联合恶意服务器-客户端攻击场景。

参考文献

[1] P. Kairouz、H. B. McMahan、B. Avent等，《联邦学习领域的进展与开放问题》，*《机器学习基础与趋势》*，第14卷，第1-2期，第1–210页，2021年。

[2] L. Melis、C. Song、E. De Cristofaro 和 V. Shmatikov，《利用协作学习中的非预期特征泄露》，收录于*IEEE 安全与隐私研讨会*，2019年，第691–706页。

[3] A. 哈塔米扎德、H. 尹、P. 莫尔恰诺夫、A. 米罗年科、W. 李等，“梯度反演攻击是否会使联邦学习不安全？”*IEEE医学影像汇刊*，第42卷第7期，2023年。

[4] K. Bonawitz、V. Ivanov、B. Kreuter等，《保护隐私的机器学习实用安全聚合》，收录于*ACM SIGSAC 计算机与通信安全会议*，2017年，第1175–1191页。

[5] J. H. Bell、K. A. Bonawitz、A. Gascʻon等，《具有（多）对数级开销的安全单服务器聚合》，收录于*ACM SIGSAC 计算机与通信安全会议*，2020年，第1253–1269页。

[6] J. So、B. Gⁱler 和 A. S. Avestimehr，《Turbo-aggregate：在安全联邦学习中突破二次聚合障碍》，*《IEEE信息论精选领域期刊》*，第479–489页，2021年。

[7] J. Bell、A. Gascʻon、T. Lepoint、B. Li、S. Meiklejohn、M. Raykova 和 C. Yun，「acorn：安全聚合的输入验证」，载于*第32届 USENIX 安全研讨会*，2023年，第4805–4822页。

[8] 郭宇、A. 波利克罗尼亚杜、E. 史、D. 拜德和T. 巴尔奇，《MicroFedML：适用于小权重场景的隐私保护联邦学习》，*密码学电子预印本档案库*，2022年。

[9] 马勇、伍德斯·J.、安吉尔·S.等人，《Flamingo：适用于私有联邦学习的多轮单服务器安全聚合技术》，收录于*IEEE安全与隐私研讨会*，2023年，第477–496页。

[10] 李浩（H. Li）、林浩（H. Lin）、波利克罗尼亚杜（A. Polychroniadou）与泰萨罗（S. Tessaro）合著，《 LERNA ：基于密钥同态掩蔽的安全单服务器聚合》，载于*密码学与信息安全理论及应用国际会议*，Springer出版社，2023年，第302–334页。

[11] 赵立、江杰、冯波等，《Sear：面向拜占庭容错联邦学习的安全高效聚合》，*《IEEE可靠与安全计算汇刊》*，第19卷第5期，第3329–3342页，2021年。

[12] F. Kato、Y. Cao 和 M. Yoshikawa，《橄榄树：在可信执行环境中针对稀疏化风险的无记忆联邦学习》，*《 VLDB 基金会会议录》*，第2404–2417页，2023年。

[13] L. Fowl、J. Geiping、W. Czaja等，“窃取已提供数据：在改进模型的联邦学习中直接获取私有数据”，载于*国际学习表示会议*，2022年。

[14] D. 帕斯奎尼、D. 弗兰卡蒂与G. 阿特内塞，《通过模型不一致性规避联邦学习中的安全聚合》，载于*ACM SIGSAC 计算机与通信安全会议*，2022年，第2429–2443页。

[15] 赵杰昌（J. C. Zhao）、A. Sharma、A. R. Elkordy等，“LOKI：通过模型操纵对联邦学习发起的大规模数据重构攻击”，收录于*IEEE安全与隐私研讨会*，2024年。

[16] 徐刚、李浩、刘帅、杨科和林旭，《VerifyNet：安全且可验证的联邦学习》，*《IEEE信息取证与安全汇刊》*，第15卷，第911–926页，2019年。

[17] 十、郭X、刘Z、李J等，《VeriFL：面向联邦学习的通信高效且快速可验证聚合方法》，*《IEEE信息取证与安全汇刊》*，第16卷，第1736–1751页，2020年。

[18] C. Hahn、H. Kim、M. Kim 和 J. Hur，《VerSA：跨设备联邦学习中的可验证安全聚合》，*《IEEE可靠与安全计算汇刊》*，第20卷第1期，第36–52页，2021年。

[19] 王Y、张A、吴S与余S，《 VOSA ：可验证且无需感知的隐私保护联邦学习安全聚合》，*《IEEE可靠与安全计算汇刊》*，2022年。

[20] T. Eltaras、F. Sabry、W. Labda、K. Alzoubi 和 Q. Malluhi，《联邦学习中保护隐私聚合的高效可验证协议》，*《IEEE信息取证与安全学报》*，2023年。

[21] N. C. Will 和 C. A. Maziero，《Intel软件保护扩展应用程序：一项调查》，*ACM计算综述*，第55卷第14s期，2023年。

[22] S. Van Schaik, A. Seto, T. Yurek, A. Batori, B. AlBassam, D. Genkin, A. Miller, E. Ronen, Y. Yarom, and C. Garman, "Sok: Sgx. fail: How stuff gets exposed," in *IEEE Symposium on Security and Privacy*, 2024, pp. 4143–4162.

[23] D. Boneh, K. Lewi, H. Montgomery, and A. Raghunathan, "Key homomorphic prfs and their applications," in *Annual Cryptology Conference*. Springer, 2013, pp. 410–428.

[24] A. Banerjee, C. Peikert, and A. Rosen, "Pseudorandom functions and lattices," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2012, pp. 719–737.

[25] S. G. Choi, J. Katz, R. Kumaresan, and C. Cid, "Multi-client non-interactive verifiable computation," in *Theory of Cryptography Conference*. Springer, 2013, pp. 499–518.

[26] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Annual international cryptology conference*. Springer, 1991, pp. 129–140.

[27] R. Pass, E. Shi, and F. Tramer, "Formal abstractions for attested execution secure processors," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2017, pp. 260–289.

[28] J. So, R. E. Ali, B. Güler *et al.*, "Securing secure aggregation: Mitigating multi-round privacy leakage in federated learning," in *AAAI Conference on Artificial Intelligence*, 2023, pp. 9864–9873.

[29] Z. Liu, H.-Y. Lin, and Y. Liu, "Long-term privacy-preserving aggregation with user-dynamics for federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 2398–2412, 2023.

[30] F. Boenisch, A. Dziedzic, R. Schuster *et al.*, "Reconstructing individual data points in federated learning hardened with differential privacy and secure aggregation," in *IEEE European Symposium on Security and Privacy*, 2023, pp. 241–257.

[31] J. Z. Yu, S. Shinde, T. E. Carlson, and P. Saxena, "Elasticlave: An efficient memory model for enclaves," in *31st USENIX Security Symposium*, 2022, pp. 4111–4128.

[32] Y. Xu, J. Pangia, C. Ye, Y. Solihin, and X. Shen, "Data enclave: A data-centric trusted execution environment," in *IEEE International Symposium on High-Performance Computer Architecture*, 2024, pp. 218–232.

[33] A. Ahmad, A. Schultz, B. Lee, and P. Fonseca, "An extensible orchestration and protection framework for confidential cloud computing," in *17th USENIX Symposium on Operating Systems Design and Implementation*, 2023, pp. 173–191.

[34] Y. Bai, P. Li, Y. Huang, M. C. Huang, S. Zhao, L. Zhao, F. Zhang, D. Meng, and R. Hou, "HyperTEE: A decoupled tee architecture with secure enclave management," in *57th IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2024, pp. 105–120.

[35] F. Toffalini, M. Payer, J. Zhou, and L. Cavallaro, "Designing a provenance analysis for sgx enclaves," in *Proceedings of the 38th Annual Computer Security Applications Conference*, 2022, pp. 102–116.

[36] Y. Jia, S. Liu, W. Wang, Y. Chen, Z. Zhai, S. Yan, and Z. He, "HyperEnclave: An open and cross-platform trusted execution environment," in *USENIX Annual Technical Conference*, 2022, pp. 437–454.

[37] J. Protzenko, B. Parno, A. Fromherz, C. Hawblitzel, M. Polubelova, K. Bhargavan, B. Beurdouche, J. Choi, A. Delignat-Lavaud, C. Fournet *et al.*, "Evercrypt: A fast, verified, cross-platform cryptographic provider," in *IEEE Symposium on Security and Privacy*, 2020, pp. 983–1002.

[38] J. Wichelmann, A. Rabich, A. Pätschke, and T. Eisenbarth, "Obelix: Mitigating side-channels through dynamic obfuscation," in *IEEE Symposium on Security and Privacy*, 2024, pp. 189–189.

[39] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Annual Network and Distributed System Security Symposium*, 2021.

[40] M. R. Albrecht, R. Player, and S. Scott, "On the concrete hardness of learning with errors," *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169–203, 2015.

[41] D. Byrd, M. Hybinette, and T. H. Balch, "ABIDES: Towards high-fidelity multi-agent market simulation," in *ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, 2020, pp. 11–22.

[42] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," *arXiv preprint arXiv:1206.6389*, 2012.

[43] N. Agarwal, A. T. Suresh, F. X. X. Yu, S. Kumar, and B. McMahan, "cpSGD: Communication-efficient and differentially-private distributed sgd," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[44] T. Stevens, C. Skalka, C. Vincent *et al.*, "Efficient differentially private secure aggregation for federated learning via hardness of learning with errors," in *31st USENIX Security Symposium*, 2022, pp. 1379–1395.

[45] W.-N. Chen, C. A. C. Choo *et al.*, "The fundamental price of secure aggregation in differentially private federated learning," in *International Conference on Machine Learning*, 2022, pp. 3056–3089.

[46] Y. Zhang, Z. Wang, J. Cao *et al.*, "ShuffleFL: Gradient-preserving federated learning using trusted execution environment," in *ACM International Conference on Computing Frontiers*, 2021, pp. 161–168.

[47] A. P. Kalapaaking, I. Khalil, M. S. Rahman, M. Atiquzzaman *et al.*, "Blockchain-based federated learning with secure aggregation in trusted execution environment for internet-of-things," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 1703–1714, 2022.

[48] C. Liu, A. Chakraborty, N. Chawla, and N. Roggel, "Frequency throttling side-channel attack," in *ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1977–1991.

[49] H. Karthikeyan and A. Polychroniadou, "OPA: one-shot private aggregation with single client interaction and its applications to federated learning," *Cryptology ePrint Archive*, 2024.

[50] J. Bell-Clark, A. Gascón, B. Li, M. Raykova, and P. Schoppmann, "Willow: Secure aggregation with one-shot clients," *Cryptology ePrint Archive*, 2024.

APPENDIX A
PROXY OBLIVIOUS TRANSFER

Here, we review the notion of non-interactive proxy oblivious transfer (POT_2^1) defined by Choi et al. [25]. A POT_2^1 protocol involves three parties: a sender, a chooser, and a proxy. The sender knows two inputs m_0 and m_1, and the chooser holds a choice $b \in \{0,1\}$. At the end of the protocol, the proxy learns m_b, while the two other parties learn nothing. Based on [25], a proxy OT protocol requires both *sender privacy* and *chooser privacy*. *Sender privacy* denotes that the proxy learns only the value of the sender input that corresponds to the chooser's input sequence. *Chooser privacy*, on the other hand, means that the proxy learns no information about the chooser's input sequence.

Now, we extend POT_2^1 to *k-out-of-n proxy oblivious transfer* (POT_n^k). First, we introduce the syntax of our POT_n^k protocol, which consists of the following algorithms:

- $A \leftarrow \mathsf{Send}(r, sk_S, pk_C, \{m_i\}_{i\in[n]})$: in the r-th POT_n^k execution, the sender takes as inputs its private key sk_S, the chooser's public key pk_C, and a plaintext set $\{m_i\}_{i\in[n]}$. It outputs an encoded message set $A = \{ct_i\}_{i\in[n]}$ to be sent to the proxy.
- $B \leftarrow \mathsf{Choose}(r, sk_C, pk_S, \{b_j\}_{j\in[k]})$: in the r-th POT_n^k protocol, the chooser takes as inputs its private key sk_C, the sender's public key pk_S, and a choice set $\{b_j\}_{j\in[k]}$. It outputs an auxiliary set $B = \{aux_j\}_{j\in[k]}$ to be sent to the proxy.
- $M \leftarrow \mathsf{Proxy}(r, pk_S, pk_C, A, B)$: in the r-th POT_n^k protocol, the proxy takes as inputs the sender's public key pk_S, the chooser's public key pk_C, the sender message A, and the chooser message B. It outputs a selected plaintext set $M = \{m_{b_j}\}_{j\in[k]}$.

APPENDIX B
SECURITY PROOFS

B.1 Proof of Theorem 1

Proof. The proof of dropout resilience is rather simple: in each round of the training phase, δ fraction of clients can drop out, i.e., $|\mathcal{D}| \leq \lfloor \delta N \rfloor$. Since $|\mathcal{V}| \geq \lceil (1-\delta)N \rceil$ (Fig. 3 and Fig. 4), the TEE instance can always help the server to reveal the secret keys. Therefore, the dropout resilience

[22] S. Van Schaik、A. Seto、T. Yurek、A. Batori、B. AlBassam、D. Genkin、A. Miller、E. Ronen、Y. Yarom 和 C. Garman，《Sok：Sgx.fail：数据如何被暴露》，收录于*IEEE安全与隐私研讨会*，2024年，第4143–4162页。

[23] D. Boneh、K. Lewi、H. Montgomery 和 A. Raghunathan，《关键同态密码学函数及其应用》，载于*年度密码学会议*，Springer出版社，2013年，第410–428页。

[24] A. Banerjee、C. Peikert 和 A. Rosen，《伪随机函数与格》，载于*《密码学技术理论与应用年度国际会议论文集》*。Springer出版社，2012年，第719–737页。

[25] S. G. Choi、J. Katz、R. Kumaresan 和 C. Cid，《多客户端非交互式可验证计算》，载于*密码学理论会议*，Springer出版社，2013年，第499–518页。

[26] T. P. Pedersen，《非交互式且基于信息论的安全可验证秘密共享》，载于*国际密码学年会论文集*，Springer出版社，1991年，第129–140页。

[27] R. Pass、E. Shi 和 F. Tramer，《用于经验证执行安全处理器的形式化抽象》，载于*《密码学技术理论与应用国际年会》*，Springer出版社，2017年，第260–289页。

[28] J. So、R. E. Ali、B. G¨uler等，《保障安全聚合：缓解联邦学习中的多轮隐私泄露》，收录于*AAAI人工智能会议*，2023年，第9864–9873页。

[29] 刘志、林浩阳和刘宇，《基于用户动态的长期隐私保护聚合方法在联邦学习中的应用》，*《IEEE信息取证与安全汇刊》*，第18卷，第2398–2412页，2023年。

[30] F. Boenisch、A. Dziedzic、R. Schuster等，“基于差分隐私与安全聚合技术加固的联邦学习中个体数据点重构”，载于*IEEE欧洲安全与隐私研讨会*，2023年，第241–257页。

[31] J. Z. Yu、S. Shinde、T. E. Carlson 和 P. Saxena，《Elasticlave：一种高效的飞地内存模型》，收录于*第31届 USENIX 安全研讨会*，2022年，第4111–4128页。

[32] 徐宇、庞佳、叶超、索利欣和沈旭，《数据飞地：一种以数据为中心的可信执行环境》，载于*IEEE高性能计算机架构国际研讨会*，2024年，第218–232页。

[33] A. Ahmad、A. Schultz、B. Lee 和 P. Fonseca，《面向机密云计算的可扩展编排与保护框架》，载于*第17届 USENIX 操作系统设计与实现研讨会*，2023年，第173–191页。

[34] 白宇、李鹏、黄宇、黄明昌、赵硕、赵磊、张峰、孟丹和侯瑞，《HyperTEE：一种具备安全飞地管理功能的解耦T型架构》，载于*第57届IEEE/ACM微架构国际研讨会*，IEEE，2024年，第105–120页。

[35] F. Toffalini、M. Payer、J. Zhou 和 L. Cavallaro，《为SGX飞地设计来源分析》，载于*第38届计算机安全应用年会论文集*，2022年，第102–116页。

[36] 贾勇、刘爽、王伟、陈宇、翟志、严松和何哲，《HyperEnclave：一个开放且跨平台的可信执行环境》，载于*USENIX 年度技术大会*，2022年，第437–454页。

[37] J. Protzenko、B. Parno、A. Fromherz、C. Hawblitzel、M. Polubelova、K. Bhargavan、B. Beurdouche、J. Choi、A. Delignat-Lavaud、C. Fournet等，《Evercrypt：一款快速、经过验证的跨平台加密解决方案》，收录于*IEEE安全与隐私研讨会*，2020年，第983–1002页。

[38] J. 维切尔曼，A. 拉比希，A. PäTscheke与T. Eisenbarth合著的《Obelix：通过动态混淆技术缓解侧信道攻击》，收录于*IEEE安全与隐私研讨会*（2024年），第189–189页。

[39] X. Cao、M. Fang、J. Liu 和 N. Z. Gong，“FLTrust：通过信任引导实现拜占庭鲁棒联邦学习”，收录于*年度网络与分布式系统安全研讨会*，2021年。

[40] M. R. 阿尔布雷希特、R. 普莱尔与S. 斯科特，《论含错误学习的具体难度》，*《数学密码学杂志》*，第9卷第3期，第169–203页，2015年。

[41] D. Byrd、M. Hybinette 和 T. H. Balch，《abides：迈向高保真多智能体市场仿真》，收录于*ACM SIGSIM 高级离散仿真原理会议*，2020年，第11–22页。

[42] B. Biggio、B. Nelson 和 P. Laskov，“针对支持向量机的中毒攻击”，*arXiv 预印本 arXiv:1206.6389*，2012年。

[43] N. 阿加瓦尔、A. T. 苏雷什、F. X. X. 余、S. 库马尔和B. 麦克马汉，《cpSGD：通信高效且差分隐私的分布式SGD算法》，*《神经信息处理系统进展》*，第31卷，2018年。

[44] T. Stevens、C. Skalka、C. Vincent等，《基于学习误差难度的联邦学习高效差分隐私安全聚合》，收录于*第31届 USENIX 安全研讨会*（2022年），第1379–1395页。

[45] 陈文南（W.-N. Chen）、周昌安（C. A. C. Choo）等，《差分隐私联邦学习中安全聚合的基本成本》，载于*国际机器学习会议*，2022年，第3056–3089页。

[46] 张宇、王泽、曹杰等，“ShuffleFL：基于可信执行环境的梯度保持联邦学习”，收录于*ACM国际计算前沿会议*，2021年，第161–168页。

[47] A. P. 卡拉帕金、I. 哈里勒、M. S. 拉赫曼、M. 阿提库扎曼等，《基于区块链的可信执行环境中具备安全聚合功能的物联网联邦学习》，*《IEEE工业信息学汇刊》*，第19卷第2期，第1703–1714页，2022年。

[48] C. 刘、A. 查克拉博蒂、N. 查瓦拉和N. 罗格尔，《频率限速侧信道攻击》，收录于*ACM SIGSAC 计算机与通信安全会议*，2022年，第1977–1991页。

[49] H. Karthikeyan 和 A. Polychroniadou，《OPA：基于单客户端交互的一次性私有聚合及其在联邦学习中的应用》，*密码学电子预印本档案库*，2024年。

[50] J. Bell-Clark、A. Gascʻon、B. Li、M. Raykova 和 P. Schoppmann，《Willow：采用一次性客户端的安全数据聚合》，*密码学电子预印本档案库*，2024年。

附录A
代理盲传

在此，我们回顾由Choi等人[25]定义的非交互式代理盲传（POT12）概念。POT12协议涉及三方参与者：发送方、选择者以及代理。发送方知晓两个输入m_0和m_1，而选择者持有一个选择值$b \in \{0,1\}$。协议结束时，代理可获知m_b，而另外两方则一无所获。[25]指出，代理盲传协议需要同时满足*发送方隐私*与*选择者隐私*。*发送方隐私*指代理仅能获取与选择者输入序列对应的发送方输入值；*选择者隐私*则意味着代理无法获取任何关于选择者输入序列的信息。

现在，我们将POT12扩展为*k-out-of-n代理无关传输*（POTkn）。首先，我们介绍POTkn协议的语法结构，该协议包含以下算法：

- $A \leftarrow$发送（$r, sk_S, pk_C, \{m_i\}_{i\in[n]}$）：在第$r$次 POTkn 执行过程中，发送方以自身私钥$sk_S$、选择者的公钥$pk_C$以及明文集合$\{m_i\}_{i\in[n]}$作为输入，并输出编码消息集$A=\{ct_i\}_{i\in[n]}$用于发送给代理。
- $B \leftarrow$选择（$r, sk_C, pk_S, \{b_j\}_{j\in[k]}$）：在第$r$次 POTkn 协议中，选择者以自身私钥$sk_C$、发送方公钥$pk_S$以及选择集 $\{b_j\}_{j\in[k]}$ 作为输入；并输出辅助集$B=\{aux_j\}_{j\in[k]}$ 供发送至代理节点。
- $M \leftarrow$代理函数（r, pk_S, pk_C, A, B）：在第r个 POTkn 协议中，该代理函数以发送方公钥pk_S、选择方公钥pk_C、发送方消息A及选择方消息B作为输入，并输出选定的明文集合$M=\{m_{b_j}\}_{j\in[k]}$。

附录B
安全证明

B.1 定理1的证明

*证明。*掉线抗性性的证明相当简单：在训练阶段的每一轮中，δ比例的客户端可能退出，即 $|D| \leq q\delta Nq$. Since $|V| \geq \mu(1-\delta)N\mu$（图3和图4），TEE实例始终能帮助服务器揭示密钥。因此，掉线抗性特性直接源自基于公式1实现

property directly follows that of the underlying key agreement protocol instantiated with Equation 1. For each client $u \in \mathcal{U}$, there are two primary approaches to mask individual models.

For KhPRF-OPSA, client u masks its individual model as $\mathbf{y}_u^{(r)} = \Delta \cdot \mathbf{x}_u^{(r)} + \mathbf{mask}_u$ where $\mathbf{mask}_u = \mathrm{KF}(F_1(k_{u,e}, r), \mathbf{b})$. Here, F_1 is a secure pseudo-random function invoked on a round $r \in [1, R]$ and a shared key $k_{u,e}$ between client u and TEE instance M_e, and KF is a 1-almost key-homomorphic PRF invoked on a temporary key $\mathbf{k}_u = F_1(k_{u,e}, r)$ and a public parameter $\mathbf{b}$. For the surviving list $\mathcal{V}^{(r)}$, TEE computes $\mathbf{k}_{\mathcal{V}^{(r)}} = \sum_{u \in \mathcal{V}^{(r)}} F_1(k_{e,u}, r)$. Now, given $k_{u,e} = k_{e,u}$, we can observe that $\sum_{u \in \mathcal{V}^{(r)}} \mathrm{KF}(F_1(k_{u,e}, r), \mathbf{b}) = \mathrm{KF}(\mathbf{k}_{\mathcal{V}^{(r)}}, \mathbf{b}) + \mathbf{e}$ where $|\mathbf{e}_i| < \Delta$ for all $i \in [1, m]$, and obtain an aggregated model as follows:

$$\begin{aligned}
\sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} &= \frac{1}{\Delta} \left(\sum_{u \in \mathcal{V}^{(r)}} \mathbf{y}_u^{(r)} - \mathrm{KF}(\mathbf{k}_{\mathcal{V}^{(r)}}, \mathbf{b}) \right) \\
&= \frac{1}{\Delta} \left(\sum_{u \in \mathcal{V}^{(r)}} \Delta \cdot \mathbf{x}_u^{(r)} + \sum_{u \in \mathcal{V}^{(r)}} \mathbf{mask}_u - \mathrm{KF}(\mathbf{k}_{\mathcal{V}^{(r)}}, \mathbf{b}) \right) \\
&= \frac{1}{\Delta} \left(\Delta \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + \mathbf{e} \right) \\
&= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + \mathbf{0}
\end{aligned}$$

For POT-OPSA, each client $u \in \mathcal{U}$ masks its individual model as $\mathbf{y}_u^{(r)} = \mathbf{x}_u^{(r)} + \mathbf{mask}_u$ where $\mathbf{mask}_u = G(\widetilde{k}_u) + \sum_{v \in \mathcal{U}, u<v} G(F_0(k_{u,v}, r)) - \sum_{v \in \mathcal{U}, u>v} G(F_0(k_{u,v}, r))$. Here, F_0 is a secure pseudo-random function invoked on a round $r \in [1, R]$ and a shared key $k_{u,v}$ between client u and its neighbor v, and G is a secure pseudo-random generator invoked on a temporary individual key $\widetilde{k}_u \xleftarrow{\$} \{0,1\}^\lambda$ or a temporary pairwise key $\widetilde{k}_{u,v} = F_0(k_{u,v}, r)$. For the client list $\mathcal{U} = \mathcal{V}^{(r)} \cup \mathcal{D}^{(r)}$, the TEE instance helps the server to recover $\{\widetilde{k}_{v,u}\}_{v \in \mathcal{D}^{(r)}, u \in \mathcal{V}^{(r)}}$ for dropped clients and $\{\widetilde{k}_u\}_{u \in \mathcal{V}^{(r)}}$ for surviving clients. Then, the server computes $\mathbf{mask}_{\mathcal{U}}^{(r)} = \sum_{u \in \mathcal{V}^{(r)}, v \in \mathcal{D}^{(r)}} \left(G(\widetilde{k}_u) + \sum_{u<v} G(\widetilde{k}_{u,v}) - \sum_{u>v} G(\widetilde{k}_{u,v}) \right)$. Now, given $k_{u,v} = k_{v,u}$ for all client u and v, we can observe that $\sum_{u,v \in \mathcal{L}, u<v} G(\widetilde{k}_{u,v}) = \sum_{u,v \in \mathcal{L}, u>v} G(\widetilde{k}_{u,v})$ for all $u, v \in \mathcal{L}$ and $\mathcal{L} \subseteq \mathcal{U}$, and obtain an aggregated model as follows:

$$\begin{aligned}
\sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} &= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{y}_u - \mathbf{mask}_{\mathcal{U}}^{(r)} \\
&= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + \sum_{u \in \mathcal{V}^{(r)}, v \in \mathcal{U}} \mathbf{mask}_u - \mathbf{mask}_{\mathcal{U}}^{(r)} \\
&= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + \sum_{u,v \in \mathcal{V}^{(r)}} \left(\sum_{u<v} G(\widetilde{k}_{u,v}) - \sum_{u>v} G(\widetilde{k}_{u,v}) \right) \\
&= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + 0
\end{aligned}$$

This is satisfied when the number of surviving clients with $|\mathcal{V}| \geq \lceil (1-\delta)N \rceil$ and $(1-\delta) \geq \alpha$, where α is defined in Definition 4. □

B.2 Proof of Theorem 2

Proof. The proof is based on a standard hybrid argument using the simulator SIM. For any round $r \in [1, R]$, all input vectors $\mathcal{X}^{(r)}$, and all sets of corrupted clients $\mathcal{C} \subseteq \mathcal{U}$ of size $|\mathcal{C}| \leq \lfloor \gamma N \rfloor$, we prove the output of SIM is computationally indistinguishable from the view of a malicious adversary $\mathcal{A}$, i.e., $\mathsf{REAL}_{\mathcal{C},r}^{\mathcal{U},S,R}(\mathcal{A}, \{\mathbf{x}_u\}_{u \in \mathcal{U} \setminus \mathcal{C}}) \approx_c \mathsf{SIM}_{\mathcal{C},r}^{\mathcal{U},S,R,F_{\alpha,\{\mathbf{x}_u^{(r)}\}_{u \in \mathcal{U} \setminus \mathcal{C}}}}(\mathcal{A})$ where $F_{\alpha,\mathcal{X}^{(r)}}$ is α-summation ideal functionality defined in Definition 4.

Following the MicroFedML [8] and Flamingo [9], we first define the behavior of our simulator SIM for KhPRF-OPSA:

- Setup phase:
  (1) Each honest client u and TEE instance M_e follow the protocol description in one-time setup.
  (2) For each corrupt client v, TEE instance M_e computes and stores $k_{e,v} = \mathsf{KA}(sk_e, pk_v)$.
  (3) For each honest clients u, the simulator chooses $k_a \xleftarrow{\$} \{0,1\}^\lambda$ and sets $k_{u,e}^* = k_{e,u}^* = k_a$.
- The r-th round of the Training phase:
  (1) Each honest client u randomly chooses a random vector $\mathbf{y}_u^*$ and sends it the server.
  (2) The server S first adds the client u to the surviving list $\mathcal{V}^{(r)}$, and then the simulator makes a query to $F_{\alpha,\mathcal{X}^{(r)}}(\mathcal{V}^{(r)} \setminus \mathcal{C})$ to get $\mathbf{x}^{(r)}$.
  (3) For all honest clients $u \in \mathcal{V}^{(r)} \setminus \mathcal{C}$, the simulator randomly samples $\mathbf{x}_u^* \xleftarrow{\$} \mathcal{M}$ under the restriction $\sum_{u \in \mathcal{V}^{(r)} \setminus \mathcal{C}} \mathbf{x}_u^* = \mathbf{x}^{(r)}$, and then computes $\mathbf{mask}_u^* = \mathbf{y}_u^* - \mathbf{x}_u^*$ by programming the random oracle to set the output of KhPRF. The simulator sets $\mathbf{k}_u^* = \mathrm{PRF}(k_{u,e}^*, r)$ and samples a random vector $\mathbf{b}$ such that $\mathbf{mask}_u^* = \mathcal{R}_{\mathsf{KhPRF}}(\mathbf{k}_u^*, \mathbf{b})$ for each $u \in \mathcal{V}^{(r)} \setminus \mathcal{C}$, reveals masked keys $\mathbf{k}_{\mathcal{V}^{(r)}}^* = \sum_{u \in \mathcal{V}^{(r)}} \mathbf{k}_u^*$ for TEE instance M_e, and computes $\mathbf{mask}_{\mathcal{V}^{(r)}}^* = \mathcal{R}_{\mathsf{KhPRF}}(\mathbf{k}_{\mathcal{V}}^*, \mathbf{b})$ for the server.

Below, we present a series of subsequent hybrids to the real execution REAL of our KhPRF-OPSA protocol.

Hyb_0: This random variable is the joint views of all parties in $\mathcal{C}$ in the real execution.

Hyb_1: In this hybrid, the simulator, which knows all secrets of honest parties in every round, runs the execution of the KhPRF-OPSA protocol with $\mathcal{A}$. Therefore, the view of the adversary is the same as the previous hybrid.

Hyb_2: This hybrid is identical to Hyb_1 except that the simulator substitutes the shared key $k_{u,e} = \mathsf{KA}(sk_u, pk_e)$ between honest client u and TEE M_e with a random key $k_a \xleftarrow{\$} \{0,1\}^\lambda$. The 2ODH assumption [4] guarantees this hybrid is indistinguishable from the previous one.

Hyb_3: This hybrid is same as Hyb_2 except that the values of $\mathbf{y}_u$ computed by SIM on behalf of the honest client u and sent to the server, are substituted with uniformly sampled values $\mathbf{y}_u^*$. Since $\mathcal{A}$ does not know the secrets of honest parities and $\mathbf{y}_u$ has the same distribution as $\mathbf{y}_u^*$, the view of the adversary in this hybrid is statistically indistinguishable from the previous one.

的基础密钥协商协议。对于每个客户端$u \in U$，存在两种主要方法来屏蔽单个模型。

对于KhPRF- OPSA，客户端u将其个体模型封装为$\mathbf{y}_u^{(r)} = \Delta \cdot \mathbf{x}_u^{(r)} + \mathbf{mask}_u$，其中$\mathbf{mask}_u = \mathrm{KF}(F_1(k_{u,e}, r), \mathbf{b})$。此处$F_1$是作用于轮次$r \in [1, R]$以及客户端$u$与TEE实例$M_e$之间共享密钥$k_{u,e}$上的安全伪随机函数；KF 则是作用于临时密钥$\mathbf{k}_u = F_1(k_{u,e}, r)$和公共参数$\mathbf{b}$上的1-几乎键同态PRF。对于存活列表$V^{(r)}$，TEE计算$\mathbf{k}_{\mathcal{V}^{(r)}} = \sum_{u \in V^{(r)}} F_1(k_{e,u}, r)$。现在，给定$k_{u,e} = k_{e,u}$，我们可以观察到$\sum_{u \in V^{(r)}} \mathrm{KF}(F_1(k_{u,e}, r), \mathbf{b}) = \mathrm{KF}(\mathbf{k}_{\mathcal{V}^{(r)}}, \mathbf{b}) + \mathbf{e}$，其中$|\mathbf{e}_i| < \Delta$对所有$i \in [1, m]$成立，并可得到如下聚合模型：

$$\begin{aligned}\sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} &= \frac{1}{\Delta}\left(\sum_{u \in \mathcal{V}^{(r)}} \mathbf{y}_u^{(r)} - \mathrm{KF}(\mathbf{k}_{\mathcal{V}^{(r)}}, \mathbf{b})\right) \\ &= \frac{1}{\Delta}\left(\sum_{u \in \mathcal{V}^{(r)}} \Delta \cdot \mathbf{x}_u^{(r)} + \sum_{u \in \mathcal{V}^{(r)}} \mathbf{mask}_u - \mathrm{KF}(\mathbf{k}_{\mathcal{V}^{(r)}}, \mathbf{b})\right) \\ &= \frac{1}{\Delta}\left(\Delta \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + \mathbf{e}\right) \\ &= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + \mathbf{0}\end{aligned}$$

对于POT- OPSA，每个客户端$u \in U$将其个体模型$\sim$vindividual伪装为$\mathbf{y}_u^{(r)} = \mathbf{x}_u^{(r)} + \mathbf{mask}_u$，其中$\mathbf{mask}_u = G(k_u) + \sum_{v \in U, u<v} G(F_0(k_{u,v}, r)) - \sum_{v \in U, u>v} G(F_0(k_{u,v}, r))$。此处$F_0$是作用于轮次上的安全伪随机函数。$r \in [1, R]$，且客户端$u$与其邻居$v$之间共享密钥$k_{u,v}$；$G$是一个安全伪随机生成器，作用于临时的$\sim$个体密钥$\sim k_u$上。$\sim \xleftarrow{\$} \{0,1\}^\lambda$或临时成对密钥$k_{u,v} = F_0(k_{u,v}, r)$。对于客户端列表$\sim U = V^{(r)} \setminus D^{(r)}$，TEE实例可帮助服务$\sim$器为丢失的客户端恢复$\{k_{v,u}\}_{v \in \mathcal{D}(r), u \in V(r)}$，并为存活的客户端恢复$\{k_u\}_{u \in \mathcal{V}(r)}$。随后，服务器计算$\mathbf{mask}_{\mathcal{U}}^{(r)} = \sum_{u \in V(r), v \in D(r)} (G(\sim k_u) + \sum_{u<v} G(\sim ku, v) - \sum_{u>v} G(\sim ku, v))$。现在，给定$k_{u,\sim} = k_{v,\sim}$以及所有客户端$u$和$v$对应的$u$值，我们可以观察到$\sum_{u,v \in L, u<v} G(k_{u,v}) = \sum_{u,v \in L, u>v} G(ku, v)$ for all $u, v \in L$ and $L \subseteq U$，并得到如下聚合模型：

$$\begin{aligned}\sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} &= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{y}_u - \mathbf{mask}_{\mathcal{U}}^{(r)} \\ &= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + \sum_{u \in \mathcal{V}^{(r)}, v \in \mathcal{U}} \mathbf{mask}_u - \mathbf{mask}_{\mathcal{U}}^{(r)} \\ &= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + \sum_{u,v \in \mathcal{V}^{(r)}} \left(\sum_{u<v} G(\tilde{k}_{u,v}) - \sum_{u>v} G(\tilde{k}_{u,v})\right) \\ &= \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)} + 0\end{aligned}$$

当存活客户端的数量满足$|V| \geq \gamma(1-\delta)N\gamma$ and $(1-\delta) \geq \alpha$时，该条件即得到满足，其中α的定义见定义4。 □

B.2 定理2的证明

证明。该证明基于使用模拟器SIM的标准混合论证方法。对于任意轮次$r \in [1, R]$、所有输入向量$X^{(r)}$以及所有被篡改客户端集合$C \subseteq U$ of size$|C| \leq q\gamma N q$，我们证明SIM的输出在计算上与恶意攻击者A的观测结果无法区分，即：$\mathrm{REAL}^{U,S,R,F}_{C, r^{(r)}, \mathrm{aux}^{(r)}}(A, \{\mathbf{x}_u\}_{u \in U \setminus C}) \approx_c \mathrm{SIM}_{\mathcal{C}, r^{(r)}, \mathrm{aux}^{(r)}}(A)$，其中$F_{\alpha,\mathcal{X}}(t)$为定义在定义4中的$\alpha$-求和理想函数性。

继MicroFedML[8]和Flamingo[9]之后，我们首先定义了针对KhPRF- OPSA 的模拟器SIM的行为：

- 设置阶段：
  (1) 每个诚实客户端u和TEE实例M_e均遵循以下规则：一次性设置中的协议描述。
  (2) 对于每个被篡改的客户端v，TEE实例M_e会计算并存储$k_{e,v} = \mathsf{KA}(sk_e, pkv)$。
  (3) 对于每个诚实客户端u，模拟器会选择$k_a \xleftarrow{\$} \{0,1\}^\lambda$并设定$k\gamma u_{,e} = k\gamma e_{,u} = k_a$。
- 培训阶段的第r轮：
  (1) 每位诚实客户端u随机选择一个随机向量$\mathbf{y}_{\gamma u}$并将其发送给服务器。
  (2) 服务器S首先将客户端u添加到存活列表$V^{(r)}$中，随后模拟器向$F_{\alpha,\mathcal{X}^{(r)}}(V^{(r)} \setminus C)$发起查询以获取$\mathbf{x}^{(r)}$。
  (3) 对于所有诚实客户端$u \in V^{(r)} \setminus C$，模拟器会随机采样$\mathbf{x}_u \xleftarrow{\$} M$在重新排列的情况下约束$\sum_{u \in V^{(r)} \setminus \mathcal{C}} \mathbf{x}_{\gamma u} = \mathbf{x}^{(r)}$，然后通过编程随机预言机设置KhPRF 的输出结果，计算出**掩码**$\gamma u = \mathbf{y}_{\gamma u} - \mathbf{x}_{\gamma u}$。该模拟器设定$\mathbf{k}_{\gamma u} = \mathrm{PRF}(k\gamma u, e, r)$，并为每个$u \in V^{(r)} \setminus C$采样一个随机向量$\mathbf{b}$，使得$\mathbf{mask}_{\gamma u} = \mathcal{R}_{\mathsf{KhPRF}}(\mathbf{k}_{\gamma u}, \mathbf{b})$；从而为TEE实例$M_e$揭示掩码密钥$\mathbf{k}_{\gamma V^{(r)}} = \sum_{u \in V^{(r)}} \mathbf{k}_{\gamma u}$，并为服务器计算$\mathbf{mask}_{\gamma V^{(r)}} = \mathcal{R}_{\mathsf{KhPRF}}(\mathbf{k}_{\gamma V}, \mathbf{b})$。

下文我们将介绍基于KhPRF- OPSA 协议实际实现REAL的一系列后续混合方案。

Hyb_0：该随机变量表示实际执行过程中C中所有参与方的联合观测结果。

Hyb_1：在此混合模型中，模拟器在每轮均掌握诚实参与方的所有信息，并与A共同执行KhPRF- OPSA 协议。因此，攻击者的观察视角与前述混合模型完全一致。

Hyb_2：该混合体与Hyb_1完全相同，区别在于模拟器将诚实客户端u与TEE M_e之间共享的密钥$k_{u,e} = \mathsf{KA}(sk_u, pke)$替换为随机密钥$k_a \xleftarrow{\$} \{0,1\}^\lambda$。2ODH假设[4]保证该混合模型与先前模型无法区分。

Hyb_3：该混合模型与Hyb_2相同，区别在于由SIM代表诚实客户端u计算并发送至服务器的$\mathbf{y}_u$值被替换为均匀采样的值$\mathbf{y}_{\gamma u}$。由于A并不知晓诚实方的机密信息且$\mathbf{y}_u$与$\mathbf{y}_{\gamma u}$具有相同的分布特性，因此该混合模型中攻击者的观测结果在统计上与先前模型无法区分。

Hyb_4: This hybrid is same as Hyb_3 except that the simulator replaces the revealed keys outputted by TEE instance M_e with $\mathbf{k}^*_{\mathcal{V}}$ and computes the aggregated mask $\mathbf{mask}^*_{\mathcal{V}}$ (as in Behavior 3 of Training phase of SIM) by querying the $F_{\alpha,\mathcal{X}^{(r)}}(\mathcal{V}^{(r)} \setminus \mathcal{C})$ and utilizing a random oracle $\mathcal{R}_{\mathsf{KhPRF}}$. Since $\mathcal{A}$ does not know the secrets of honest parities and the aggregated mask in SIM have the same distribution as the one in REAL, this hybrid is indistinguishable from the previous one.

Hyb_5: This hybrid is same as Hyb_4 except one modification: SIM sets the output of the ideal functionality $F_{\alpha,\mathcal{X}^{(r)}}(\mathcal{V}^{(r)} \setminus \mathcal{C})$ (as in Behavior 2 of Training phase of SIM) as the aggregated model $\mathbf{x}^{(r)}$. Note that the ideal functionality will not return $\perp$ in this case. This modification does not change the view seen by the adversary, and hence this hybrid is indistinguishable from the previous one.

After completing these hybrids, SIM simulates REAL without relying on inputs from the honest parties, indicating that the joint view of $\mathcal{A}$ in the real execution (Hyb_0) is computationally indistinguishable from the output of SIM (Hyb_5).

Here, we redefine the behavior of our simulator SIM for POT-OPSA:

- Setup phase:
  (1) Each honest client u and TEE instance M_e follow the protocol description in one-time setup.
  (2) For each corrupt client v, an honest client u computes and stores $k_{u,v} = \mathsf{KA}(sk_u, pk_v)$, and TEE instance M_e computes and stores $k_{e,v} = \mathsf{KA}(sk_e, pk_v)$.
  (3) For each honest clients u, the simulator chooses $k_a \xleftarrow{\$} \{0,1\}^\lambda$ and sets $k^*_{u,e} = k^*_{e,u} = k_a$. For each pair of honest clients u, v, the simulator chooses $k_b \xleftarrow{\$} \{0,1\}^\lambda$ and sets $k^*_{u,v} = k^*_{v,u} = k_b$.
- The r-th round of the Training phase:
  (1) Each honest client u randomly chooses a random vector $\mathbf{y}^*_u$ and sends it the server. For each corrupt client v, Each honest client u randomly chooses $ct^*_{u,v}$ as a ciphertext and sends it to the server.
  (2) The server S first adds the client u to the surviving list $\mathcal{V}^{(r)}$, and then the simulator makes a query to $F_{\alpha,\mathcal{X}^{(r)}}(\mathcal{V}^{(r)} \setminus \mathcal{C})$ to get $\mathbf{x}^{(r)}$.
  (3) For all honest clients $u \in \mathcal{V}^{(r)} \setminus \mathcal{C}$, the simulator randomly samples $\mathbf{x}^*_u \xleftarrow{\$} \mathcal{M}$ under the restriction $\sum_{u \in \mathcal{V}^{(r)} \setminus \mathcal{C}} \mathbf{x}^*_u = \mathbf{x}^{(r)}$, and then computes $\mathbf{mask}^*_u = \mathbf{y}^*_u - \mathbf{x}^*_u$ by programming the random oracle to set the output of PRG. The simulator sets $\{\widetilde{\mathbf{k}}^*_{u,v}\}_{v \in \mathcal{U}} = \{\mathcal{R}_{\mathsf{PRG}}(\widetilde{k}^*_{u,v})\}_{v \in \mathcal{U}}$ where $\{\widetilde{k}^*_{u,v}\}_{v \in \mathcal{U}} = \{\mathrm{PRF}(k^*_{u,v}, r)\}_{v \in \mathcal{U}}$, and samples a random $\widetilde{k}^*_{u}$ to set $\widetilde{\mathbf{k}}^*_u = \mathcal{R}_{\mathsf{PRG}}(\widetilde{k}^*_u)$ such that $\mathbf{mask}^*_u = \widetilde{\mathbf{k}}^*_u + \sum_{u<v} \widetilde{\mathbf{k}}^*_{u,v} - \sum_{u>v} \widetilde{\mathbf{k}}^*_{u,v}$ for each $u \in \mathcal{V}^{(r)} \setminus \mathcal{C}$ and $v \in \mathcal{U}$. The simulator computes $\{\widetilde{\mathbf{k}}^*_u, \widetilde{\mathbf{k}}^*_{u,v}\}_{u \in \mathcal{V}^{(r)} \setminus \mathcal{C}, v \in \mathcal{D}^{(r)}}$ for M_e, and computes $\mathbf{mask}^*_{\mathcal{U}} = \sum \widetilde{\mathbf{k}}^*_u + \sum \sum_{u<v} \widetilde{\mathbf{k}}^*_{u,v} - \sum \sum_{u>v} \widetilde{\mathbf{k}}^*_{u,v}$ where $u \in \mathcal{V}^{(r)} \setminus \mathcal{C}$ and $v \in \mathcal{D}^{(r)}$ for the server.

Below, we present a series of subsequent hybrids to the real execution REAL of our POT-OPSA protocol.

Hyb_0: This random variable is the joint views of all parties in $\mathcal{C}$ in the real execution.

Hyb_1: In this hybrid, the simulator, which knows all secrets of honest parties in every round, runs the execution of the POT-OPSA protocol with $\mathcal{A}$. Therefore, the view of the adversary is the same as the previous hybrid.

Hyb_2: This hybrid is identical to Hyb_1 except that the simulator substitutes the shared key $k_{u,e} = \mathsf{KA}(sk_u, pk_e)$ between honest client u and TEE M_e with a random key $k_a \xleftarrow{\$} \{0,1\}^\lambda$, and replaces the shared key $k_{u,v} = \mathsf{KA}(sk_u, pk_v)$ between honest client u and v with a random key $k_b \xleftarrow{\$} \{0,1\}^\lambda$. The 2ODH assumption [4] guarantees this hybrid is indistinguishable from the previous one.

Hyb_3: This hybrid is same as Hyb_2 except that the values of $(\mathbf{y}_u, \{ct_{u,v}\}_{v \in \mathcal{U}})$ computed by SIM on behalf of the honest client u and sent to the server, are substituted with uniformly sampled values $(\mathbf{y}^*_u, \{ct^*_{u,v}\}_{v \in \mathcal{U}})$ at random. Since $\mathcal{A}$ does not know the secrets of honest parities and $(\mathbf{y}_u, \{ct_{u,v}\}_v)$ has the same distribution as $(\mathbf{y}^*_u, \{ct^*_{u,v}\}_v)$, the view of the adversary in this hybrid is statistically indistinguishable from the previous one.

Hyb_4: This hybrid is same as Hyb_3 except that the simulator replaces the revealed keys outputted by TEE instance M_e with $\{\widetilde{\mathbf{k}}^*_u, \widetilde{\mathbf{k}}^*_{u,v}\}_{u,v}$ and computes the aggregated mask $\mathbf{mask}^*_{\mathcal{U}}$ (as in Step 3 of Training phase behavior of SIM) by querying the $F_{\alpha,\mathcal{X}^{(r)}}(\mathcal{V}^{(r)} \setminus \mathcal{C})$ and utilizing a random oracle $\mathcal{R}_{\mathsf{PRG}}$. Since $\mathcal{A}$ does not know the secrets of honest parities and the aggregated mask in SIM have the same distribution as the one in REAL, this hybrid is indistinguishable from the previous one.

Hyb_5: This hybrid is same as Hyb_4 except one modification: SIM sets the output of the ideal functionality $F_{\alpha,\mathcal{X}^{(r)}}(\mathcal{V}^{(r)} \setminus \mathcal{C})$ (as in Step 2 of Training phase behavior of SIM) as the aggregated model $\mathbf{x}^{(r)}$. Note that the ideal functionality will not return $\perp$ in this case. This modification does not change the view seen by the adversary, and hence this hybrid is indistinguishable from the previous one.

After completing these hybrids, SIM simulates REAL without relying on inputs from the honest parties, indicating that the joint view of $\mathcal{A}$ in the real execution (Hyb_0) is computationally indistinguishable from the output of SIM (Hyb_5).

For KhPRF-OPSA and POT-OPSA, SIM can simulate REAL without knowledge of the inputs from the honest parties, indicating the security of our OPSA framework in the malicious setting. Note that our malicious setting includes the semi-honest setting, thereby extending the scope of our security proof to encompass the latter.

□

B.3 Proof of Theorem 3

Proof. We first prove the security of individual models in the construction instantiated with Construction 1 or 2 providing result verification (subsection IV-C), following the proof of Theorem 2. As aforementioned, we assume that all clients are semi-honest.

Hyb_4：该混合模型与Hyb_3相同，区别在于模拟器用$\mathbf{k}_{rV}$替换了TEE实例M_e输出的公开密钥，并通过查询$F_{\alpha,\mathcal{X}}(r)(V^{(r)}\backslash C)$并利用随机预言机$\mathcal{R}_{\mathsf{KhPRF}}$，计算出聚合掩码$\mathbf{mask}_{rV}$（如SIM训练阶段行为3所述）。由于$A$并不知晓诚实校验的秘密值，且SIM中的聚合掩码分布与REAL中的分布一致，因此该混合模型与前一种模型无法区分。

Hyb_5：该混合模型与Hyb_4相同，仅存在一项修改：SIM将理想功能的输出$F_{\alpha,\mathcal{X}}(r)(V^{(r)}\backslash C)$（如SIM训练阶段行为2所示）设为聚合模型$\mathbf{x}^{(r)}$。需注意在此情况下理想功能不会返回$rv$。此修改不会改变攻击者所观察到的视图，因此该混合模型与前一种模型无法区分。

在完成这些混合模型后，SIM无需依赖诚实参与方的输入即可模拟REAL，这表明A在真实执行过程中的联合视图（Hyb_0）在计算上与SIM的输出结果（Hyb_5）无法区分。

在此，我们重新定义了仿真器SIM的行为。

POT-OPSA:

- 设置阶段：
  (1) 每个诚实客户端u与TEE实例M_e均需遵循一次性配置中的协议描述。
  (2) 对于每个腐败客户端v，诚实客户端u会计算并存储$k_{u,v}$=$\mathsf{KA}(sk_u, pkv)$；而TEE实例M_e则会计算并存储$k_{e,v}$=$\mathsf{KA}(sk_e, pkv)$。
  (3) 对于每位诚实客户端u，模拟器均会进行选择。$k_a \xleftarrow{\$} \{0,1\}^\lambda$并令$k_{ru}, e=k_{rv}, u=k_a$。对于对于每一对诚实客户端$u$，$v$，模拟器会选择$k_b \xleftarrow{\$} \{0,1\}^\lambda$并设$k_{ru,v}=k_{rv,u}=k_b$。
- 培训阶段的第r轮：
  (1) 每个诚实客户端u随机选择一个随机向量$\mathbf{y}_{ru}$并将其发送给服务器。对于每个恶意客户端v，每个诚实客户端u随机选择$ct_{ru,v}$作为密文并将其发送给服务器。
  (2) 服务器S首先将客户端u添加到存活列表$V^{(r)}$中，随后模拟器向$F_{\alpha,\mathcal{X}^{(r)}}(V^{(r)}\backslash C)$发起查询以获取$\mathbf{x}^{(r)}$。
  (3) 对于所有诚实客户端$u \in V^{(r)}\backslash C$，模拟器会随机采样$\mathbf{x}^* \xleftarrow{\$} M$在重新排列的情况下约束$\sum_{u \in V^{(r)}\backslash C} \mathbf{x}_{ru}=\mathbf{x}^{(r)}$，然后通过编程随机预言机使$\mathsf{Prg}$的输出为$\mathbf{mask}_{ru}=\mathbf{y}_{ru}-\mathbf{x}_{ru}$。模拟器或集合$\{\mathbf{k}_{ru,v}\}_{v\in\mathcal{U}}=\{\mathcal{R}_{\mathsf{PRG}}(k_{ru}, v)\}_{v\in\mathcal{U}}$ where $\{k_{ru,v}\}_{v\in\mathcal{U}}=\{\mathsf{PRF}_{(k_{ru,v}}, r)\}_{v\in\mathcal{U}}$，会随机抽取一个$\mathsf{d}$个样本$\tilde{k}_{ru}$来确定$\mathbf{k}_{ru}=\mathcal{R}_{\mathsf{PRG}}(k_u^*)$，使得对于每个$\tilde{u} \in \tilde{V}^{(r)}\backslash C$ and $v \in U$，有$\mathbf{mask}_{ru}=\mathbf{k}_{ru}+\sum_{u<v}\mathbf{k}_{ru,v}-\sum_{u>v}\mathbf{k}_{ru,v}$。该模拟器计算出$\{\mathbf{k}_{ru}, \mathbf{k}_{ru,v}\}_{u\in\mathcal{V}}(r)\backslash C, v \in D(r)$对于$M_e$，并计算掩码$\mathbf{mask}_{rU}=\sum \mathbf{k}_{ru}+\sum\sum_{u<v_k}\mathbf{k}_{ru,v}-\sum\sum_{u>v_k}\mathbf{k}_{ru,v}$其中$u \in V^{(r)}\backslash C$ and $v \in D^{(r)}$对于服务器。

下文我们将介绍我们POT- OPSA 协议中实际执行模块REAL的一系列后续混合版本。

Hyb_0：该随机变量表示实际执行过程中C中所有参与方的联合视图。

Hyb_1：在此混合模型中，模拟器在每轮均掌握诚实参与方的所有信息，并与A共同执行POT- OPSA 协议。因此，攻击者的观察视角与前述混合模型完全一致。

Hyb_2：该混合体与Hyb_1完全相同，区别在于模拟器将诚实客户端u与TEE M_e之间共享的密钥$k_{u,e}$=$\mathsf{KA}(sk_u, pke)$替换为随机密钥k_a。$\xleftarrow{\$} \{0,1\}^\lambda$，并将诚实客户端$u$与$v$之间的共享密钥$k_{u,v}$=$\mathsf{KA}(sk_u, pkv)$ 替换为随机密钥$k_b \xleftarrow{\$} \{0,1\}^\lambda$。2ODH假设[4]保证该混合模型与先前模型无法区分。

Hyb_3：该混合模型与Hyb_2相同，区别在于由SIM代表诚实客户端u计算并发送至服务器的值（$\mathbf{y}_u, \{ct_{u,v}\}_{v\in\mathcal{U}}$）被随机替换为均匀采样值（$\mathbf{y}_{ru}, \{ct_{ru,v}\}_{v\in\mathcal{U}}$）。由于$A$并不知晓诚实方的机密信息，且（$\mathbf{y}_u$, $\{ct_{u,v}\}_v$）与（$\mathbf{y}_{ru}, \{ct_{ru,v}\}_v$）具有相同的分布特性，因此攻击者在此混合模型中的观测结果在统计上与先前模型无法区分。

Hyb_4：该混合模型与Hyb_3相同，区别在于模拟器用$\{\mathbf{k}_{ru}, \mathbf{k}_{ru,v}\}_{u,v}$替换了TEE实例$M_e$输出的揭示密钥，并通过查询$F_{\alpha,\mathcal{X}}(r)(V^{(r)}\backslash C)$ 并利用随机预言机$\mathcal{R}_{\mathsf{PRG}}$来计算聚合掩码$\mathbf{mask}_{rU}$（如SIM训练阶段行为步骤3所示）。由于$A$并不知晓诚实校验的秘密且SIM中的聚合掩码分布与REAL中的分布一致，因此该混合模型与前一种模型无法区分。

Hyb_5：该混合模型与Hyb_4相同，仅存在一项修改：SIM将理想功能的输出$F_{\alpha,\mathcal{X}}(r)(V^{(r)}\backslash C)$（如SIM训练阶段行为步骤2所示）设为聚合模型$\mathbf{x}^{(r)}$。需注意在此情况下理想功能不会返回$rv$。此项修改不会改变攻击者所观察到的视图，因此该混合模型与前一种模型无法区分。

在完成这些混合模型后，SIM无需依赖诚实参与方的输入即可模拟REAL，这表明A在真实执行过程中的联合视图（Hyb_0）在计算上与SIM的输出结果（Hyb_5）无法区分。

对于KhPRF- OPSA 和POT- OPSA，SIM能够在无需知晓诚实方输入信息的情况下模拟REAL协议，这证明了我们 OPSA 框架在恶意攻击场景下的安全性。需注意，我们的恶意攻击场景涵盖了半诚实场景，从而将安全证明的适用范围扩展至后者。

□

B.3 定理3的证明

*证明。*我们首先按照定理2的证明方法，证明在采用构造1或构造2并提供结果验证的情况下（子节IV-C），各独立模型的安全性。如前所述，我们假设所有客户端均为半诚实参与者。

Now, we define how our simulator SIM behaves during result verification of KhPRF-OPSA or POT-OPSA. In fact, SIM behaves similarly to that in in the proof of Theorem 2, with the following new additions and modifications:

- The r-th round of the Training phase:
  (1) Each honest client u samples com_u^* at random. It then computes a signature $\sigma_u^* = \mathsf{Sign}(sk_u, \mathsf{com}_u^*)$ and sends $(\mathsf{com}_u^*, \sigma_u^*)$ to the server.
  (3) For all honest clients $u \in \mathcal{V}^{(r)} \setminus \mathcal{C}$, the simulator samples $\mathsf{rand}_u^* = \mathsf{PRF}(k_{u,e}^*, 0||r)$ such that $\mathsf{com}_u^* = \mathsf{Commit}(\mathbf{x}_u^*, \mathsf{rand}_u^*)$. For TEE instance M_e, the simulator computes $\mathsf{rand}^* = \sum_u \mathsf{rand}_u^*$ and $\mathsf{com}^* = \prod_u \mathsf{com}_u^*$ for all $u \in \mathcal{V}^{(r)} \setminus \mathcal{C}$ and generates a signature $\sigma_e^* := \mathsf{Sign}(sk_e, \mathsf{com}^*||\mathsf{rand}^*||r)$. The server sends $(\mathbf{x}^r, \mathsf{rand}^*, \sigma_e^*)$ to all clients.

Below, we present a series of subsequent hybrids to the real execution REAL of our KhPRF-OPSA protocol with result verification.

Hyb_0: This random variable is the joint views of all parties in $\mathcal{C}$ in the real execution.

Hyb_1: In this hybrid, the simulator, which knows all secrets of honest parties in every round, runs the execution of the KhPRF-OPSA protocol with result verification with $\mathcal{A}$. Therefore, the view of the adversary is the same as the previous hybrid.

Hyb_2: This hybrid is identical to Hyb_1 except that the simulator substitutes the shared key $k_{u,e} = \mathsf{KA}(sk_u, pk_e)$ between honest client u and TEE M_e with a random key $k_a \xleftarrow{\$} \{0,1\}^\lambda$. The 2ODH assumption [4] guarantees this hybrid is indistinguishable from the previous one.

Hyb_3: This hybrid is same as Hyb_2 except that the values of $(\mathbf{y}_u, \mathsf{com}_u, \sigma_u)$ computed by SIM on behalf of the honest client u and sent to the server, are substituted with uniformly sampled values $(\mathbf{y}_u^*, \mathsf{com}_u^*, \sigma_u^*)$. Since $\mathcal{A}$ does not know the secrets of honest parities and $(\mathbf{y}_u, \mathsf{com}_u, \sigma_u)$ has the same distribution as $(\mathbf{y}_u^*, \mathsf{com}_u^*, \sigma_u^*)$, the view of the adversary in this hybrid is statistically indistinguishable from the previous one.

Hyb_4: This hybrid is same as Hyb_3 except that the simulator replaces the output of TEE instance M_e with $(\mathbf{k}_{\mathcal{V}}^*, \mathsf{com}^*, \mathsf{rand}^*, \sigma_e^*)$ and computes the aggregated mask $\mathbf{mask}_{\mathcal{V}}^*$ (as in Behavior 3 of Training phase of SIM) by querying the $F_{\alpha,\mathcal{X}^{(r)}}(\mathcal{V}^{(r)} \setminus \mathcal{C})$ and utilizing a random oracle $\mathcal{R}_{\mathsf{KhPRF}}$. Since $\mathcal{A}$ does not know the secrets of honest parities and the aggregated mask in SIM have the same distribution as the one in REAL, this hybrid is indistinguishable from the previous one. Note that the indistinguishability of this hybrid relies on the hiding property of the underlying vector Pedersen commitment scheme and the security property of the signature schemes.

Hyb_5: This hybrid is same as Hyb_4 except one modification: SIM sets the output of the ideal functionality $F_{\alpha,\mathcal{X}^{(r)}}(\mathcal{V}^{(r)} \setminus \mathcal{C})$ (as in Behavior 2 of Training phase of SIM) as the aggregated model $\mathbf{x}^{(r)}$. The server sends $(\mathbf{x}^r, \mathsf{rand}^*, \sigma_e^*)$ to all clients. Note that the ideal functionality will not return $\perp$ in this case. This modification does not change the view seen by the adversary, and hence this hybrid is indistinguishable from the previous one.

After completing these hybrids, SIM simulates REAL without relying on inputs from the honest parties, and we can see that the joint view of $\mathcal{A}$ in the real execution (Hyb_0) is computationally indistinguishable from the output of SIM (Hyb_5).

Here, we omit the hybrids to the real execution REAL of our POT-OPSA protocol with result verification, as they undergo the same modifications as described above.

Next, we give the proof for result authenticity against a malicious server. At the end of round r, the server sends $(\mathbf{x}^{(r)}, \mathsf{com}^{(r)}, \mathsf{rand}^{(r)}, \sigma_e^{(r)})$ to all clients. Here, $\mathbf{x}^{(r)} = \sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u^{(r)}$ represents the sum of honest survivors (i.e, $\mathbf{z}_{\mathcal{H}}$), $\mathsf{com}^{(r)} = \mathsf{Commit}(\mathbf{x}^{(r)}, \mathsf{rand}^{(r)})$ denotes a valid commitment with a random number $\mathsf{rand}^{(r)}$, and $\sigma_e^{(r)} := \mathsf{Sign}(sk_e, \mathsf{com}^{(r)}||\mathsf{rand}^{(r)}||r)$ is a signature generated by TEE instance M_e. To forge the aggregation output, $\mathcal{A}$ can either tamper with an individual model from a specific client or generate random vectors as output results. Without loss of generality, let $\mathbf{z}_{\mathcal{A}} = \mathbf{z}_{\mathcal{H}} + \widetilde{\mathbf{x}_{\mathcal{A}}} + \widetilde{\mathbf{e}}$ be a tampered global model, where $\widetilde{\mathbf{x}_{\mathcal{A}}}$ denotes a tampered individual model and $\widetilde{\mathbf{e}}$ is an error vector introduced by the malicious server. There are two situations where a semi-honest client might accept $\mathbf{z}_{\mathcal{A}}$: one happens if the server can generate $\mathbf{z}_{\mathcal{A}}$ based on the same $\mathsf{com}^{(r)}$ and $\mathsf{rand}^{(r)}$, and the other happens if the server can forge the signature of TEE instance M_e for the new $\widetilde{\mathsf{com}}^{(r)}$ and $\widetilde{\mathsf{rand}}^{(r)}$ corresponding to $\mathbf{z}_{\mathcal{A}}$. Clearly, the former contradicts the binding property of vector Pedersen commitment scheme, and the latter violates the EUF-CMA secure signature scheme.

Therefore, for KhPRF-OPSA and POT-OPSA, we prove the security and result authenticity of our OPSA framework against the malicious server. □

现在，我们明确说明我们的模拟器SIM在KhPRF- OPSA 或POT- OPSA 结果验证过程中的运行机制。实际上，SIM的运作方式与定理2证明中所述类似，但增加了以下新功能并进行了相应调整：

- 培训阶段的第r轮：
  (1) 每位诚实客户端u会随机采样商品v_u，随后计算签名σ_{v_u}= Sign(sk_u，com v_u)，并将（com v_u，σ_{v_u}）发送至服务器。
  (3) 对于所有诚实客户端$u \in V^{(r)}\backslash C$，模拟器会生成随机数rand v_u= PRF(k_{v_u}，e，0||r)，使得com v_u= Commit($\mathbf{x}_{v_u}$，rand v_u)。对于TEE实例M_e，模拟器计算rand *=$\sum_u$ rand $_{v_u}$且com *= $\Pi_u$com v_u（针对所有$u \in V^{(r)}\backslash C$），并生成签名$\sigma_{v_e}$:=Sign($sk_e$，com *||rand *||$r$)。服务器将（$\mathbf{x}^r$，rand *，$\sigma_{v_e}$）发送给所有客户端。

下文我们展示了基于KhPRF- OPSA 协议实际执行流程REAL的一系列后续混合方案，并附有结果验证。

Hyb_0：该随机变量表示实际执行过程中C中所有参与方的联合观测结果。

Hyb_1：在此混合模型中，模拟器在每轮均掌握诚实参与方的所有信息，并与A共同执行KhPRF- OPSA 协议并验证结果。因此，攻击者的观察视角与前述混合模型完全一致。

Hyb_2：该混合体与Hyb_1完全相同，区别在于模拟器将诚实客户端u与TEE M_e之间共享的密钥$k_{u,e}$=KA(sk_u，pke)替换为随机密钥$k_a \xleftarrow{\$} \{0,1\}^\lambda$。2ODH假设[4]保证该混合模型与先前模型无法区分。

Hyb_3：该混合模型与Hyb_2相同，区别在于由SIM代表诚实客户端u计算并发送至服务器的$(\mathbf{y}_u, \mathsf{com}_u, \sigma_u)$值，被替换为均匀采样的$(\mathbf{y}_{v_u}, \mathsf{com}_{v_u}, \sigma_{v_u})$值。由于$A$并不知晓诚实方的机密信息，且$(\mathbf{y}_u, \mathsf{com}_u, \sigma_u)$与$(\mathbf{y}_{v_u}, \mathsf{com}_{v_u}, \sigma_{v_u})$具有相同的分布特性，因此攻击者在此混合模型中的观测结果在统计上与先前模型无法区分。

Hyb_4：该混合方案与Hyb_3相同，区别在于模拟器用（$\mathbf{k}_{v_V}$，com *，rand *，σ_{v_e}）替换TEE实例M_e的输出，并通过查询$F_{\alpha,\mathcal{X}}(r)(V^{(r)}\backslash C)$并利用随机预言机$\mathcal{R}_{\mathsf{KhPRF}}$来计算聚合掩码$\mathbf{mask}_{v_V}$（如SIM训练阶段行为3所述）。由于$A$并不知晓诚实校验位的秘密信息，且SIM中的聚合掩码分布与REAL中的分布一致，因此该混合方案与前一种方案无法区分。需注意，这种不可区分性依赖于底层向量Pedersen承诺方案的隐藏特性以及签名方案的安全特性。

Hyb_5：该混合模型与Hyb_4相同，仅存在一项修改：SIM将理想功能的输出$F_{\alpha,\mathcal{X}}(r)(V^{(r)}\backslash C)$（如SIM训练阶段行为2所述）设为聚合模型$\mathbf{x}^{(r)}$。服务器向所有客户端发送($\mathbf{x}^r$，rand *，$\sigma_{v_e}$)。需注意在此情况下理想功能不会返回$v$。此修改不会改变攻击者所观察到的视图，因此该混合模型与前一种模型无法区分。

在完成这些混合模型后，SIM无需依赖诚实参与方的输入即可模拟REAL；我们可以看到，在实际执行过程中A的联合视图（Hyb_0）在计算上与SIM的输出结果（Hyb_5）无法区分。

在此，我们省略了POT- OPSA 协议中包含结果验证的真实执行流程REAL中的混合步骤，因为这些步骤均经历了上述相同的修改。

接下来，我们给出针对恶意服务器的结果真实性证明。在第r轮结束时，服务器向所有客户端发送（$\mathbf{x}^{(r)}$，com$^{(r)}$，rand$^{(r)}$，$\sigma_e{}^{(r)}$）。其中，$\mathbf{x}^{(r)}$=$\sum_{u \in \mathcal{V}^{(r)}} \mathbf{x}_u{}^{(r)}$表示诚实存活节点的总和（即$\mathbf{z}_{\mathcal{H}}$）；com$^{(r)}$= Commit($\mathbf{x}^{(r)}$，rand($r$))表示带有随机数rand$^{(r)}$的有效承诺；而$\sigma_e{}^{(r)}$:=Sign($sk_e$，com$^{(r)}$||rand$^{(r)}$||$r$)是由TEE实例$M_e$生成的签名。为伪造聚合输出，$A$可篡改特定客户端的单个模型或生成随机向量作为输出结果。不失一般性，设$\mathbf{z}_{\mathcal{A}} = \mathbf{z}_{\mathcal{H}} + \mathbf{xA} + \tilde{\mathbf{e}}$这是一个被篡改的全局模型，其中$\mathbf{xA}$表示一个被篡改的个体模型；$\tilde{\mathbf{e}}$是由恶意服务器引入的错误向量。半诚实客户端可能接受$\mathbf{z}_{\mathcal{A}}$的情况有两种：一种是当服务器能够基于相同的公钥$^{(r)}$和随机数$^{(r)}$生成$\mathbf{z}_{\mathcal{A}}$时；另一种是当服务器能够针对新的公钥$^{(r)}$以及与$\mathbf{z}_{\mathcal{A}}$对应的随机数，伪造trh)e signature of TEE instance M_e时。显然，前者违背了Pedersen向量承诺方案的绑定特性，后者则违反了 EUF -CMA安全签名方案的要求。因此，对于KhPRF- OPSA 和POT- OPSA，我们证明了我们的 OPSA 框架在面对恶意服务器攻击时的安全性及结果真实性。 □